How many minutes is Leeds? Mapping Local Accessibility and Spatial Inequality in Daily Life?

Preparing Notebook

In [ ]:
```
# Preparing notebook
import requests
import pandas as pd
import geopandas as gpd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from shapely.geometry import Point
import osmnx as ox # to download form OSM
from libpysal.weights import Queen # to population weight centreids
from matplotlib.patches import Patch # to have multiple legends in
import contextily as cx # to add basemap
from matplotlib_scalebar.scalebar import ScaleBar # to add scalebar
from sklearn.cluster import DBSCAN # DBSCAN clustering
import sklearn.metrics as metrics # to calculate Silhouette Score
from tqdm import tqdm
import networkx as nx
from matplotlib.gridspec import GridSpec  # for nested map layout
from shapely.geometry import LineString # to recalculate lenghtes i
import pyproj
from sklearn.preprocessing import StandardScaler # for z score
from sklearn.preprocessing import MinMaxScaler # for min-max normal
import matplotlib.patches as mpatches
from matplotlib.cm import get_cmap
from scipy.stats import pearsonr # correlation
from mgwr.gwr import GWR # GWR model
from mgwr.sel_bw import Sel_BW # for selecting bandwidth
import statsmodels.api as sm # QWR residuals
import libpysal
from libpysal.weights import Queen # Queen Neighbours
from esda.moran import Moran # Moran's I
from splot.esda import moran_scatterplot, lisa_cluster, plot_local_
import esda
```

```
/Users/naiemegolzari/Library/Python/3.9/lib/python/site-packages/url
lib3/__init__.py:35: NotOpenSSLWarning: urllib3 v2 only supports Ope
nSSL 1.1.1+, currently the 'ssl' module is compiled with 'LibreSSL
2.8.3'. See: https://github.com/urllib3/urllib3/issues/3020
  warnings.warn(
```

Data Collection, Cleaning and Wrangling

Leeds Boundaries

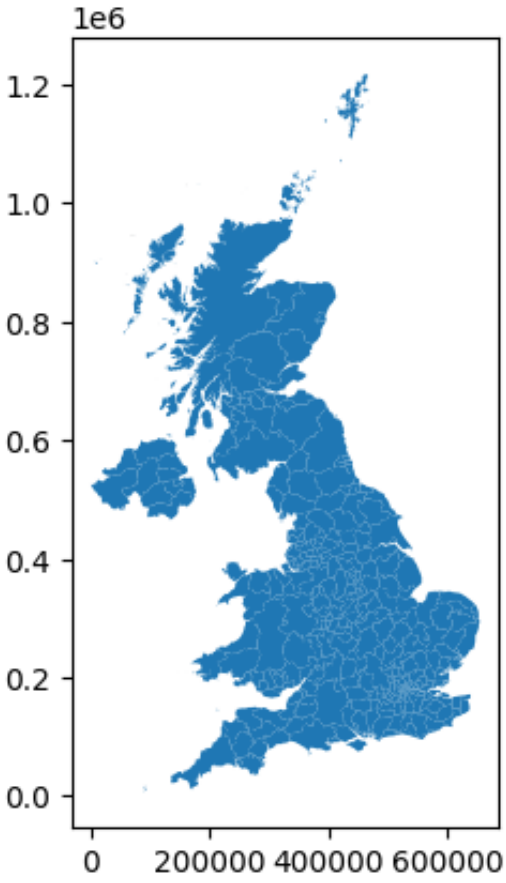
In [2]:
```
LocalBou= gpd.read_file('./Local_Authority_Districts_December_2022_
```

In [3]:
```
LocalBou.crs
```

Out[3]:
```
<Projected CRS: EPSG:27700>
Name: OSGB36 / British National Grid
Axis Info [cartesian]:
- E[east]: Easting (metre)
- N[north]: Northing (metre)
Area of Use:
- name: United Kingdom (UK) - offshore to boundary of UKCS within
49°45'N to 61°N and 9°W to 2°E; onshore Great Britain (England, Wa
les and Scotland). Isle of Man onshore.
- bounds: (-9.01, 49.75, 2.01, 61.01)
Coordinate Operation:
- name: British National Grid
- method: Transverse Mercator
Datum: Ordnance Survey of Great Britain 1936
- Ellipsoid: Airy 1830
- Prime Meridian: Greenwich
```

In [4]:
```
LocalBou.plot()
```

Out[4]:
```
<Axes: >
```



In [5]:
```
LocalBou.head()
```

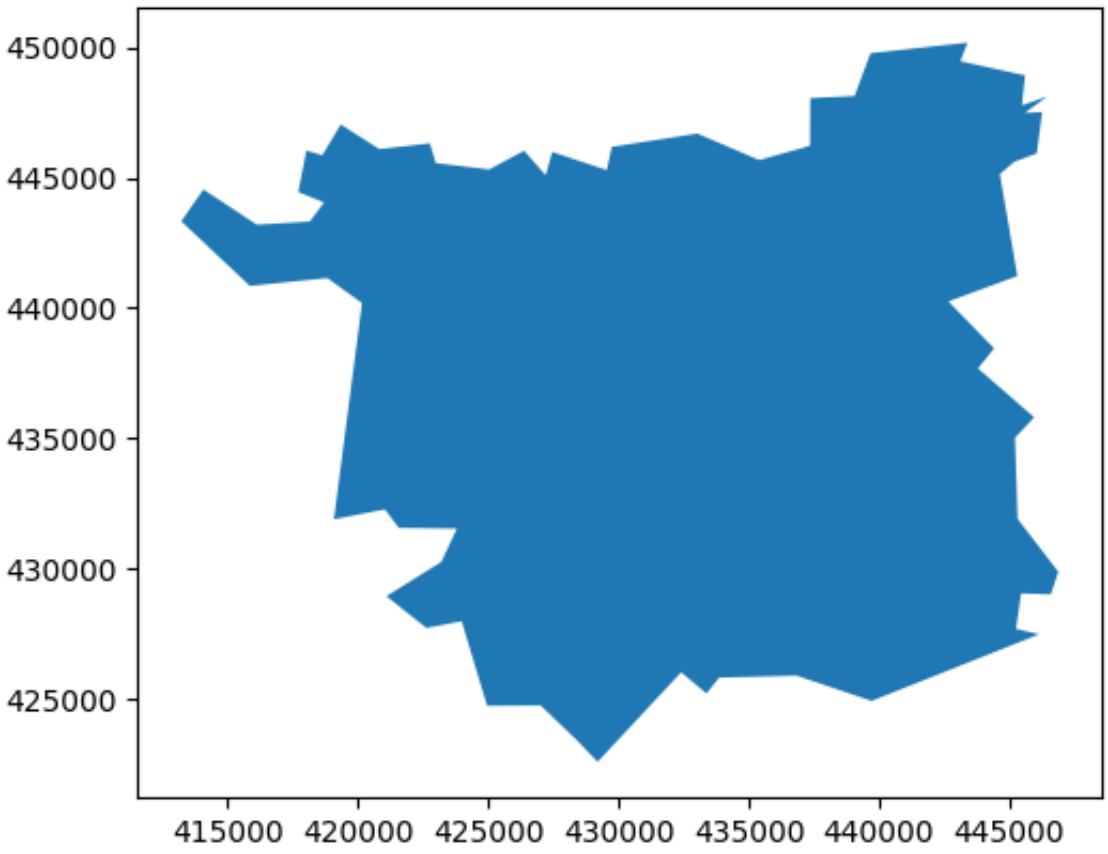
Out[5]:

	LAD22CD	LAD22NM	BNG_E	BNG_N	LONG	LAT	Glo
0	E06000001	Hartlepool	447160	531474	-1.27018	54.6761	0c2c a186-4 ff4fcc43
1	E06000002	Middlesbrough	451141	516887	-1.21099	54.5447	d8f6 675d-4 fed1eb03
2	E06000003	Redcar and Cleveland	464361	519597	-1.00608	54.5675	1844 5fd0-4 f45369a
3	E06000004	Stockton-on-Tees	444940	518183	-1.30664	54.5569	4156 c6d2-4 0157974
4	E06000005	Darlington	428029	515648	-1.56835	54.5353	2ef3 b57d-4 df2cabb4

In [6]:
```
# filter local boundaries to Leeds
LeedsBou= LocalBou[LocalBou['LAD22NM']=='Leeds']
```

In [7]:
```
LeedsBou.plot()
```

Out[7]: <Axes: >



In [8]:
```
LeedsBou.head()
```

Out[8]:

	LAD22CD	LAD22NM	BNG_E	BNG_N	LONG	LAT	Globa
273	E08000035	Leeds	432528	436384	-1.50736	53.8227	8203867 7a96-4aa a8b 3e739fb43f

Output Areas and their population

Output Areas Boundaries

In [9]:
```
"""
# Load Output Areas from ONS
OA= gpd.read_file('Output_Areas_(December_2021)_Boundaries_EW_BFE_(
"""
```

Out[9]:
```
"\n# Load Output Areas from ONS\nOA= gpd.read_file('Output_Areas_(
December_2021)_Boundaries_EW_BFE_(V9).geojson')\n"
```

In [10]:
```
"""
OA.crs
"""
```

Out[10]:
```
'\nOA.crs\n'
```

In [11]:
```
"""
# Set CRS to British National Grid
OA = OA.set_crs(epsg=4326, allow_override=True) # reset manually to

OA = OA.to_crs(epsg=27700) # set to British National Grid
"""
```

Out[11]:
```
'\n# Set CRS to British National Grid\nOA = OA.set_crs(epsg=4326,
allow_override=True) # reset manually to 4326 to prevent errors fa
ced in not changing the crs \n\nOA = OA.to_crs(epsg=27700) # set t
o British National Grid\n'
```

In [12]:
```
"""
OA.crs
"""
```

Out[12]:
```
'\nOA.crs\n'
```

In [13]:
```
"""
# Plot both layers to see the matching crs
fig, ax = plt.subplots()
OA.boundary.plot(ax=ax, color='red', linewidth=0.5)
LeedsBou.boundary.plot(ax=ax, color='blue', linewidth=1)
plt.show()
"""
```

Out[13]:
```
"\n# Plot both layers to see the matching crs\nfig, ax = plt.subpl
ots()\nOA.boundary.plot(ax=ax, color='red', linewidth=0.5)\nLeedsB
ou.boundary.plot(ax=ax, color='blue', linewidth=1)\nplt.show()\n"
```

In [14]:
```
"""
# Clip OA to Leeds boundaries
OA_Leeds= OA.clip(LeedsBou)
"""
```

Out[14]:
```
'\n# Clip OA to Leeds boundaries\nOA_Leeds= OA.clip(LeedsBou)\n'
```

In [15]:
```
"""
# save a geojson copy
OA_Leeds.to_file('OA_Leeds.geojson', driver='GeoJSON')
"""
```

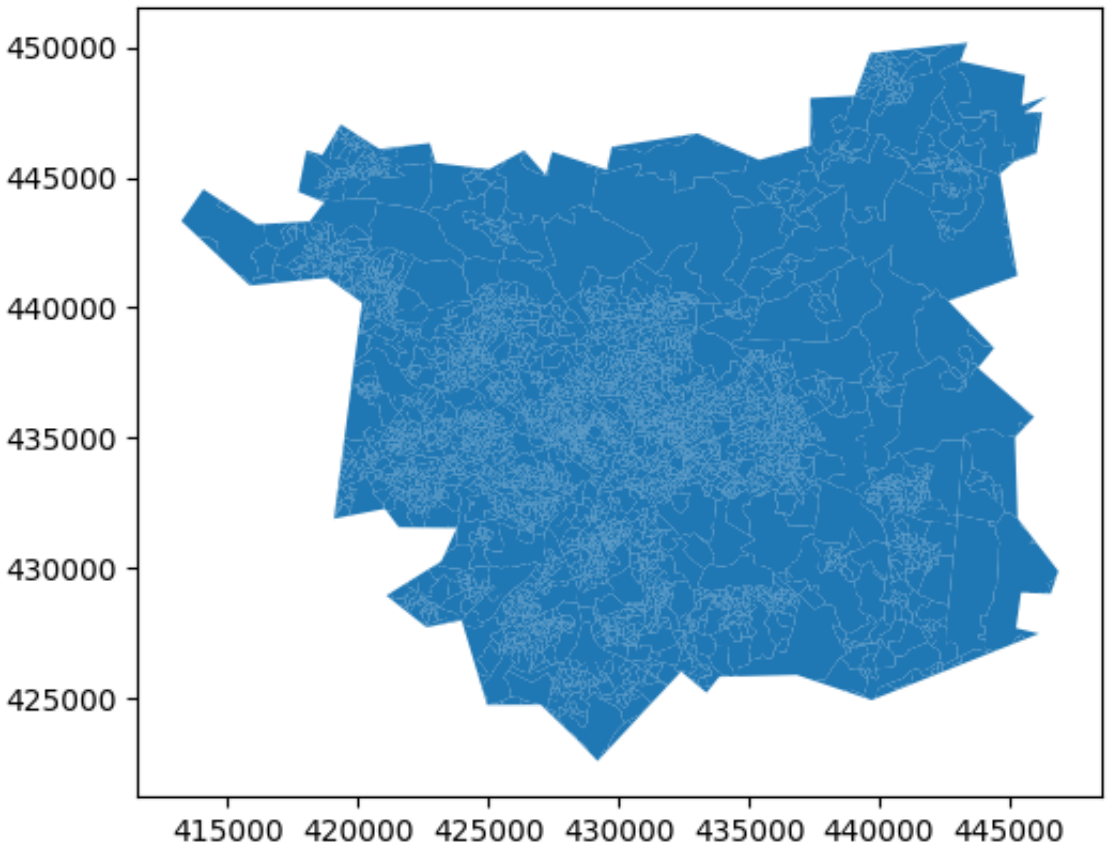
Out[15]:
```
"\n# save a geojson copy\nOA_Leeds.to_file('OA_Leeds.geojson', dri
ver='GeoJSON')\n"
```

In [16]:
```
# load the copy
OA_Leeds = gpd.read_file('OA_Leeds.geojson')

OA_Leeds=OA_Leeds.to_crs(epsg=27700)

OA_Leeds.crs
```

Out[16]:
```
<Projected CRS: EPSG:27700>
Name: OSGB36 / British National Grid
Axis Info [cartesian]:
- E[east]: Easting (metre)
- N[north]: Northing (metre)
Area of Use:
- name: United Kingdom (UK) - offshore to boundary of UKCS within
49°45'N to 61°N and 9°W to 2°E; onshore Great Britain (England, Wa
les and Scotland). Isle of Man onshore.
- bounds: (-9.01, 49.75, 2.01, 61.01)
Coordinate Operation:
- name: British National Grid
- method: Transverse Mercator
Datum: Ordnance Survey of Great Britain 1936
- Ellipsoid: Airy 1830
- Prime Meridian: Greenwich
```

In [17]:
```
OA_Leeds.plot()
```

Out[17]: <Axes: >



In [18]:
```
OA_Leeds.head()
```

Out[18]:

	FID	OA21CD	LSOA21CD	LSOA21NM	LSOA21NMW	BNG_E	BNG_
0	56551	E00059981	E01011889	Wakefield 022A		429706	4223
1	56234	E00059644	E01011825	Wakefield 021C		428768	4223
2	56548	E00059978	E01011885	Wakefield 014A		429737	4229
3	54838	E00058165	E01011536	Leeds 108B		428955	4239
4	54800	E00058127	E01011537	Leeds 107A		429714	4248

Population of OAs

In [19]:
```
# Load population from ONS
population= pd.read_csv('291842303455028.csv', skiprows=6) # skip t
```

In [20]:
```
population.head()
```

Out[20]:

	2021 output area	mnemonic	row	Total: All usual residents	Lives in a household	Lives in a communal establishment
0	E00056750	E00056750	1.0	314.0	314.0	0.0
1	E00056751	E00056751	2.0	286.0	284.0	2.0
2	E00056752	E00056752	3.0	459.0	459.0	0.0
3	E00056753	E00056753	4.0	226.0	226.0	0.0
4	E00056754	E00056754	5.0	275.0	275.0	0.0

In [21]:
```
# drop unwanted columns
population= population.drop(columns=['row','Lives in a household',
```

In [22]:
```
population.head()
```

Out[22]:

	2021 output area	mnemonic	Total: All usual residents
0	E00056750	E00056750	314.0
1	E00056751	E00056751	286.0
2	E00056752	E00056752	459.0
3	E00056753	E00056753	226.0
4	E00056754	E00056754	275.0

In [23]:
```
# rename the population column
population = population.rename(columns={'Total: All usual residents
```

Merge population with OAs boundries

In [24]:
```
OA_Leeds = OA_Leeds.merge(population, how='left', left_on='OA21CD',
```

In [25]:
```
OA_Leeds.head()
```

Out[25]:

	FID	OA21CD	LSOA21CD	LSOA21NM	LSOA21NMW	BNG_E	BNG_
0	56551	E00059981	E01011889	Wakefield 022A		429706	42234
1	56234	E00059644	E01011825	Wakefield 021C		428768	42232
2	56548	E00059978	E01011885	Wakefield 014A		429737	42295
3	54838	E00058165	E01011536	Leeds 108B		428955	42399
4	54800	E00058127	E01011537	Leeds 107A		429714	42489

In [26]:
```
OA_Leeds['population'].info()
```

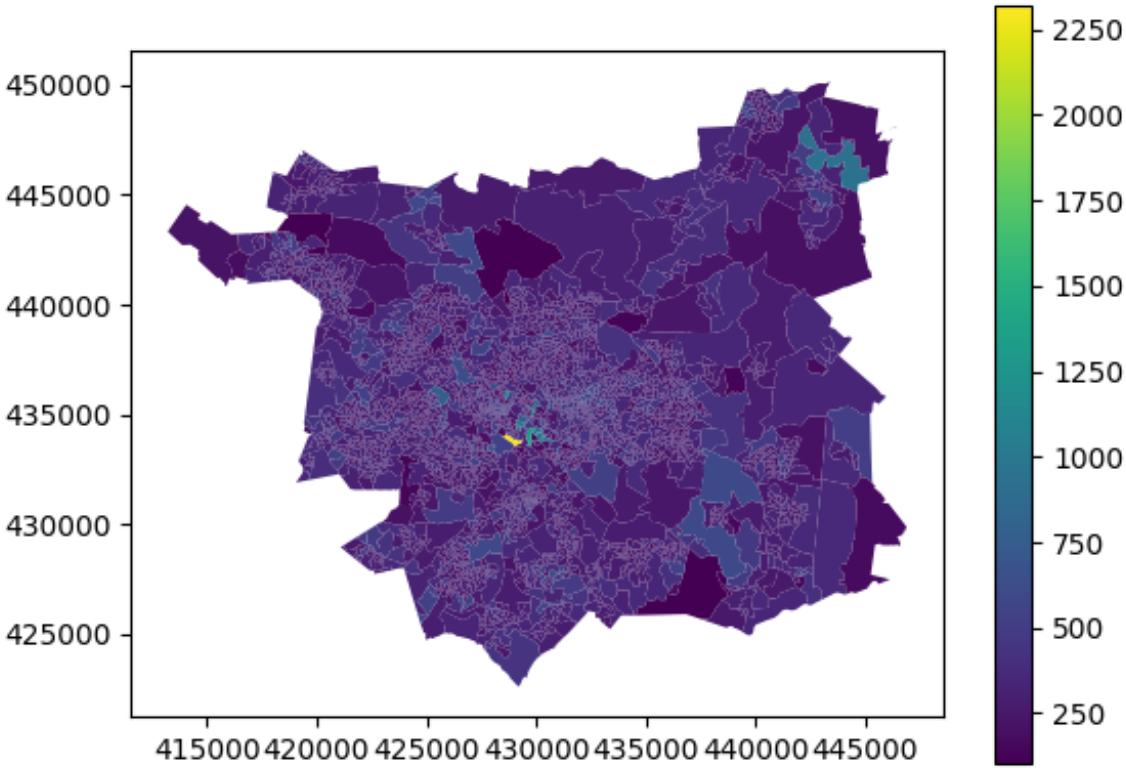
```
<class 'pandas.core.series.Series'>
RangeIndex: 2686 entries, 0 to 2685
Series name: population
Non-Null Count  Dtype
--------------  -----
2603 non-null   float64
dtypes: float64(1)
memory usage: 21.1 KB
```

83 OAs with NaN popoulation

In [27]:
```
# filter OAs to those are not null
OA_Leeds = OA_Leeds[OA_Leeds['population'].notnull()]
```

In [28]:
```
OA_Leeds.plot(column='population', legend=True)
```

Out[28]: <Axes: >



LSOA2011 in Leeds

In [29]:
```
"""
# read shp file
LSOA = gpd.read_file('LSOA_2011_EW_BFC_V3.shp')
"""
```

Out[29]: "\n# read shp file\nLSOA = gpd.read_file('LSOA_2011_EW_BFC_V3.sh
p')\n"

In [30]:
```
"""
LSOA.crs
"""
```

Out[30]: '\nLSOA.crs\n'

In [31]:
```
"""
LSOA.plot()
"""
```

Out[31]: '\nLSOA.plot()\n'

In [32]:
```
"""
# Clip OA to Leeds boundaries
Leeds_LSOAs= LSOA.clip(LeedsBou)
"""
```

Out[32]: '\n# Clip OA to Leeds boundaries\nLeeds_LSOAs= LSOA.clip(LeedsBou)\n'

In [33]:
```
"""
Leeds_LSOAs['LSOA11NM'].unique()
"""
```

Out[33]: "\nLeeds_LSOAs['LSOA11NM'].unique()\n"

In [34]:
```
"""
# filter to Leeds
Leeds_LSOA = Leeds_LSOAs[Leeds_LSOAs['LSOA11NM'].str.startswith('Le
"""
```

Out[34]: "\n# filter to Leeds\nLeeds_LSOA = Leeds_LSOAs[Leeds_LSOAs['LSOA11NM'].str.startswith('Leeds')]\n"

In [35]:
```
"""
# save a geojson copy
Leeds_LSOA.to_file('Leeds_LSOA.geojson', driver='GeoJSON')
"""
```

Out[35]: "\n# save a geojson copy\nLeeds_LSOA.to_file('Leeds_LSOA.geojson', driver='GeoJSON')\n"

In [36]:
```
# load the copy
Leeds_LSOA = gpd.read_file('Leeds_LSOA.geojson')

Leeds_LSOA=Leeds_LSOA.to_crs(epsg=27700)

Leeds_LSOA.crs
```

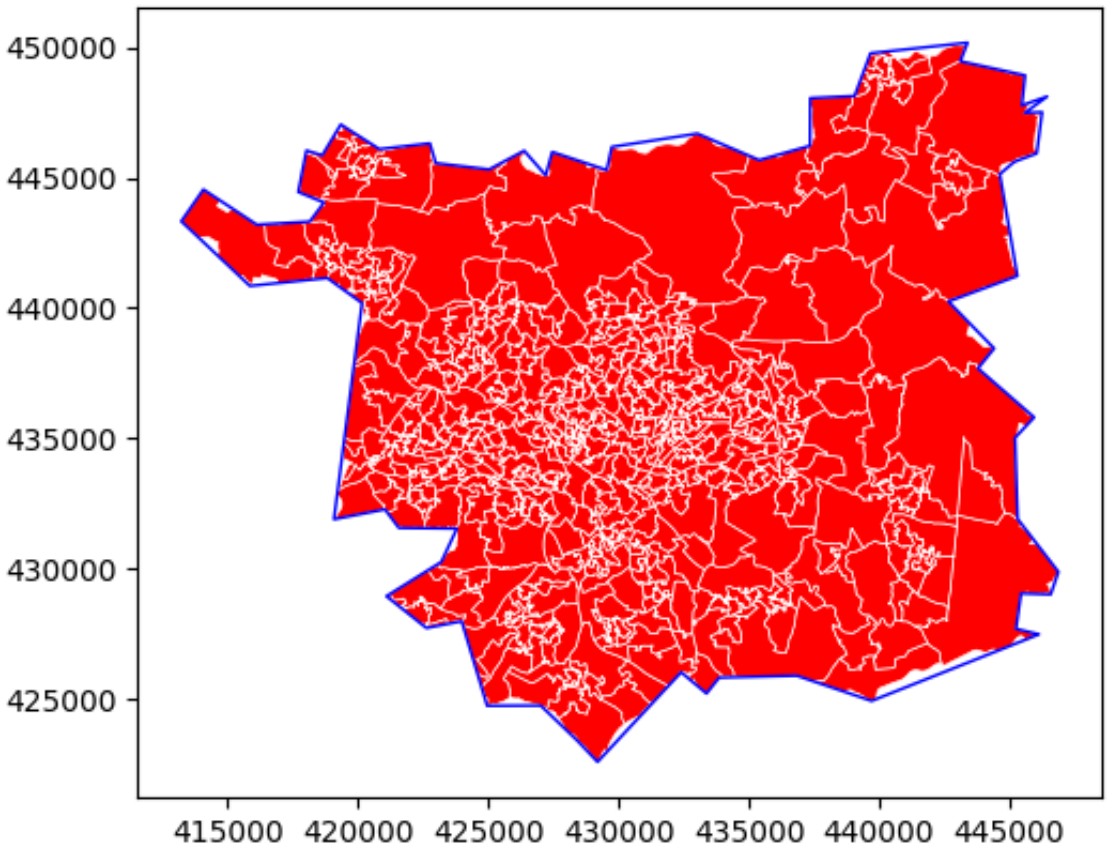
Out[36]:
```
<Projected CRS: EPSG:27700>
Name: OSGB36 / British National Grid
Axis Info [cartesian]:
- E[east]: Easting (metre)
- N[north]: Northing (metre)
Area of Use:
- name: United Kingdom (UK) - offshore to boundary of UKCS within
49°45'N to 61°N and 9°W to 2°E; onshore Great Britain (England, Wa
les and Scotland). Isle of Man onshore.
- bounds: (-9.01, 49.75, 2.01, 61.01)
Coordinate Operation:
- name: British National Grid
- method: Transverse Mercator
Datum: Ordnance Survey of Great Britain 1936
- Ellipsoid: Airy 1830
- Prime Meridian: Greenwich
```

In [37]:
```
Leeds_LSOA.head()
```

Out[37]:

	LSOA11CD	LSOA11NM	BNG_E	BNG_N	LONG_	LAT	Shape_Leng
0	E01011500	Leeds 105D	433469	425875	-1.49420	53.7282	6135.480039
1	E01032496	Leeds 105F	432533	426892	-1.50828	53.7374	5194.489125
2	E01011537	Leeds 107A	430307	424904	-1.54222	53.7197	6196.842276
3	E01011551	Leeds 107C	431131	425643	-1.52966	53.7263	6218.144001
4	E01011499	Leeds 105C	431716	426294	-1.52072	53.7321	9158.803851

In [38]:
```
# Plot both layers
fig, ax = plt.subplots()
Leeds_LSOA.plot(ax=ax, color='red', edgecolor='white', linewidth=0.
LeedsBou.boundary.plot(ax=ax, color='blue', linewidth=1)
plt.show()
```



In [39]:
```
Leeds_LSOA.head()
```

Out[39]:

	LSOA11CD	LSOA11NM	BNG_E	BNG_N	LONG_	LAT	Shape_Leng
0	E01011500	Leeds 105D	433469	425875	-1.49420	53.7282	6135.480039
1	E01032496	Leeds 105F	432533	426892	-1.50828	53.7374	5194.489125
2	E01011537	Leeds 107A	430307	424904	-1.54222	53.7197	6196.842276
3	E01011551	Leeds 107C	431131	425643	-1.52966	53.7263	6218.144001
4	E01011499	Leeds 105C	431716	426294	-1.52072	53.7321	9158.803851

IMD

In [40]:
```
IMD= pd.read_excel('File_1_-_IMD2019_Index_of_Multiple_Deprivation.
```

In [41]:
```
IMD.head()
```

Out[41]:

	LSOA code (2011)	LSOA name (2011)	Local Authority District code (2019)	Local Authority District name (2019)	Index of Multiple Deprivation (IMD) Rank	Index of Multiple Deprivation (IMD) Decile
0	E01000001	City of London 001A	E09000001	City of London	29199	9
1	E01000002	City of London 001B	E09000001	City of London	30379	10
2	E01000003	City of London 001C	E09000001	City of London	14915	5
3	E01000005	City of London 001E	E09000001	City of London	8678	3
4	E01000006	Barking and Dagenham 016A	E09000002	Barking and Dagenham	14486	5

In [42]:
```
IMD = IMD[IMD['LSOA name (2011)'].str.startswith('Leeds')]
```

In [43]:
```
IMD.head()
```

Out[43]:

	LSOA code (2011)	LSOA name (2011)	Local Authority District code (2019)	Local Authority District name (2019)	Index of Multiple Deprivation (IMD) Rank	Index of Multiple Deprivation (IMD) Decile
10947	E01011264	Leeds 011A	E08000035	Leeds	13915	5
10948	E01011265	Leeds 009A	E08000035	Leeds	20368	7
10949	E01011266	Leeds 008A	E08000035	Leeds	29666	10
10950	E01011267	Leeds 009B	E08000035	Leeds	9111	3
10951	E01011268	Leeds 010A	E08000035	Leeds	6082	2

In [44]:
```
IMD.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 482 entries, 10947 to 32185
Data columns (total 6 columns):
 #   Column                                  Non-Null Count  Dty
pe
---  ------                                  --------------  ---
--
 0   LSOA code (2011)                        482 non-null    obj
ect
 1   LSOA name (2011)                        482 non-null    obj
ect
 2   Local Authority District code (2019)    482 non-null    obj
ect
 3   Local Authority District name (2019)    482 non-null    obj
ect
 4   Index of Multiple Deprivation (IMD) Rank  482 non-null    int
64
 5   Index of Multiple Deprivation (IMD) Decile  482 non-null    int
64
dtypes: int64(2), object(4)
memory usage: 26.4+ KB
```

Is merged to LSOAs and spatial diostribbution provided in the DAI-IMD anlysis section.

Residential Landuses in Leeds

In [45]:
```
# Load residential landuse polygons for Leeds from OpenStreetMap
residential_landuse = ox.features_from_place('Leeds, UK', tags={'la
```

In [46]:
```
residential_landuse.head()
```

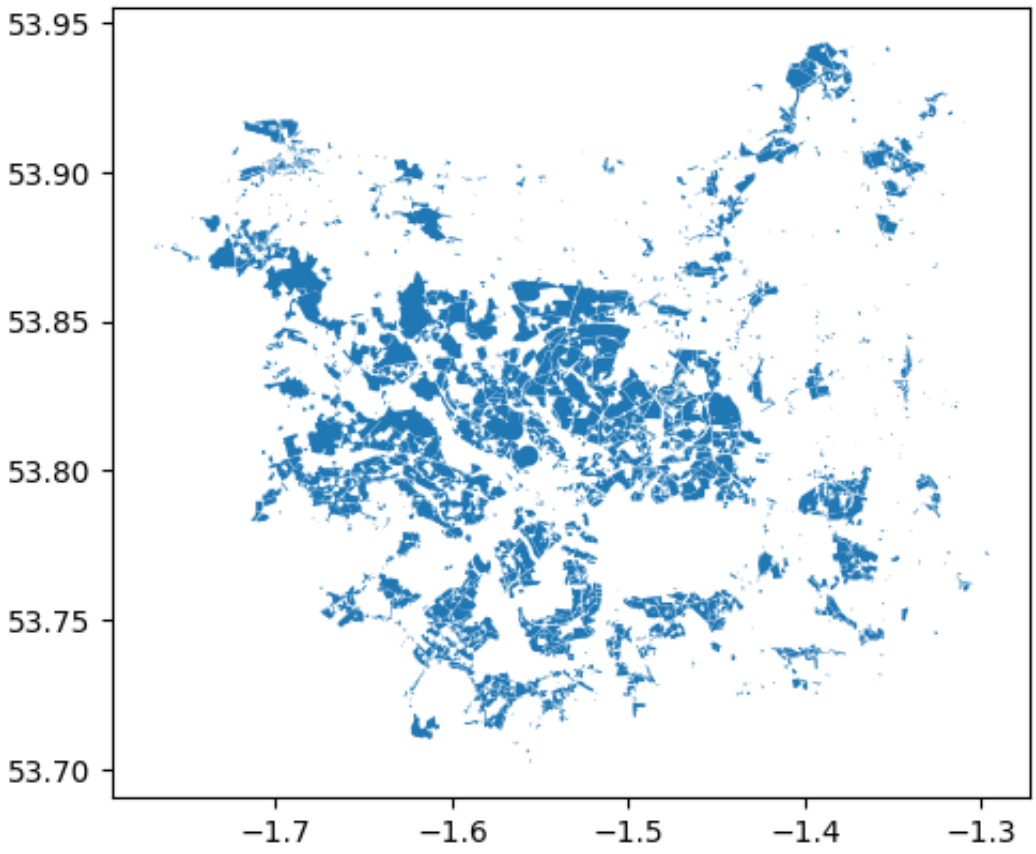
Out[46]:

element	id	geometry	building	landuse	name	addr:city
node	**20621535**	POINT (-1.56566 53.81375)	yes	residential	High Lea Court	NaN
	12175100769	POINT (-1.55729 53.80561)	NaN	residential	Storm Jameson reception, University of Leeds	Leeds
relation	**31299**	POLYGON ((-1.55192 53.72712, -1.55214 53.72721...	NaN	residential	NaN	NaN
	38179	POLYGON ((-1.48324 53.75628, -1.4832 53.75612,...	NaN	residential	NaN	NaN
	3060189	POLYGON ((-1.56553 53.82399, -1.56651 53.82397...	NaN	residential	NaN	NaN

5 rows × 48 columns

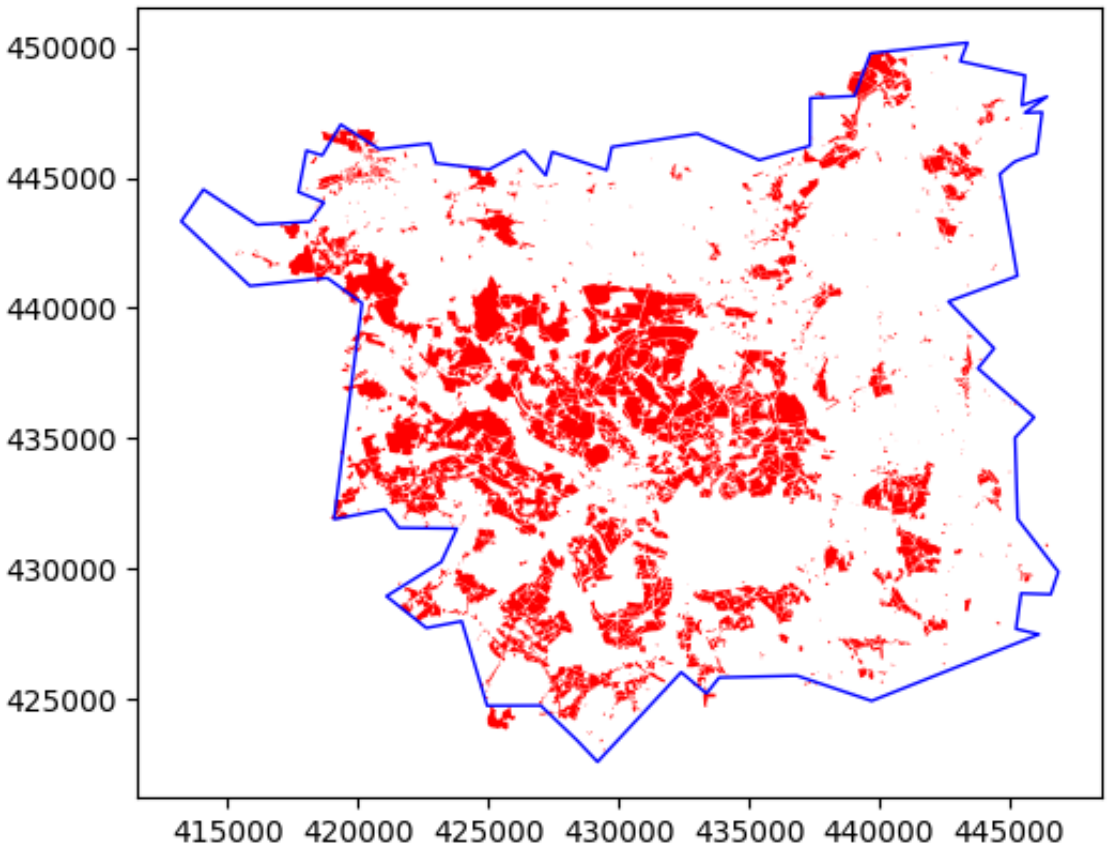
In [47]:
```
residential_landuse.plot()
```

Out[47]: <Axes: >



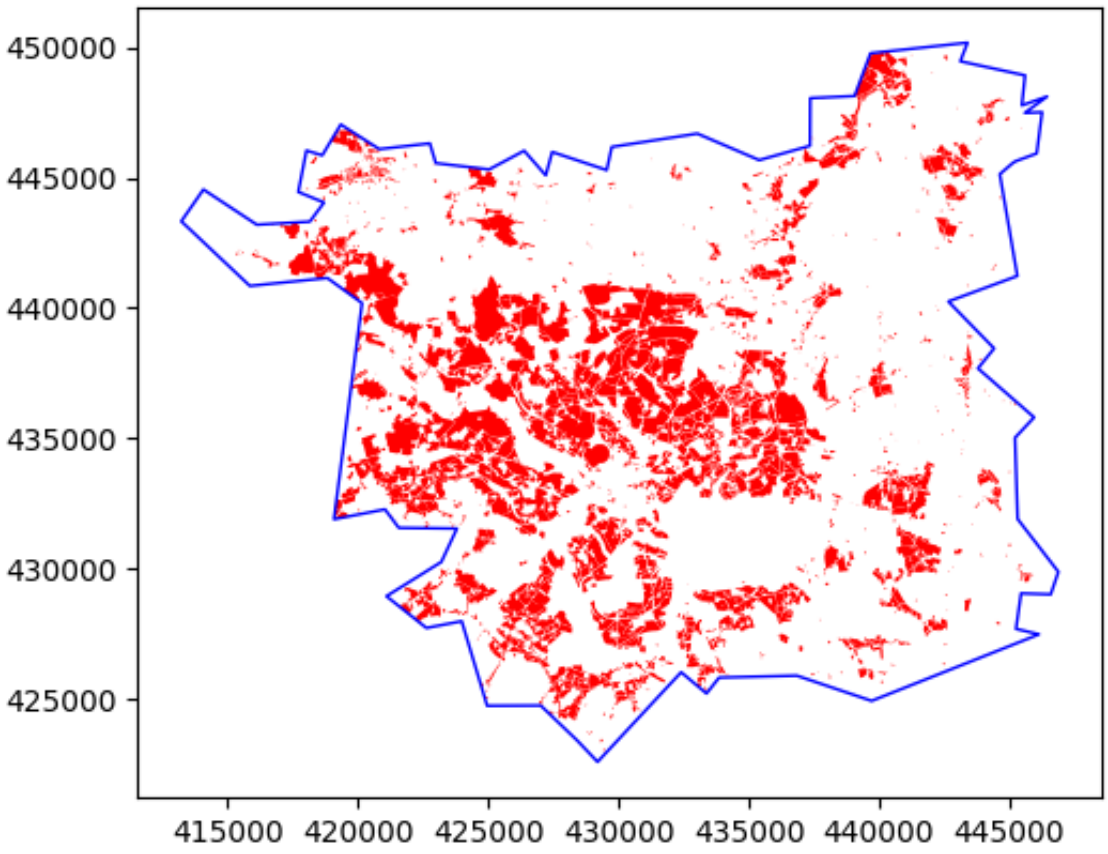
In [48]:
```
residential_landuse = residential_landuse.to_crs(epsg=27700) # set
```

In [49]:
```
# Plot both layers
fig, ax = plt.subplots()
residential_landuse.plot(ax=ax, color='red', linewidth=0.5)
LeedsBou.boundary.plot(ax=ax, color='blue', linewidth=1)
plt.show()
```



In [50]:
```
# Clip residential landuses to Leeds boundaries
residential_landuse= residential_landuse.clip(LeedsBou)
```

In [51]:
```
# Plot both layers
fig, ax = plt.subplots()
residential_landuse.plot(ax=ax, color='red', linewidth=0.5)
LeedsBou.boundary.plot(ax=ax, color='blue', linewidth=1)
plt.show()
```



In [52]:
```
residential_landuse.info()
```

```
<class 'geopandas.geodataframe.GeoDataFrame'>
MultiIndex: 2540 entries, ('way', np.int64(297704086)) to ('way', n
p.int64(891571466))
Data columns (total 48 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   geometry                  2540 non-null   geometry
 1   building                  1 non-null      object
 2   landuse                   2540 non-null   object
 3   name                      185 non-null    object
 4   addr:city                 9 non-null      object
 5   addr:postcode             7 non-null      object
 6   addr:street               6 non-null      object
 7   residential               21 non-null     object
 8   addr:housenumber          3 non-null      object
 9   source                    472 non-null    object
 10  disused                   1 non-null      object
 11  note                      14 non-null     object
 12  operator                  20 non-null     object
 13  old_name                  12 non-null     object
 14  alt_name                  7 non-null      object
 15  wikidata                  7 non-null      object
 16  wikipedia                 6 non-null      object
 17  amenity                   3 non-null      object
 18  operator:type             1 non-null      object
 19  social_facility           2 non-null      object
 20  social_facility:for       2 non-null      object
 21  wheelchair                2 non-null      object
 22  animal_boarding           1 non-null      object
 23  access                    3 non-null      object
 24  description               5 non-null      object
 25  website                   6 non-null      object
 26  source:name               19 non-null     object
 27  source:old_name           3 non-null      object
 28  natural                   1 non-null      object
 29  fixme                     5 non-null      object
 30  construction              10 non-null     object
 31  fhrs:id                   1 non-null      object
 32  place                     23 non-null     object
 33  layer                     1 non-null      object
 34  contact:website           1 non-null      object
 35  building:levels           2 non-null      object
 36  listed_status             2 non-null      object
 37  surface                   3 non-null      object
 38  landuse_2                 1 non-null      object
 39  area                      2 non-null      object
 40  addr:suburb               1 non-null      object
 41  leisure                   1 non-null      object
 42  name:etymology            2 non-null      object
 43  name:etymology:wikidata   2 non-null      object
 44  name:etymology:wikipedia  2 non-null      object
 45  start_date                2 non-null      object
 46  official_name             1 non-null      object
 47  type                      23 non-null     object
dtypes: geometry(1), object(47)
memory usage: 1.0+ MB
```

clip residential landuses into OAs boundaries to locate OAs' weighted centroids based on actual residential areas:

In [53]:
```
# check the geometry types
residential_landuse.geom_type.value_counts()
```

Out[53]:
```
Polygon         2533
MultiPolygon       4
Point              2
LineString         1
Name: count, dtype: int64
```

In [54]:
```
# filter residential landuse to polygons and multipolygons
residential_polygons = residential_landuse[residential_landuse.geom
```

In [55]:
```
# clip residential land use to OA boundaries
OA_residential = gpd.overlay(OA_Leeds, residential_polygons, how='i
```

In [56]:
```
OA_residential.head()
```

Out[56]:

	FID	OA21CD	LSOA21CD	LSOA21NM	LSOA21NMW	BNG_E	BNG_
0	54838	E00058165	E01011536	Leeds 108B		428955	42399
1	54838	E00058165	E01011536	Leeds 108B		428955	42399
2	54838	E00058165	E01011536	Leeds 108B		428955	42399
3	54838	E00058165	E01011536	Leeds 108B		428955	42399
4	54838	E00058165	E01011536	Leeds 108B		428955	42399

5 rows × 63 columns

In [57]:
```
# count number of centroids per OA
centroid_counts = OA_residential.groupby('OA21CD').size().reset_ind
```

In [58]:
```
# merge centroid_counts into OA residential
OA_residential = OA_residential.merge(centroid_counts, on='OA21CD',
```

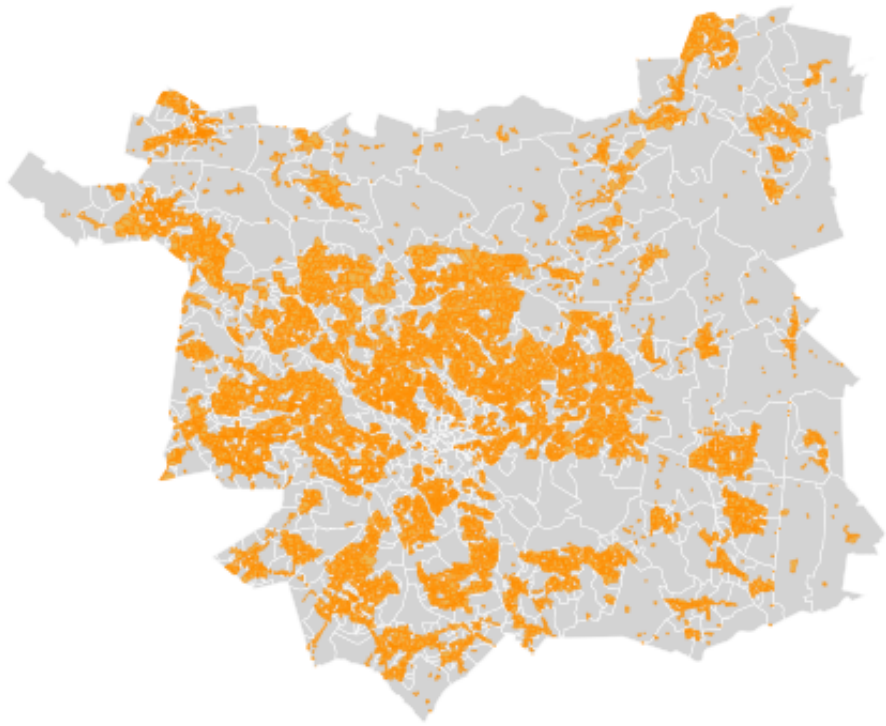
In [59]:
```
# split each OA population between shared residentials
OA_residential['population_split'] = OA_residential['population'] /
```

In [60]:
```
OA_residential.info()
```

```
<class 'geopandas.geodataframe.GeoDataFrame'>
RangeIndex: 6457 entries, 0 to 6456
Data columns (total 65 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   FID                     6457 non-null   int32
 1   OA21CD                  6457 non-null   object
 2   LSOA21CD                6457 non-null   object
 3   LSOA21NM                6457 non-null   object
 4   LSOA21NMW               6457 non-null   object
 5   BNG_E                   6457 non-null   int32
 6   BNG_N                   6457 non-null   int32
 7   LAT                     6457 non-null   float64
 8   LONG                    6457 non-null   float64
 9   Shape__Area             6457 non-null   float64
 10  Shape__Length           6457 non-null   float64
 11  GlobalID                6457 non-null   object
 12  2021 output area        6457 non-null   object
 13  mnemonic                6457 non-null   object
 14  population              6457 non-null   float64
 15  building                0 non-null      object
 16  landuse                 6457 non-null   object
 17  name                    315 non-null    object
 18  addr:city               15 non-null     object
 19  addr:postcode           11 non-null     object
 20  addr:street             9 non-null      object
 21  residential             31 non-null     object
 22  addr:housenumber        5 non-null      object
 23  source                  909 non-null    object
 24  disused                 1 non-null      object
 25  note                    25 non-null     object
 26  operator                32 non-null     object
 27  old_name                24 non-null     object
 28  alt_name                31 non-null     object
 29  wikidata                18 non-null     object
 30  wikipedia               16 non-null     object
 31  amenity                 5 non-null      object
 32  operator:type           2 non-null      object
 33  social_facility         4 non-null      object
 34  social_facility:for     4 non-null      object
 35  wheelchair              4 non-null      object
 36  animal_boarding         1 non-null      object
 37  access                  5 non-null      object
 38  description             8 non-null      object
 39  website                 10 non-null     object
 40  source:name             20 non-null     object
 41  source:old_name         3 non-null      object
```

```
 42  natural                   1 non-null      object
 43  fixme                     9 non-null      object
 44  construction              22 non-null     object
 45  fhrs:id                   2 non-null      object
 46  place                     35 non-null     object
 47  layer                     2 non-null      object
 48  contact:website           3 non-null      object
 49  building:levels           3 non-null      object
 50  listed_status             3 non-null      object
 51  surface                   3 non-null      object
 52  landuse_2                 1 non-null      object
 53  area                      1 non-null      object
 54  addr:suburb               2 non-null      object
 55  leisure                   2 non-null      object
 56  name:etymology            3 non-null      object
 57  name:etymology:wikidata   3 non-null      object
 58  name:etymology:wikipedia  3 non-null      object
 59  start_date                3 non-null      object
 60  official_name             2 non-null      object
 61  type                      218 non-null    object
 62  geometry                  6457 non-null   geometry
 63  n_centroids               6457 non-null   int64
 64  population_split          6457 non-null   float64
dtypes: float64(6), geometry(1), int32(3), int64(1), object(54)
memory usage: 3.1+ MB
```

In [61]:
```
# Plot both layers of residential landuses and OAs
fig, ax = plt.subplots()
OA_Leeds.plot(ax=ax, color='lightgrey',edgecolor='white', linewidth
OA_residential.plot(ax=ax, color='orange',edgecolor='darkorange', l

plt.title('Residential Footprints within Output Areas in Leeds')
plt.axis('off')
plt.show()
```



In [62]:
```
# contolling the accuracy of population split between residential l
print(f"Original total OA population: {OA_Leeds['population'].sum()
print(f"Population after splitting across residential centroids: {O
print(f"Percentage of unassigned population: {(((OA_Leeds['populati
```

```
Original total OA population: 810843.0
Population after splitting across residential centroids: 805236.0
Percentage of unassigned population: 0.7%
```

The percentage of the unassigned population is likely showing a technical loses of data in the process of clipping and reassigning population. The gap is minimal, lass than 1% (0.7%), and does not meaningfully affect the analysis. However, by considering OAs' population which does not have residential landuse (above figure), this might be fixed later on:

Create centroids considering residential landuses

In [63]:
```
# creating centroid per residential polygon (Shows where people acc
res_centroids = OA_residential.copy()
res_centroids['geometry'] = res_centroids.geometry.centroid
```

In [64]:
```
# identifying OAs missing residential landuse data
missing_oas = OA_Leeds[OA_Leeds['OA21CD'].isin(res_centroids['OA21C
```

In [65]:
```
# creating centroids for OAs missing residential footprint
missing_res_centroids_list = [] # an empty list to store basepoints
for row in OA_Leeds.itertuples(index=False): # iteriorate in each r
    if row.OA21CD in missing_oas['OA21CD'].values:           # add
        missing_res_centroids_list.append({
```

```
            'OA21CD': row.OA21CD,
            'population': row.population,
            'geometry': row.geometry.centroid
        })
```

In [66]:
```
# check the list created correctly
missing_res_centroids_list
```

Out[66]:
```
[{'OA21CD': 'E00169629',
  'population': 320.0,
  'geometry': <POINT (430822.434 434188.488)>},
 {'OA21CD': 'E00170475',
  'population': 119.0,
  'geometry': <POINT (426666.853 427578.275)>},
 {'OA21CD': 'E00058189',
  'population': 228.0,
  'geometry': <POINT (426411.196 427798.319)>},
 {'OA21CD': 'E00170268',
  'population': 264.0,
  'geometry': <POINT (430878.549 432698.329)>},
 {'OA21CD': 'E00187126',
  'population': 113.0,
  'geometry': <POINT (430965.919 432736.797)>},
 {'OA21CD': 'E00170037',
  'population': 254.0,
  'geometry': <POINT (430832.981 432784.209)>},
 {'OA21CD': 'E00170040',
  'population': 168.0,
  'geometry': <POINT (430890.042 432797.012)>},
 {'OA21CD': 'E00170267',
  'population': 207.0,
  'geometry': <POINT (430813.963 432909.391)>},
 {'OA21CD': 'E00170329',
  'population': 167.0,
  'geometry': <POINT (431009.214 433007.286)>},
 {'OA21CD': 'E00170265',
  'population': 321.0,
  'geometry': <POINT (430801.482 433023.325)>},
 {'OA21CD': 'E00169796',
  'population': 153.0,
  'geometry': <POINT (430167.246 433206.514)>},
 {'OA21CD': 'E00187034',
  'population': 214.0,
  'geometry': <POINT (429562.074 433222.706)>},
 {'OA21CD': 'E00169811',
  'population': 324.0,
  'geometry': <POINT (429861.713 433255.971)>},
 {'OA21CD': 'E00187033',
  'population': 220.0,
  'geometry': <POINT (429513.585 433354.719)>},
 {'OA21CD': 'E00187122',
  'population': 154.0,
  'geometry': <POINT (429272.544 433393.62)>},
 {'OA21CD': 'E00170262',
  'population': 113.0,
  'geometry': <POINT (429542.119 433397.653)>},
```

```
 {'OA21CD': 'E00169810',
  'population': 161.0,
  'geometry': <POINT (429615.039 433462.586)>},
 {'OA21CD': 'E00169797',
  'population': 154.0,
  'geometry': <POINT (430096.276 433498.69)>},
 {'OA21CD': 'E00169813',
  'population': 117.0,
  'geometry': <POINT (429930.839 433668.944)>},
 {'OA21CD': 'E00170051',
  'population': 170.0,
  'geometry': <POINT (430058.799 433704.308)>},
 {'OA21CD': 'E00187074',
  'population': 300.0,
  'geometry': <POINT (429424.042 433664.24)>},
 {'OA21CD': 'E00169783',
  'population': 165.0,
  'geometry': <POINT (429916.239 433943.781)>},
 {'OA21CD': 'E00170052',
  'population': 140.0,
  'geometry': <POINT (430553.755 433411.815)>},
 {'OA21CD': 'E00169788',
  'population': 373.0,
  'geometry': <POINT (430856.91 433608.608)>},
 {'OA21CD': 'E00169786',
  'population': 190.0,
  'geometry': <POINT (430469.093 433640.656)>},
 {'OA21CD': 'E00169780',
  'population': 263.0,
  'geometry': <POINT (429510.704 434051.125)>},
 {'OA21CD': 'E00058342',
  'population': 235.0,
  'geometry': <POINT (418970.677 445680.309)>}]
```

In [67]:
```
# converting to GeoDataFrame
missing_res_centroids = gpd.GeoDataFrame(missing_res_centroids_list
```

In [68]:
```
# adding source column for each centreoid
res_centroids['source'] = 'Residential Footprint Centroid'
missing_res_centroids['source'] = 'OA Centroid'
```

In [69]:
```
missing_res_centroids.head()
```

Out[69]:

	OA21CD	population	geometry	source
0	E00169629	320.0	POINT (430822.434 434188.488)	OA Centroid
1	E00170475	119.0	POINT (426666.853 427578.275)	OA Centroid
2	E00058189	228.0	POINT (426411.196 427798.319)	OA Centroid
3	E00170268	264.0	POINT (430878.549 432698.329)	OA Centroid
4	E00187126	113.0	POINT (430965.919 432736.797)	OA Centroid

In [70]:
```
# filter columns
```

```
res_centroids= res_centroids[['OA21CD', 'population_split', 'geomet
res_centroids
```

Out[70]:

	OA21CD	population_split	geometry	source
0	E00058165	36.333333	POINT (429433.352 423049.885)	Residential Footprint Centroid
1	E00058165	36.333333	POINT (429301.237 423372.278)	Residential Footprint Centroid
2	E00058165	36.333333	POINT (429343.656 423416.469)	Residential Footprint Centroid
3	E00058165	36.333333	POINT (428853.124 423693.511)	Residential Footprint Centroid
4	E00058165	36.333333	POINT (428954.448 423741.087)	Residential Footprint Centroid
...	...	...	...	...
6452	E00058347	32.000000	POINT (421035.143 445634.06)	Residential Footprint Centroid
6453	E00058347	32.000000	POINT (421125.548 445718.036)	Residential Footprint Centroid
6454	E00058347	32.000000	POINT (421349.37 445709.761)	Residential Footprint Centroid
6455	E00058347	32.000000	POINT (421102.448 445747.304)	Residential Footprint Centroid
6456	E00058347	32.000000	POINT (421817.262 445866.126)	Residential Footprint Centroid

6457 rows × 4 columns

In [71]:
```
# rename population column
res_centroids.rename(columns={'population_split': 'population'}, in
```

In [72]:
```
# joining residential and missing_residential centroids nad reading
combined_centroids = gpd.GeoDataFrame(pd.concat([res_centroids, mis

combined_centroids.head()
```

Out[72]:

	OA21CD	population	geometry	source
0	E00058165	36.333333	POINT (429433.352 423049.885)	Residential Footprint Centroid
1	E00058165	36.333333	POINT (429301.237 423372.278)	Residential Footprint Centroid
2	E00058165	36.333333	POINT (429343.656 423416.469)	Residential Footprint Centroid
3	E00058165	36.333333	POINT (428853.124 423693.511)	Residential Footprint Centroid
4	E00058165	36.333333	POINT (428954.448 423741.087)	Residential Footprint Centroid

In [73]:
```
# check if there is any null value in the geodataframe
combined_centroids.info()
```

```
<class 'geopandas.geodataframe.GeoDataFrame'>
RangeIndex: 6484 entries, 0 to 6483
Data columns (total 4 columns):
 #   Column      Non-Null Count  Dtype   
---  ------      --------------  -----   
 0   OA21CD      6484 non-null   object  
 1   population  6484 non-null   float64 
 2   geometry    6484 non-null   geometry
 3   source      6484 non-null   object  
dtypes: float64(1), geometry(1), object(2)
memory usage: 202.8+ KB
```

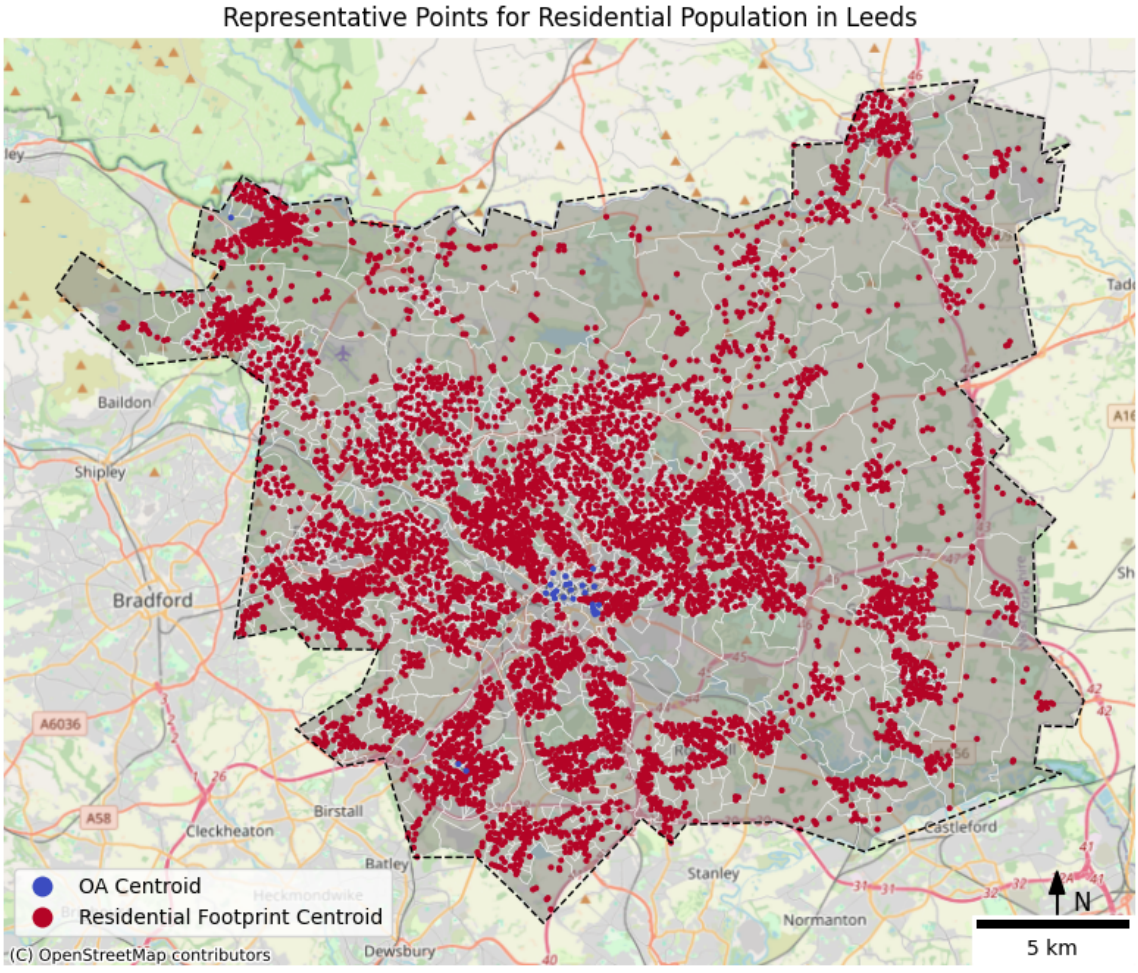
In [74]:
```
# controlling the split poulation between centreoids
print(f"Centreoid population: {combined_centroids['population'].sum
print(f"Population across OAs: {OA_Leeds['population'].sum()}")
```

```
Centreoid population: 810843.0
Population across OAs: 810843.0
```

The small unassigned population (0.7%) was likely due to OAs without intersecting residential land use polygons during clipping. After adding the missing centroids, the total population now matches the original OA data exactly (810,843). So the issue is resolved and no longer affects the analysis.

In [75]:
```
fig, ax = plt.subplots(figsize=(8, 8))

# Plot Output Areas
OA_Leeds.plot(ax=ax, color='grey', edgecolor='white', linewidth=0.6
LeedsBou.boundary.plot(ax=ax, color='k', linewidth=1, linestyle='da
#OA_residential.plot(ax=ax, color='orange', linewidth=0.4, alpha=0.

# Plot centroids coloured by source
combined_centroids.plot(ax=ax, column='source', markersize=4, cmap=

# title
plt.title('Representative Points for Residential Population in Leed
```

```
# add legend and set location details
leg = ax.get_legend()
leg.set_bbox_to_anchor((0, 0.02))
leg.set_loc('lower left')

# add basemap (EPSG:27700)
cx.add_basemap(ax, crs=27700, source=cx.providers.OpenStreetMap.Map

# add North arrow and N
ax.arrow(446000, 422800, 0, 1500, head_width=500, length_includes_h
ax.text(446800, 423000, 'N', fontsize=12, ha='center')


scale1 = ScaleBar(dx=1,location='lower right') #create a scale bar
ax.add_artist(scale1) #plot the scale bar

plt.axis('off')
plt.tight_layout()
plt.show()
```



Points of Interests

In [76]:
```
# Load POIs from Ordnance Survey
POI = pd.read_csv('/Users/naiemegolzari/Desktop/UniLeeds/Analytics
                  sep='|',            # split using |
                  engine='python' # flexible reading
```

```
                    )
```

In [77]:
```
POI.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 47367 entries, 0 to 47366
Data columns (total 25 columns):
 #   Column             Non-Null Count  Dtype  
---  ------             --------------  -----  
 0   ref_no             47367 non-null  int64  
 1   name               47367 non-null  object 
 2   pointx_class       47367 non-null  int64  
 3   feature_easting    47367 non-null  float64
 4   feature_northing   47367 non-null  float64
 5   pos_accuracy       47367 non-null  int64  
 6   uprn               28923 non-null  float64
 7   topo_toid          47367 non-null  object 
 8   topo_toid_version  47367 non-null  int64  
 9   usrn               47367 non-null  int64  
 10  usrn_mi            47367 non-null  int64  
 11  distance           47367 non-null  float64
 12  address_detail     27995 non-null  object 
 13  street_name        28450 non-null  object 
 14  locality           29735 non-null  object 
 15  geographic_county  47367 non-null  object 
 16  postcode           47367 non-null  object 
 17  admin_boundary     47367 non-null  object 
 18  telephone_number   22928 non-null  float64
 19  url                11893 non-null  object 
 20  brand              6251 non-null   object 
 21  qualifier_type     7784 non-null   object 
 22  qualifier_data     7784 non-null   object 
 23  provenance         47367 non-null  object 
 24  supply_date        47367 non-null  object 
dtypes: float64(5), int64(6), object(14)
memory usage: 9.0+ MB
```

In [78]:
```
POI.head()
```

Out[78]:

	ref_no	name	pointx_class	feature_easting	feature_northing	pos
0	18254163	Friends Meeting House	6340459	426420.0	439307.0	
1	18254542	Friends Meeting House	6340459	420794.0	440064.0	
2	18255695	Harehills Gospel Hall	6340459	432135.0	435649.0	
3	18253865	Friends Meeting House	6340459	424344.0	429069.0	
4	18254521	Kingdom Hall of Jehovah's Witnesses	6340459	428997.0	436788.0	

5 rows × 25 columns

In [79]:
```
# Create geometry column from easting/northing
POI['geometry'] = POI.apply(lambda row: Point(row['feature_easting'
```

In [80]:
```
POI.head()
```

Out[80]:

	ref_no	name	pointx_class	feature_easting	feature_northing	pos
0	18254163	Friends Meeting House	6340459	426420.0	439307.0	
1	18254542	Friends Meeting House	6340459	420794.0	440064.0	
2	18255695	Harehills Gospel Hall	6340459	432135.0	435649.0	
3	18253865	Friends Meeting House	6340459	424344.0	429069.0	
4	18254521	Kingdom Hall of Jehovah's Witnesses	6340459	428997.0	436788.0	

5 rows × 26 columns

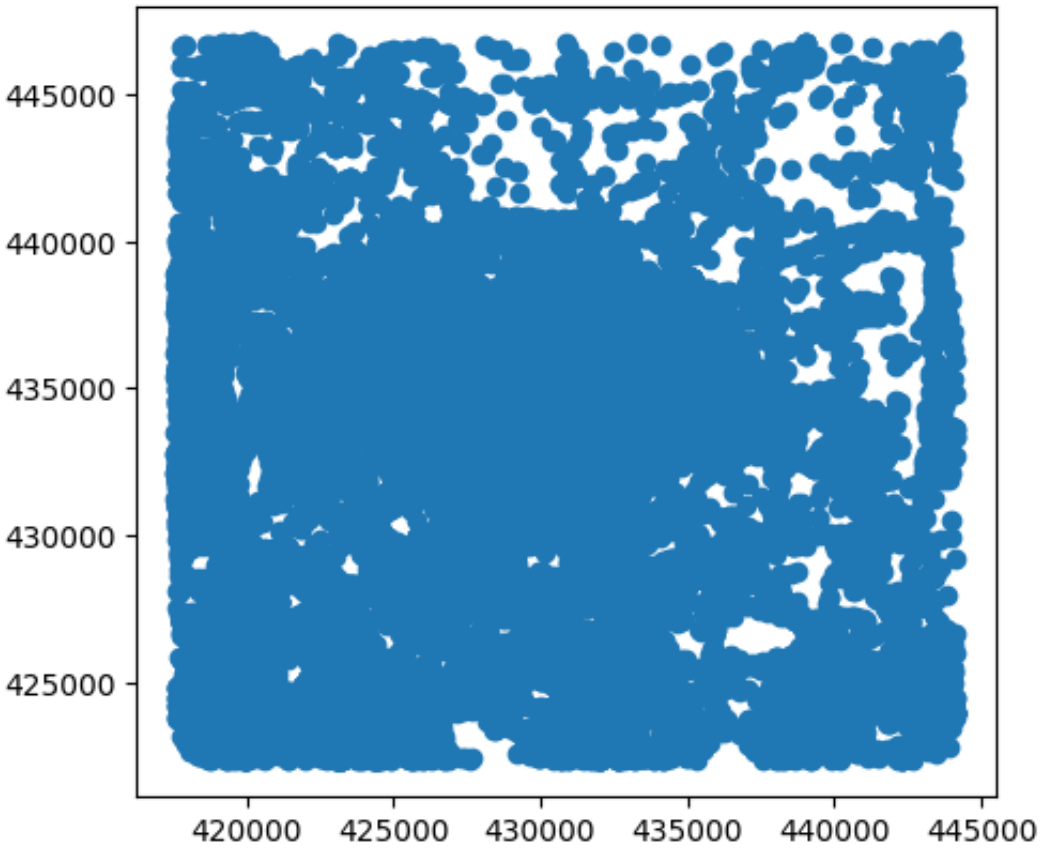
In [81]:
```
# Convert to GeoDataFrame
POI2 = gpd.GeoDataFrame(POI, geometry='geometry', crs='EPSG:27700')
```

```
POI2.crs
```

Out[81]:
```
<Projected CRS: EPSG:27700>
Name: OSGB36 / British National Grid
Axis Info [cartesian]:
- E[east]: Easting (metre)
- N[north]: Northing (metre)
Area of Use:
- name: United Kingdom (UK) - offshore to boundary of UKCS within
49°45'N to 61°N and 9°W to 2°E; onshore Great Britain (England, Wa
les and Scotland). Isle of Man onshore.
- bounds: (-9.01, 49.75, 2.01, 61.01)
Coordinate Operation:
- name: British National Grid
- method: Transverse Mercator
Datum: Ordnance Survey of Great Britain 1936
- Ellipsoid: Airy 1830
- Prime Meridian: Greenwich
```

In [82]:
```
POI2.plot()
```

Out[82]:
```
<Axes: >
```

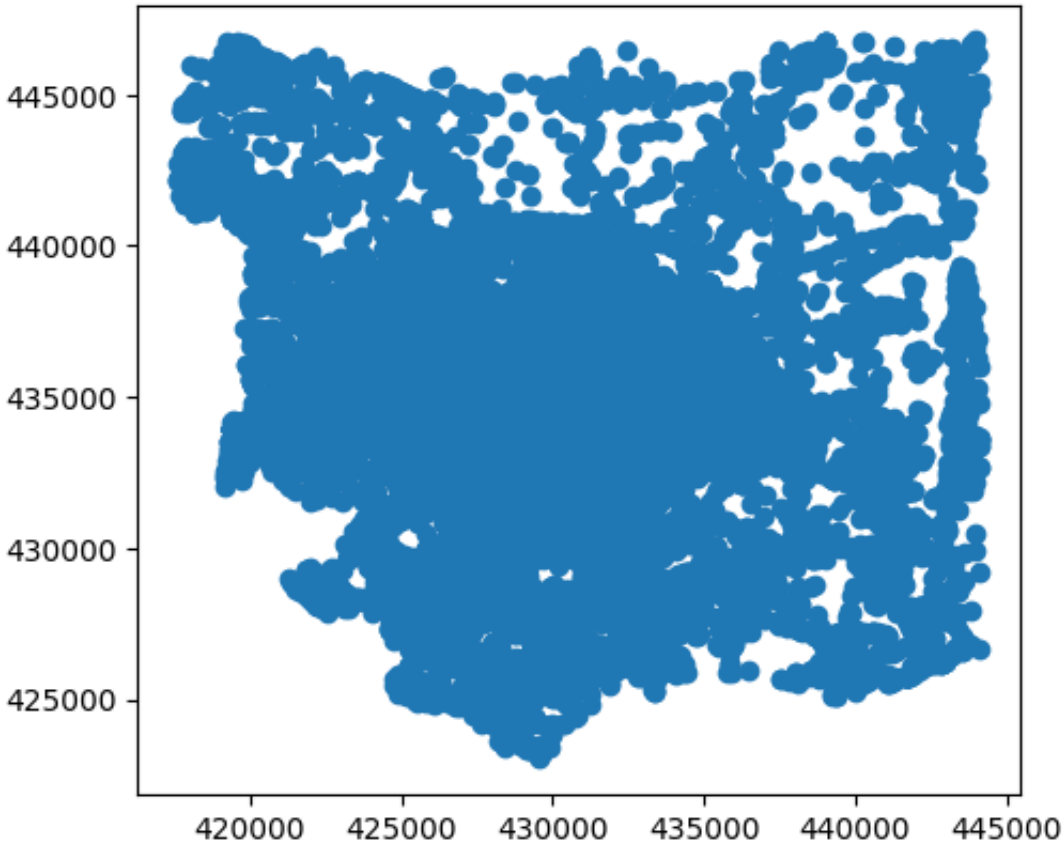


In [83]:
```
# Clip POIs to Leeds boundaries
POI_Leeds= POI2.clip(LeedsBou)

POI_Leeds.crs
```

Out[83]:
```
<Projected CRS: EPSG:27700>
Name: OSGB36 / British National Grid
Axis Info [cartesian]:
- E[east]: Easting (metre)
- N[north]: Northing (metre)
Area of Use:
- name: United Kingdom (UK) - offshore to boundary of UKCS within
49°45'N to 61°N and 9°W to 2°E; onshore Great Britain (England, Wa
les and Scotland). Isle of Man onshore.
- bounds: (-9.01, 49.75, 2.01, 61.01)
Coordinate Operation:
- name: British National Grid
- method: Transverse Mercator
Datum: Ordnance Survey of Great Britain 1936
- Ellipsoid: Airy 1830
- Prime Meridian: Greenwich
```

In [84]:
```
POI_Leeds.plot()
```

Out[84]:
```
<Axes: >
```



In [85]:
```
# checking the POIs category (the supporting doc provided by OS has
POI_Leeds['pointx_class'].value_counts()
```

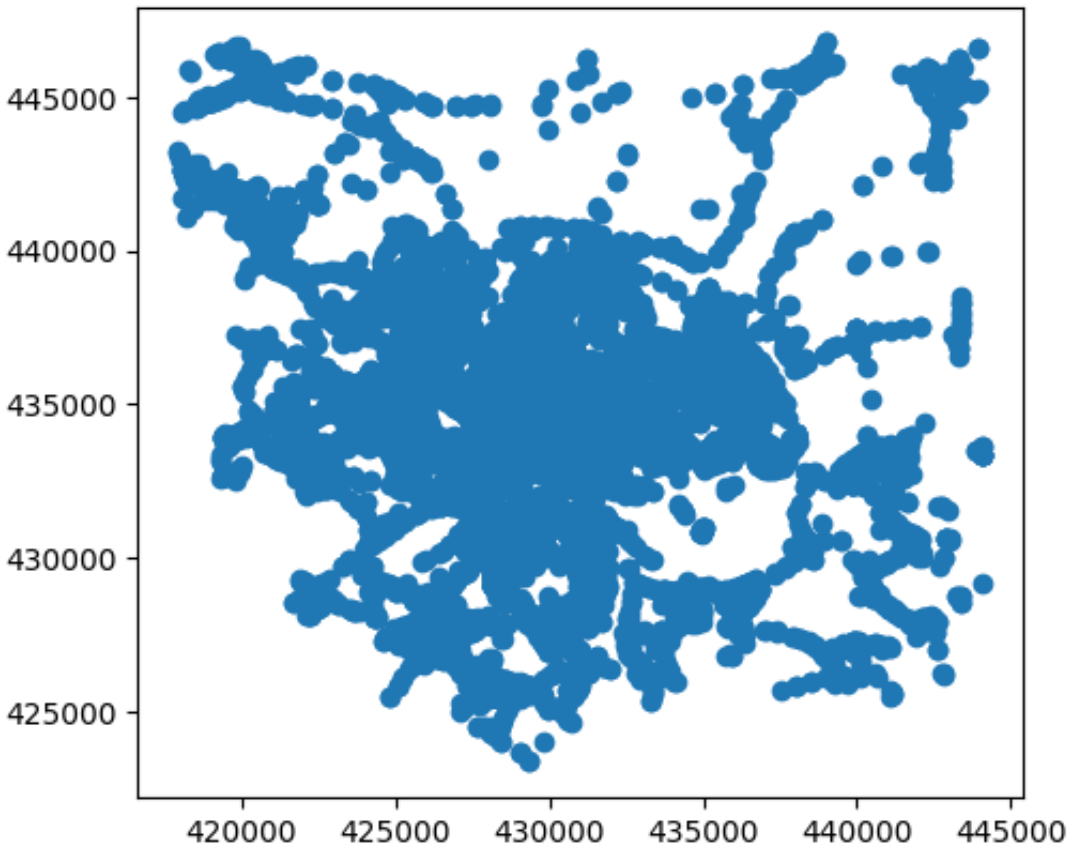
Out[85]:
```
pointx_class
10590732    4177
6340433     2903
2100156     1271
1020018      928
6340457      788
            ...
2050079        1
5320388        1
5260318        1
2130211        1
2600104        1
Name: count, Length: 527, dtype: int64
```

In [86]:
```
# filter POIs to daily amenities
daily_codes = [
    1020018,  # Fast Food and Takeaway Outlets
    1020020,  # Fish and Chip Shops
    1020043,  # Restaurants
    9470662,  # Butchers
    9470699,  # Convenience Stores and Independent Supermarkets
    9470705,  # Markets
    9470819,  # Supermarket Chains

    2090141,  # Cash Machines
    9480763,  # Post Offices

    3180255,  # Playgrounds
    3180814,  # Municipal Parks and Gardens

    4240293,  # Gymnasiums, Sports Halls and Leisure Centres

    5280364,  # Chemists and Pharmacies
    5280365,  # Clinics and Health Centres
    5280373,  # Nursing and Residential Care Homes

    5310375,  # First, Primary and Infant Schools
    5320397,  # Nursery Schools and Pre and After School Care
    5320403,  # Training Providers and Centres

    10570731, # Bus and Coach Stations, Depots and Companies
    10570738, # Railway Stations, Junctions and Halts
    10570756, # Tram, Metro and Light Railway Stations and Stops
    10570761, # Underground Network Stations
    10590732  # Bus Stops
]

# Filter the POI dataset
POI_daily = POI_Leeds[POI_Leeds['pointx_class'].isin(daily_codes)]
```

In [87]:
```
print(POI_daily.crs)
POI_daily.plot()
```

```
EPSG:27700
```

Out[87]:
```
<Axes: >
```



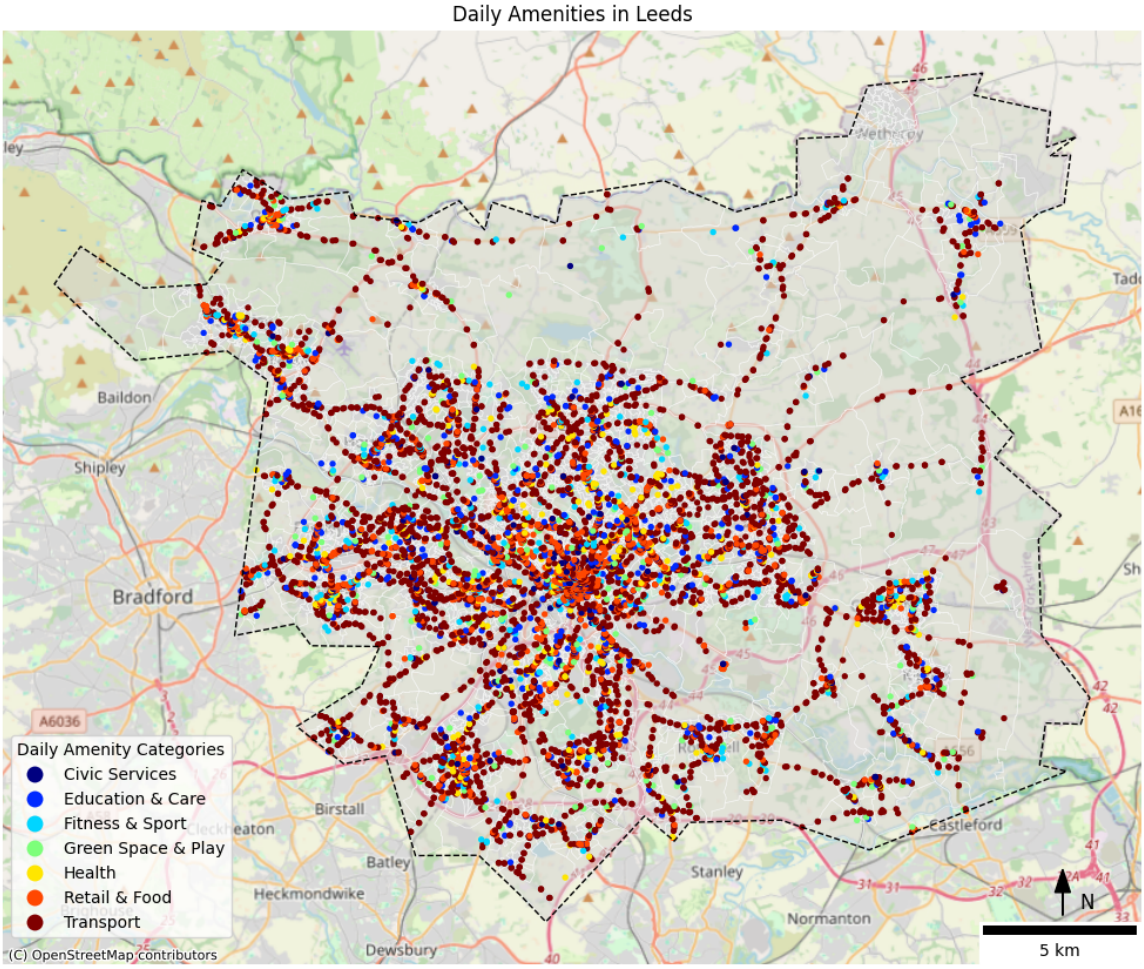
In [88]:
```
# Dictionary for assigning each POI code to a category
poi_categories = {
    1020018: 'Retail & Food',
    1020020: 'Retail & Food',
    1020043: 'Retail & Food',
    9470662: 'Retail & Food',
    9470699: 'Retail & Food',
    9470705: 'Retail & Food',
    9470819: 'Retail & Food',

    2090141: 'Civic Services',
    9480763: 'Civic Services',

    3180255: 'Green Space & Play',
    3180814: 'Green Space & Play',

    4240293: 'Fitness & Sport',

    5280364: 'Health',
    5280365: 'Health',
    5280373: 'Health',

    5310375: 'Education & Care',
    5320397: 'Education & Care',
    5320403: 'Education & Care',

    10570731: 'Transport',
    10570738: 'Transport',
    10570756: 'Transport',
    10570761: 'Transport',
    10590732: 'Transport'
```

```
}
```

In [89]:
```
# map labels to POI_daily_category
POI_daily['poi_category'] = POI_daily['pointx_class'].map(poi_categ
```

```
/Users/naiemegolzari/Library/Python/3.9/lib/python/site-packages/geo
pandas/geodataframe.py:1819: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pand
as-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-co
py
  super().__setitem__(key, value)
```

In [90]:
```
# plot
fig, ax = plt.subplots(figsize=(10, 10))

# basemap (Output Areas and boundary)
OA_Leeds.plot(ax=ax, color='lightgrey', edgecolor='white', linewidt
LeedsBou.boundary.plot(ax=ax, color='k', linewidth=1, linestyle='da

# POIs
POI_daily.plot(ax=ax, column='poi_category', cmap='jet', markersize

# Add basemap (EPSG:27700)
cx.add_basemap(ax, crs=27700, source=cx.providers.OpenStreetMap.Map

# North Arrow
ax.arrow(446000, 422800, 0, 1500, head_width=500, length_includes_h
ax.text(446800, 423000, 'N', fontsize=12, ha='center')

# Scale Bar
scale = ScaleBar(dx=1, location='lower right')
ax.add_artist(scale)

# add legend and set location details
leg = ax.get_legend()
leg.set_bbox_to_anchor((0, 0.02))
leg.set_loc('lower left')
leg.set_title('Daily Amenity Categories')

plt.title('Daily Amenities in Leeds')

plt.axis('off')
plt.tight_layout()
plt.show()
```



In [91]:
```
POI_daily.info()
```

```
<class 'geopandas.geodataframe.GeoDataFrame'>
Index: 8937 entries, 4316 to 7930
Data columns (total 27 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   ref_no             8937 non-null   int64
 1   name               8937 non-null   object
 2   pointx_class       8937 non-null   int64
 3   feature_easting    8937 non-null   float64
 4   feature_northing   8937 non-null   float64
 5   pos_accuracy       8937 non-null   int64
 6   uprn               4194 non-null   float64
 7   topo_toid          8937 non-null   object
 8   topo_toid_version  8937 non-null   int64
 9   usrn               8937 non-null   int64
 10  usrn_mi            8937 non-null   int64
 11  distance           8937 non-null   float64
 12  address_detail     3940 non-null   object
 13  street_name        4262 non-null   object
 14  locality           4356 non-null   object
 15  geographic_county  8937 non-null   object
 16  postcode           8937 non-null   object
 17  admin_boundary     8937 non-null   object
 18  telephone_number   2746 non-null   float64
 19  url                1565 non-null   object
 20  brand              1228 non-null   object
 21  qualifier_type     4785 non-null   object
 22  qualifier_data     4785 non-null   object
 23  provenance         8937 non-null   object
 24  supply_date        8937 non-null   object
 25  geometry           8937 non-null   geometry
 26  poi_category       8937 non-null   object
dtypes: float64(5), geometry(1), int64(6), object(15)
memory usage: 1.9+ MB
```

Pedestrain Network

In [92]:
```
"""
# download the pedestrain network from OSM
Leeds_W_Network = ox.graph_from_place('Leeds, UK', network_type='wa
"""
```

Out[92]:
```
"\n# download the pedestrain network from OSM\nLeeds_W_Network = o
x.graph_from_place('Leeds, UK', network_type='walk')\n"
```

In [93]:
```
"""
# save
ox.save_graphml(Leeds_W_Network, filepath='leeds_walk_network.graph
"""
```

Out[93]:
```
"\n# save \nox.save_graphml(Leeds_W_Network, filepath='leeds_walk_
network.graphml')\n"
```

In [94]:
```
# load the pedestrain network
Leeds_W_Network= ox.load_graphml('leeds_walk_network.graphml')
```

In [95]:
```
# choose undirected connection to the graph, more compatible with w
Leeds_W_Network = Leeds_W_Network.to_undirected()
```

In [96]:
```
# project to crs British Natinal Grid
Leeds_W_Network= ox.project_graph(Leeds_W_Network, to_crs='EPSG:277
```

In [97]:
```
# Convert the network to GeoDataFrames (nodes and edges)
nodes, edges = ox.graph_to_gdfs(Leeds_W_Network)
```

In [98]:
```
"""
# Save nodes & edges (road network)
nodes.to_file('LWN_Nodes.shp')
edges.to_file('LWN_Edges.shp')
"""
```

Out[98]:
```
"\n# Save nodes & edges (road network)\nnodes.to_file('LWN_Nodes.s
hp')\nedges.to_file('LWN_Edges.shp')\n"
```

In [99]:
```
# load & check crs
nodes = gpd.read_file('LWN_Nodes.shp')
edges = gpd.read_file('LWN_Edges.shp')

print(nodes.crs)
print(edges.crs)
```

```
EPSG:27700
EPSG:27700
```

Spatial Analysis

Creating Network of Walking Paths & POIs

Connect POIs with their nearest walking node

In [100…
```
# checking crs
POI_daily.crs
```

Out[100…
```
<Projected CRS: EPSG:27700>
Name: OSGB36 / British National Grid
Axis Info [cartesian]:
- E[east]: Easting (metre)
- N[north]: Northing (metre)
Area of Use:
- name: United Kingdom (UK) - offshore to boundary of UKCS within
49°45'N to 61°N and 9°W to 2°E; onshore Great Britain (England, Wa
les and Scotland). Isle of Man onshore.
- bounds: (-9.01, 49.75, 2.01, 61.01)
Coordinate Operation:
- name: British National Grid
- method: Transverse Mercator
Datum: Ordnance Survey of Great Britain 1936
- Ellipsoid: Airy 1830
- Prime Meridian: Greenwich
```

In [101…
```
POI_daily.info()
```

```
<class 'geopandas.geodataframe.GeoDataFrame'>
Index: 8937 entries, 4316 to 7930
Data columns (total 27 columns):
 #   Column             Non-Null Count  Dtype   
---  ------             --------------  -----   
 0   ref_no             8937 non-null   int64   
 1   name               8937 non-null   object  
 2   pointx_class       8937 non-null   int64   
 3   feature_easting    8937 non-null   float64 
 4   feature_northing   8937 non-null   float64 
 5   pos_accuracy       8937 non-null   int64   
 6   uprn               4194 non-null   float64 
 7   topo_toid          8937 non-null   object  
 8   topo_toid_version  8937 non-null   int64   
 9   usrn               8937 non-null   int64   
 10  usrn_mi            8937 non-null   int64   
 11  distance           8937 non-null   float64 
 12  address_detail     3940 non-null   object  
 13  street_name        4262 non-null   object  
 14  locality           4356 non-null   object  
 15  geographic_county  8937 non-null   object  
 16  postcode           8937 non-null   object  
 17  admin_boundary     8937 non-null   object  
 18  telephone_number   2746 non-null   float64 
 19  url                1565 non-null   object  
 20  brand              1228 non-null   object  
 21  qualifier_type     4785 non-null   object  
 22  qualifier_data     4785 non-null   object  
 23  provenance         8937 non-null   object  
 24  supply_date        8937 non-null   object  
 25  geometry           8937 non-null   geometry
 26  poi_category       8937 non-null   object  
dtypes: float64(5), geometry(1), int64(6), object(15)
memory usage: 1.9+ MB
```

In [102…
```
# a subset of the data frame
POI_subset = POI_daily[['geometry', 'poi_category']].copy()
```

In [103…
```
# Extract x, y as separate lists
xs = POI_subset.geometry.x.values
ys = POI_subset.geometry.y.values
```

In [104…
```
# undirected graph
Leeds_W_Network = Leeds_W_Network.to_undirected()

# check if the walking netwrok graph is connected
nx.is_connected(Leeds_W_Network)
```

Out[104… True

In [105…
```
POI_subset.head()
```

Out[105…

	geometry	poi_category
4316	POINT (427661 424503)	Transport
7349	POINT (427662 424512)	Transport
6182	POINT (427133 425013)	Transport
7123	POINT (427641 425256)	Transport
3828	POINT (427629 425257)	Transport

In [106…

```
POI_subset = POI_subset.to_crs(epsg=27700)

POI_subset.crs
```

Out[106…

```
<Projected CRS: EPSG:27700>
Name: OSGB36 / British National Grid
Axis Info [cartesian]:
- E[east]: Easting (metre)
- N[north]: Northing (metre)
Area of Use:
- name: United Kingdom (UK) - offshore to boundary of UKCS within
49°45'N to 61°N and 9°W to 2°E; onshore Great Britain (England, Wa
les and Scotland). Isle of Man onshore.
- bounds: (-9.01, 49.75, 2.01, 61.01)
Coordinate Operation:
- name: British National Grid
- method: Transverse Mercator
Datum: Ordnance Survey of Great Britain 1936
- Ellipsoid: Airy 1830
- Prime Meridian: Greenwich
```

snapping POIs to the walking network of Leeds

In [107…

```
Leeds_W_Network.graph['crs']
```

Out[107…

```
<Projected CRS: EPSG:27700>
Name: OSGB36 / British National Grid
Axis Info [cartesian]:
- E[east]: Easting (metre)
- N[north]: Northing (metre)
Area of Use:
- name: United Kingdom (UK) - offshore to boundary of UKCS within
49°45'N to 61°N and 9°W to 2°E; onshore Great Britain (England, Wa
les and Scotland). Isle of Man onshore.
- bounds: (-9.01, 49.75, 2.01, 61.01)
Coordinate Operation:
- name: British National Grid
- method: Transverse Mercator
Datum: Ordnance Survey of Great Britain 1936
- Ellipsoid: Airy 1830
- Prime Meridian: Greenwich
```

In [108…

```
edges.crs
```

Out[108…
```
<Projected CRS: EPSG:27700>
Name: OSGB36 / British National Grid
Axis Info [cartesian]:
- E[east]: Easting (metre)
- N[north]: Northing (metre)
Area of Use:
- name: United Kingdom (UK) - offshore to boundary of UKCS within
49°45'N to 61°N and 9°W to 2°E; onshore Great Britain (England, Wa
les and Scotland). Isle of Man onshore.
- bounds: (-9.01, 49.75, 2.01, 61.01)
Coordinate Operation:
- name: British National Grid
- method: Transverse Mercator
Datum: Ordnance Survey of Great Britain 1936
- Ellipsoid: Airy 1830
- Prime Meridian: Greenwich
```

In [109…
```
# spatial index on edges
edge_sindex = edges.sindex
```

In [110…
```
# List to store distances from each POI to its nearest edge
poi_to_edge_distances = []

for poi_geom in POI_subset.geometry:

    # Get index of the nearest edge
    nearest_idx = list(edge_sindex.nearest([poi_geom], return_all=T

    # Safely get the geometry — force scalar, not Series
    if isinstance(nearest_idx, (list, np.ndarray, pd.Series)):
        nearest_idx = int(nearest_idx[0])
        nearest_edge = edges.geometry.iloc[nearest_idx]

        # Compute distance
        dist = poi_geom.distance(nearest_edge)
        poi_to_edge_distances.append(dist)
```

In [111…
```
# Convert to NumPy array for statistics
distances_array = np.array(poi_to_edge_distances)
```

In [112…
```
# Summary stats
print(f"Total POIs checked: {len(distances_array)}")
print(f"Min distance: {distances_array.min():.2f} m")
print(f"Mean distance: {distances_array.mean():.2f} m")
print(f"Median distance: {np.median(distances_array):.2f} m")
print(f"95th percentile: {np.percentile(distances_array, 95):.2f} m
print(f"Max distance: {distances_array.max():.2f} m")
```

```
Total POIs checked: 8937
Min distance: 13.15 m
Mean distance: 11240.16 m
Median distance: 10933.49 m
95th percentile: 20822.87 m
Max distance: 25646.86 m
```

The above results, especially the mean distance of 11 km, show that for some reason that couldn't be fixed, the .nearest() function doesn't work properly on this machine. So, I decided to use a two-layer buffer system to assign POIs to their nearest edge — first trying a 200 meter buffer, and then 2000 meters to include larger areas like parks. Any POIs that are still not assigned after this will be considered outliers and removed.

In [113…
```
def snap_poi_with_double_buffer(poi_geom, sindex, edges_gdf, inner_

    for radius in [inner_buffer, outer_buffer]:
        candidate_idxs = list(sindex.query(poi_geom.buffer(radius)))
        if candidate_idxs:
            candidate_edges = edges_gdf.iloc[candidate_idxs]
            distances = candidate_edges.geometry.distance(poi_geom)
            closest_idx = distances.idxmin()
            row = edges_gdf.loc[closest_idx]
            nearest_edge = row.geometry
            u = row['u']
            v = row['v']
            projected_point = nearest_edge.interpolate(nearest_edge
            return u, v, projected_point
    return None, None, None  # If nothing found
```

In [114…
```
# Store snapped distances
poi_to_edge_distances = []

for poi_geom in POI_subset.geometry:
    u, v, projected_point = snap_poi_with_double_buffer(poi_geom, e

    if projected_point is not None:
        dist = poi_geom.distance(projected_point)
        poi_to_edge_distances.append(dist)
```

In [115…
```
# Convert to NumPy array
distances_array = np.array(poi_to_edge_distances)
```

In [116…
```
# Summary stats
print(f"Total POIs snapped: {len(distances_array)}")
print(f"Min distance: {distances_array.min():.2f} m")
print(f"Mean distance: {distances_array.mean():.2f} m")
print(f"Median distance: {np.median(distances_array):.2f} m")
print(f"95th percentile: {np.percentile(distances_array, 95):.2f} m
print(f"Max distance: {distances_array.max():.2f} m")
```

```
Total POIs snapped: 8937
Min distance: 0.01 m
Mean distance: 11.82 m
Median distance: 8.26 m
95th percentile: 28.70 m
Max distance: 589.94 m
```

In [117…
```
# check the number of POIs in each subset
print("Original POIs:", len(POI_subset))
```

```
print("Successfully snapped:", len(poi_to_edge_distances))
print("Unassigned POIs:", len(POI_subset) - len(poi_to_edge_distanc
```

```
Original POIs: 8937
Successfully snapped: 8937
Unassigned POIs: 0
```

The outputs show that all POIs were assigned with sensible distance to the nearest edge.

In [119…

```
G_with_POIs = Leeds_W_Network.copy()
poi_node_ids = []

for idx, row in POI_subset.iterrows():
    u, v, projected_point = snap_poi_with_double_buffer(row.geometr

    if u is None or v is None or projected_point is None:
        continue

    poi_node = f'POI_{idx}'
    G_with_POIs.add_node(poi_node, x=projected_point.x, y=projected

    def euclidean(p1, p2): return p1.distance(p2)

    try:
        u_geom = Point(G_with_POIs.nodes[u]['x'], G_with_POIs.nodes
        v_geom = Point(G_with_POIs.nodes[v]['x'], G_with_POIs.nodes

        G_with_POIs.add_edge(poi_node, u, length=euclidean(projecte
        G_with_POIs.add_edge(poi_node, v, length=euclidean(projecte

        poi_node_ids.append(poi_node)
    except KeyError:
        continue
```

In [120…

```
# Create POI_ID list from node names in correct order
poi_node_ids = [n for n in G_with_POIs.nodes if str(n).startswith('

# Extract geometries
snapped_geoms = [G_with_POIs.nodes[n]['geometry'] for n in poi_node

# Build GeoDataFrame with POI_IDs included
gdf_snapped = gpd.GeoDataFrame({
    'POI_ID': poi_node_ids,
    'geometry': snapped_geoms
}, crs=POI_subset.crs)
```

In [121…

```
# a copy for testing graph
G_test = G_with_POIs.copy()

# Check a few POI pairs for connectivity
for i in range(0, 10, 2):  # pairs for check
    poi1 = poi_node_ids[i]
    poi2 = poi_node_ids[i + 1]
    try:
```

```
        path = nx.shortest_path(G_test, source=poi1, target=poi2, w
        print(f'Path from {poi1} to {poi2} ({len(path)} steps)')
    except nx.NetworkXNoPath:
        print(f'No path found between {poi1} and {poi2}')
```

```
Path from POI_4316 to POI_7349 (3 steps)
Path from POI_6182 to POI_7123 (9 steps)
Path from POI_3828 to POI_12456 (7 steps)
Path from POI_255 to POI_43179 (3 steps)
Path from POI_2713 to POI_36921 (5 steps)
```

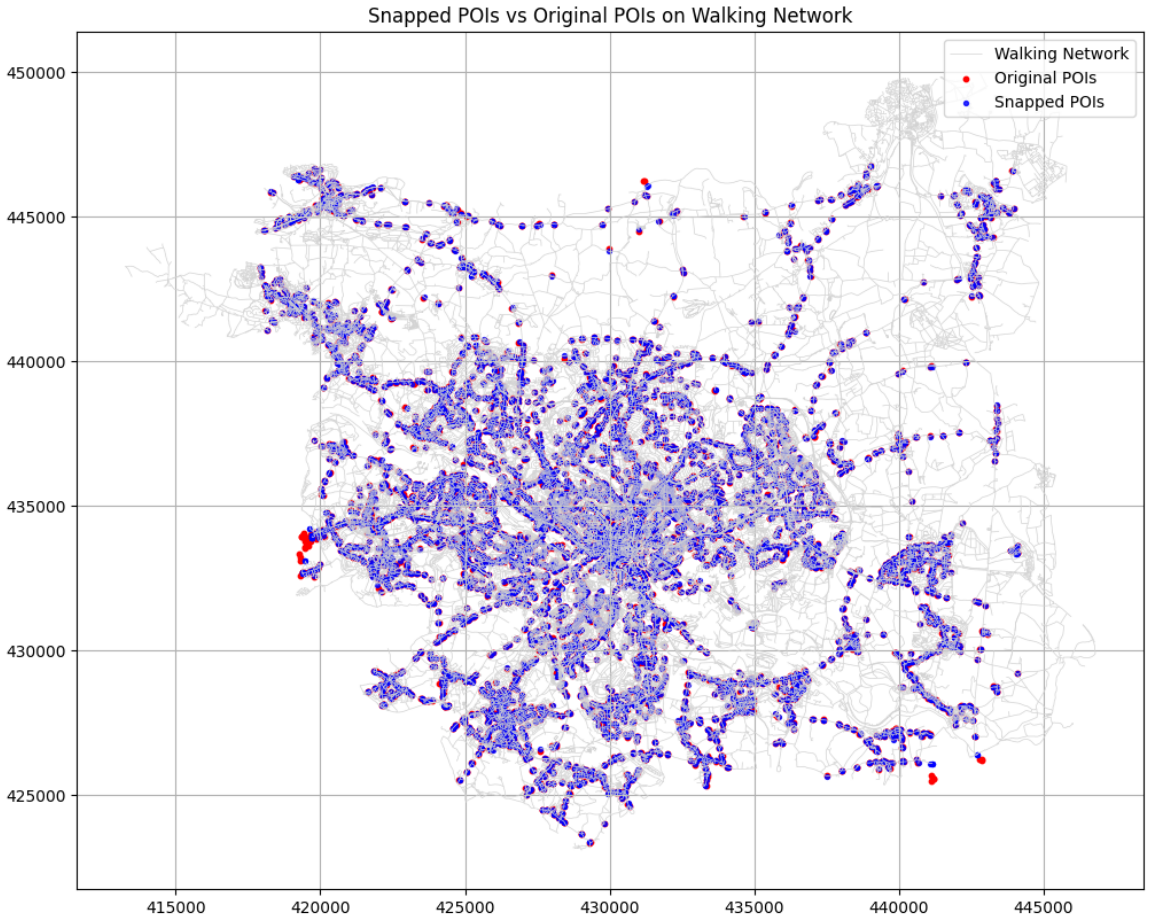
In [122…

```
# check the path length between POIs
poi = poi_node_ids[0]       # first POI for test
neighbors = list(G_test.neighbors(poi))     # neighbors

for neighbor in neighbors:
    edge_data = G_test.get_edge_data(poi, neighbor)
    print(f"Edge {poi} ↔ {neighbor}: length = {edge_data[0]['length
```

```
Edge POI_4316 ↔ 295769444: length = 20.241204244272016
Edge POI_4316 ↔ 5599859861: length = 36.79108110709183
```

In [123…

```
# Plot
fig, ax = plt.subplots(figsize=(12, 10))

# Walking network edges
edges.plot(ax=ax, color='lightgrey', linewidth=0.5, label='Walking

# Original POIs
POI_subset.plot(ax=ax, color='red', markersize=10, label='Original

# Snapped POIs
gdf_snapped.plot(ax=ax, color='blue', markersize=8, label='Snapped

# Styling
plt.title('Snapped POIs vs Original POIs on Walking Network')
plt.legend()
plt.grid(True)
plt.show()
```



In [124…
```
# Get largest connected component
largest_cc_nodes = max(nx.connected_components(G_with_POIs.to_undir

unconnected_pois = []

for poi in poi_node_ids:
    if poi not in largest_cc_nodes:
        unconnected_pois.append(poi)

print(f"Total POIs: {len(poi_node_ids)}")
print(f"Unconnected POIs: {len(unconnected_pois)}")
print(f"Connected POIs: {len(poi_node_ids) - len(unconnected_pois)}
```

```
Total POIs: 8937
Unconnected POIs: 0
Connected POIs: 8937
```

In [125…
```
"""
# a marix with only snapped pois in the new garph
distance_matrix3 = pd.DataFrame(index=poi_node_ids, columns=poi_nod
"""
```

Out[125…
```
'\n# a marix with only snapped pois in the new garph\ndistance_mat
rix3 = pd.DataFrame(index=poi_node_ids, columns=poi_node_ids, dtyp
e=float)\n'
```

In [126…
```
"""
# Fill in only upper triangle of walking distances between POIs
```

```
for i, source in enumerate(tqdm(poi_node_ids, desc='calculating dis

    # Get shortest path lengths from this source
    lengths = nx.single_source_dijkstra_path_length(G_with_POIs, so

    for j in range(i, len(poi_node_ids)):   # Loop only through upp
        target = poi_node_ids[j]
        dist = lengths[target]  # all should be reachable
        distance_matrix3.at[source, target] = dist
        distance_matrix3.at[target, source] = dist  # fill both sid
"""
```

Out[126…
```
"\n# Fill in only upper triangle of walking distances between POIs
\nfor i, source in enumerate(tqdm(poi_node_ids, desc='calculating
distance matrix')):    # Loop through POIs\n    \n    # Get shorte
st path lengths from this source\n    lengths = nx.single_source_d
ijkstra_path_length(G_with_POIs, source=source, weight='length')\n
\n    for j in range(i, len(poi_node_ids)):   # Loop only through
upper triangle\n        target = poi_node_ids[j]\n        dist = l
engths[target]  # all should be reachable\n        distance_matrix
3.at[source, target] = dist\n        distance_matrix3.at[target, s
ource] = dist  # fill both sides\n"
```

In [127…
```
"""
# testin the distance matrix results
print('Min:', distance_matrix3.min().min())
print('Max:', distance_matrix3.max().max())
print('NaN count:', distance_matrix3.isna().sum().sum())
"""
```

Out[127…
```
"\n# testin the distance matrix results\nprint('Min:', distance_ma
trix3.min().min())\nprint('Max:', distance_matrix3.max().max())\np
rint('NaN count:', distance_matrix3.isna().sum().sum())\n"
```

In [128…
```
"""
# save
distance_matrix3.to_csv('distance_matrix.csv')
"""
```

Out[128…
```
"\n# save\ndistance_matrix3.to_csv('distance_matrix.csv')\n"
```

In [129…
```
# load
df_distance = pd.read_csv('distance_matrix.csv', index_col=0)

df_distance.head()
```

Out[129…

	POI_4316	POI_7349	POI_6182	POI_7123	POI_3828	
POI_4316	0.000000	40.167708	1141.844435	812.995242	818.363784	9
POI_7349	40.167708	0.000000	1141.531479	812.682286	818.050828	9
POI_6182	1141.844435	1141.531479	0.000000	697.357109	702.725651	1
POI_7123	812.995242	812.682286	697.357109	0.000000	56.551436	1
POI_3828	818.363784	818.050828	702.725651	56.551436	0.000000	1

5 rows × 8937 columns

In [130…

```
print('Shape:', df_distance.shape)
print('Min:', df_distance.min().min())
print('Max:', df_distance.max().max())
print('Mean:', df_distance.mean().mean())
print('NaN count:', df_distance.isna().sum().sum())
```

```
Shape: (8937, 8937)
Min: 0.0
Max: 34732.60038660463
Mean: 10054.424693660572
NaN count: 0
```

In [131…

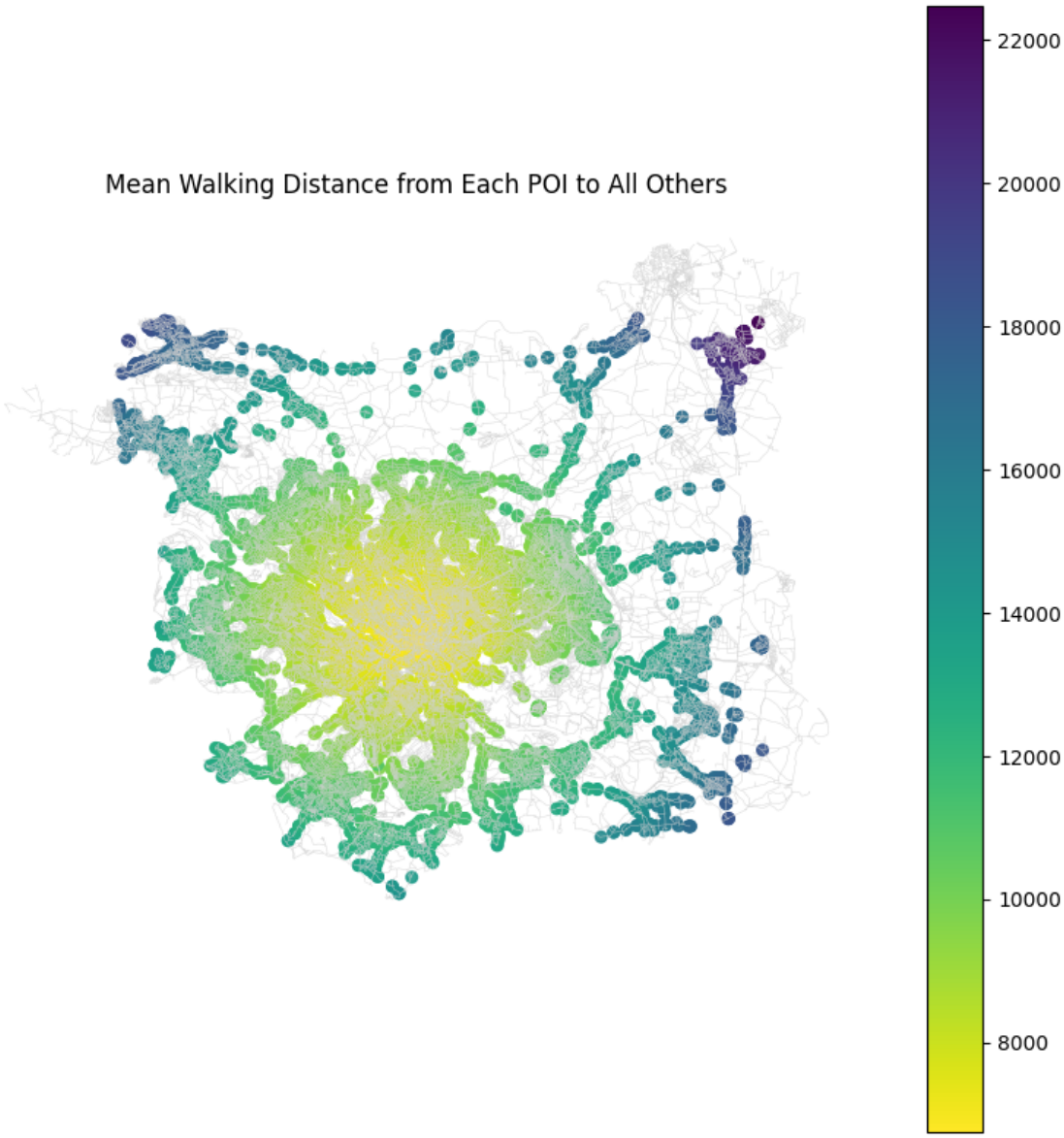
```
# Each row is a source POI, each column is a target POI
mean_distances = df_distance.mean(axis=1)  # Mean distance *from* e
mean_distances.name = 'mean_distance'
```

In [132…

```
# Extract geometries for POI nodes
snapped_geoms = [G_with_POIs.nodes[pid]['geometry'] for pid in poi_
gdf_mean_distance = gpd.GeoDataFrame({'POI_ID': poi_node_ids, 'mean
                                      geometry=snapped_geoms, crs=PO
```

In [ ]:

```
fig, ax = plt.subplots(figsize=(10, 10))

edges.plot(ax=ax, color='lightgrey', linewidth=0.3)
gdf_mean_distance.plot(ax=ax, column='mean_distance', cmap='viridis

plt.title('Mean Walking Distance from Each POI to All Others')
plt.axis('off')
plt.show()
```



Clustering and Scoring Clusters

DBSCAN Clustering Daily Amenities

In [136…
```
gdf_snapped.head()
```

Out[136…

	POI_ID	geometry
0	POI_4316	POINT (427661.502 424509.584)
1	POI_7349	POINT (427661.814 424509.56)
2	POI_6182	POINT (427125.565 425015.945)
3	POI_7123	POINT (427636.596 425254.246)
4	POI_3828	POINT (427634.609 425259.234)

In [137…
```
# extract original POI index (e.g., 4316 from 'POI_4316')
gdf_snapped['orig_index'] = gdf_snapped['POI_ID'].str.replace('POI_
```

```
gdf_snapped.head()
```

Out[137…

	POI_ID	geometry	orig_index
0	POI_4316	POINT (427661.502 424509.584)	4316
1	POI_7349	POINT (427661.814 424509.56)	7349
2	POI_6182	POINT (427125.565 425015.945)	6182
3	POI_7123	POINT (427636.596 425254.246)	7123
4	POI_3828	POINT (427634.609 425259.234)	3828

In [138…

```
POI_subset.head()
```

Out[138…

	geometry	poi_category
4316	POINT (427661 424503)	Transport
7349	POINT (427662 424512)	Transport
6182	POINT (427133 425013)	Transport
7123	POINT (427641 425256)	Transport
3828	POINT (427629 425257)	Transport

In [139…

```
POI_subset_C = POI_subset.drop(columns='geometry')

POI_subset_C.head()
```

Out[139…

	poi_category
4316	Transport
7349	Transport
6182	Transport
7123	Transport
3828	Transport

In [140…

```
# merge original POI info from POI_subset (based on original index)
gdf_snapped = gdf_snapped.merge(POI_subset_C, left_on='orig_index',

gdf_snapped.head()
```

Out[140…

	POI_ID	geometry	orig_index	poi_category
0	POI_4316	POINT (427661.502 424509.584)	4316	Transport
1	POI_7349	POINT (427661.814 424509.56)	7349	Transport
2	POI_6182	POINT (427125.565 425015.945)	6182	Transport
3	POI_7123	POINT (427636.596 425254.246)	7123	Transport
4	POI_3828	POINT (427634.609 425259.234)	3828	Transport

In [141…
```
gdf_snapped.info()
```

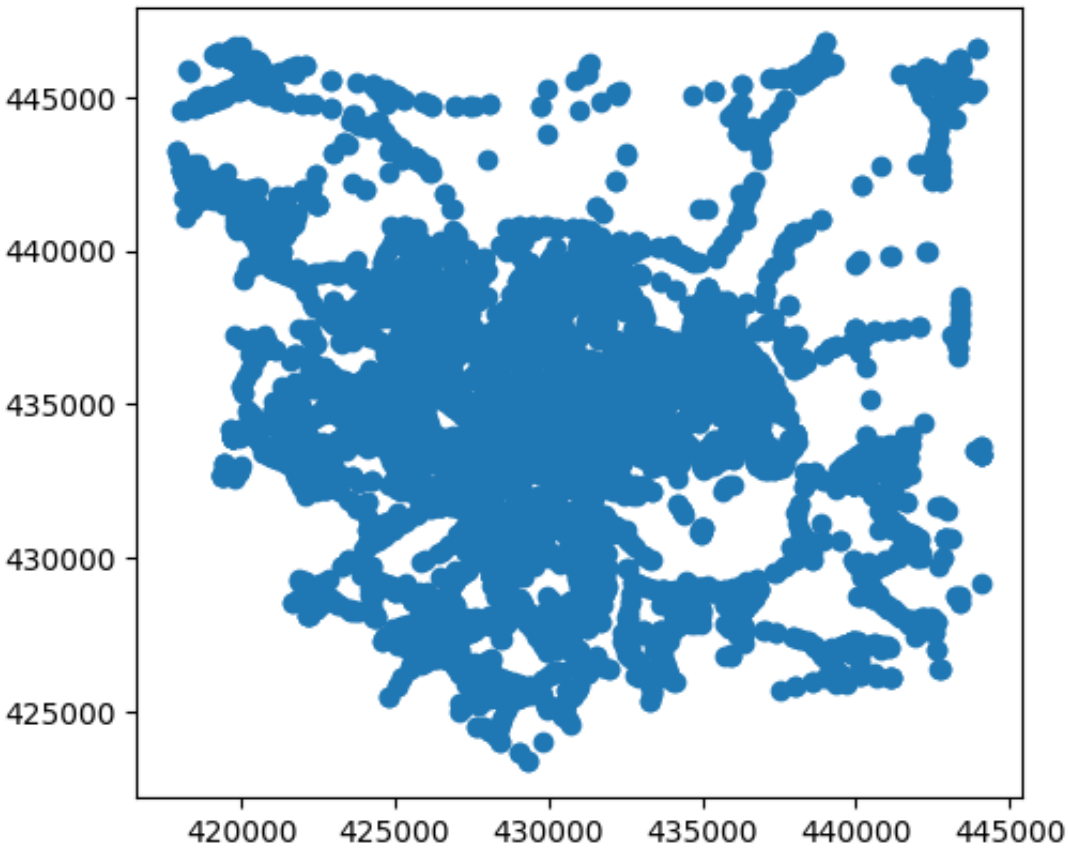
```
<class 'geopandas.geodataframe.GeoDataFrame'>
RangeIndex: 8937 entries, 0 to 8936
Data columns (total 4 columns):
 #   Column        Non-Null Count  Dtype   
---  ------        --------------  -----   
 0   POI_ID        8937 non-null   object  
 1   geometry      8937 non-null   geometry
 2   orig_index    8937 non-null   int64   
 3   poi_category  8937 non-null   object  
dtypes: geometry(1), int64(1), object(2)
memory usage: 279.4+ KB
```

In [142…
```
gdf_snapped.crs
```

Out[142…
```
<Projected CRS: EPSG:27700>
Name: OSGB36 / British National Grid
Axis Info [cartesian]:
- E[east]: Easting (metre)
- N[north]: Northing (metre)
Area of Use:
- name: United Kingdom (UK) - offshore to boundary of UKCS within
49°45'N to 61°N and 9°W to 2°E; onshore Great Britain (England, Wa
les and Scotland). Isle of Man onshore.
- bounds: (-9.01, 49.75, 2.01, 61.01)
Coordinate Operation:
- name: British National Grid
- method: Transverse Mercator
Datum: Ordnance Survey of Great Britain 1936
- Ellipsoid: Airy 1830
- Prime Meridian: Greenwich
```

In [143…
```
gdf_snapped.plot()
```

Out[143…
```
<Axes: >
```



In [144…
```
# apply the DBSCAN based on the walking distance matrix
db = DBSCAN(eps=200, min_samples=7, metric='precomputed')  # 200 is
                                                           #min of
```

In [145…
```
# fit the model
db.fit(df_distance)
```

Out[145…
```
DBSCAN(eps=200, metric='precomputed', min_samples=7)
```

In [146…
```
dbscan_labels = db.labels_
```

In [147…
```
#add the clusters as a new column
gdf_snapped['cluster'] = dbscan_labels
gdf_snapped.head()
```

Out[147…

	POI_ID	geometry	orig_index	poi_category	cluster
0	POI_4316	POINT (427661.502 424509.584)	4316	Transport	-1
1	POI_7349	POINT (427661.814 424509.56)	7349	Transport	-1
2	POI_6182	POINT (427125.565 425015.945)	6182	Transport	-1
3	POI_7123	POINT (427636.596 425254.246)	7123	Transport	0
4	POI_3828	POINT (427634.609 425259.234)	3828	Transport	0

In [148…
```
# convert cluster labels to string so they're treated as categories
gdf_snapped['dClusters_str'] = gdf_snapped['cluster'].astype(str)
```

In [149…
```
# define cluster sizes
cluster_sizes = gdf_snapped['dClusters_str'].value_counts()
```

In [150…
```
cluster_sizes.describe()
```

Out[150…
```
count     249.000000
mean       35.891566
std       233.929236
min         4.000000
25%         8.000000
50%        10.000000
75%        18.000000
max      3640.000000
Name: count, dtype: float64
```

In [151…
```
cluster_sizes.head(10)
```

Out[151…
```
dClusters_str
-1     3640
35      656
143     164
42      131
12      124
109     120
53      115
186     104
126      93
240      81
Name: count, dtype: int64
```

In [152…
```
# calculate the Silhouette Score
metrics.silhouette_score(df_distance, dbscan_labels, metric='precom
```

Out[152…
```
np.float64(-0.08380966083988473)
```

The Silhouette Score is almost zero and negative, also some clusters had

fewer than 7 POIs. Still, the DBSCAN settings were kept because they were chosen based on the goals of the analysis, not just statistics. To make the results clearer and more useful, only clusters with at least 7 POIs and POIs not marked as noise will be selected. This matches the min_samples=7 rule and helps focus on meaningful local clusters.

In [153…
```
# filter out noises
clustered_pois = gdf_snapped[gdf_snapped['cluster'] != -1]

# count cluster sizes
cluster_sizes = clustered_pois['cluster'].value_counts()

cluster_sizes
```

Out[153…
```
cluster
35     656
143    164
42     131
12     124
109    120
      ...
87       7
83       7
76       6
78       5
233      4
Name: count, Length: 248, dtype: int64
```

In [154…
```
# clusters with enough members
valid_cluster_labels = cluster_sizes[cluster_sizes >= 7].index

# filter POIs that belong to valid clusters
valid_pois = clustered_pois[clustered_pois['cluster'].isin(valid_cl
```

In [155…
```
valid_pois
```

Out [155…

	POI_ID	geometry	orig_index	poi_category	cluster	dClusters
3	POI_7123	POINT (427636.596 425254.246)	7123	Transport	0	
4	POI_3828	POINT (427634.609 425259.234)	3828	Transport	0	
5	POI_12456	POINT (427590.108 425392.657)	12456	Civic Services	0	
6	POI_255	POINT (427590.108 425392.657)	255	Civic Services	0	
7	POI_43179	POINT (427590.108 425392.657)	43179	Retail & Food	0	
…	…	…	…	…	…	
8901	POI_6389	POINT (420253.628 445728.138)	6389	Transport	240	
8902	POI_7704	POINT (420173.004 445776.814)	7704	Transport	240	
8911	POI_36831	POINT (420673.813 445714.828)	36831	Fitness & Sport	240	
8913	POI_37038	POINT (420722.259 445750.877)	37038	Fitness & Sport	240	
8922	POI_2402	POINT (420071.293 445933.293)	2402	Transport	240	

5282 rows × 6 columns

In [156…
```
valid_poi_ids = valid_pois['POI_ID']
```

In [157…
```
# a subset of the distance matrix with only valid POIs
valid_distance_matrix = df_distance.loc[valid_poi_ids, valid_poi_id
```

In [158…
```
# cluster labels
valid_clusters = valid_pois.set_index('POI_ID').loc[valid_distance_
len(valid_clusters)
```

Out [158… 5282

In [159…
```
# calculate silhouette score only on valid clusters (removed noises
```

```
round(metrics.silhouette_score(valid_distance_matrix, valid_cluster
```

Out[159… `np.float64(0.498)`

After filtering out noise and small clusters, the silhouette score improved significantly to almost 0.5. This indicates much better clusters.

In [160…
```
valid_cluster_labels_str = valid_clusters.astype(str)
```

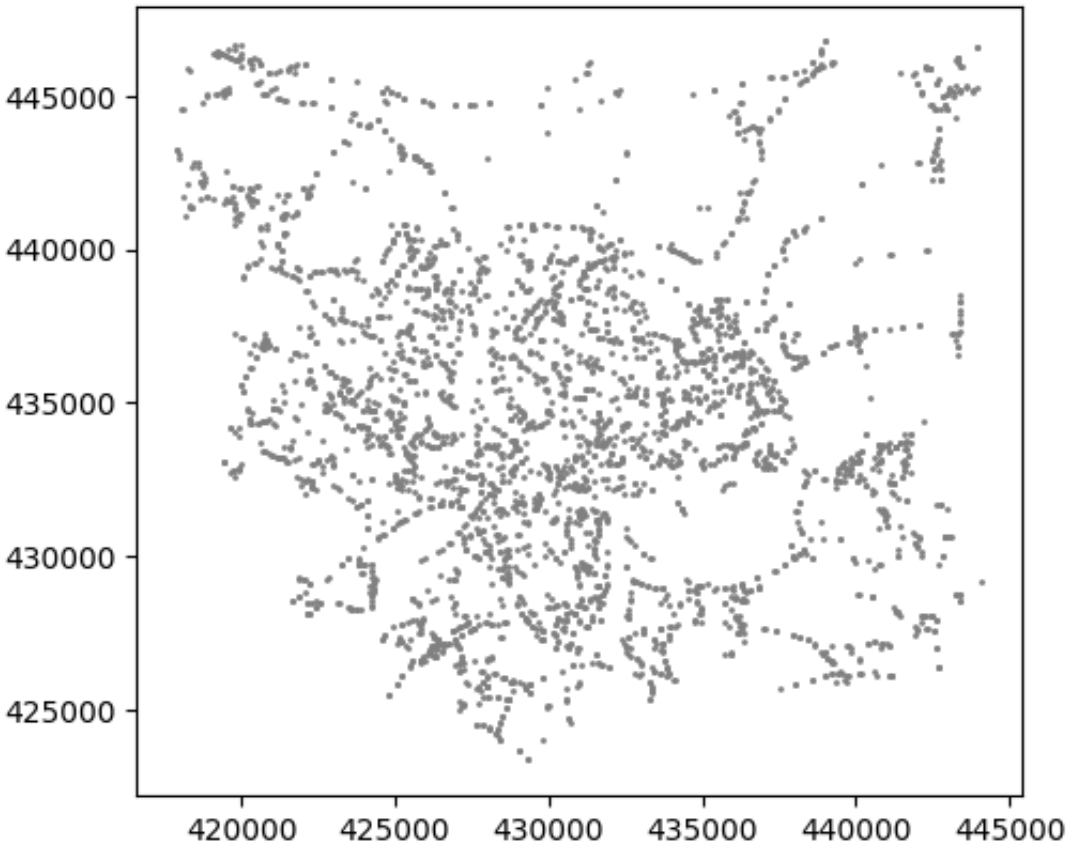
In [161…
```
not_clustered_pois = gdf_snapped[~gdf_snapped['dClusters_str'].isin
```

In [162…
```
len(not_clustered_pois)
```

Out[162… `3655`

In [163…
```
not_clustered_pois.plot(color='grey', markersize=1)
```

Out[163… `<Axes: >`



The first map displays all identified clusters (excluding noises also small clusters), each with a different color. The second map highlights the noise points: POIs that didn't meet the minimum density condition (7+ POIs within 200m). This is a significant share of the total, showing that many POIs are isolated in terms of walkable daily-access grouping. I used eps=200 to represent the idea that people might chain multiple services within a 400m roundtrip (roughly 5 minutes of walking). This setup gave me more spatially compact clusters, avoiding one giant city centre blob from earlier runs, which I

have deleted from the notebook now to reduce redundants.

Also I want to check the Sil Socre on the filtered dataset:

filtering clusters to the ones which meet every categories

In [164…
```
# group by cluster and get list of categories in each
cluster_categories = clustered_pois.groupby('dClusters_str')['poi_c

cluster_categories.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 248 entries, 0 to 247
Data columns (total 2 columns):
 #   Column         Non-Null Count  Dtype 
---  ------         --------------  ----- 
 0   dClusters_str  248 non-null    object
 1   poi_category   248 non-null    object
dtypes: object(2)
memory usage: 4.0+ KB
```

In [165…
```
# create a set of the 7 unique categories of the daily amenities
required_categories = set(POI_subset['poi_category'].unique())

required_categories
```

Out[165…
```
{'Civic Services',
 'Education & Care',
 'Fitness & Sport',
 'Green Space & Play',
 'Health',
 'Retail & Food',
 'Transport'}
```

In [166…
```
# check which clusters contain all required categories
cluster_categories['is_complete'] = cluster_categories['poi_categor
```

In [167…
```
cluster_categories.head()
```

Out[167…

	dClusters_str	poi_category	is_complete
0	0	{Retail & Food, Transport, Civic Services}	False
1	1	{Retail & Food, Transport, Civic Services}	False
2	10	{Transport}	False
3	100	{Retail & Food, Transport, Civic Services}	False
4	101	{Education & Care, Transport, Health, Retail &...	False

In [168…
```
# complete clusters
complete_cluster_ids = cluster_categories[cluster_categories['is_co
```

In [169…
```
print(complete_cluster_ids.tolist())
```

```
['110', '12', '121', '126', '143', '173', '195', '234', '27', '35',
'97']
```

So that, there is only a very limited number of clusters (10 cluster) providing all the categories of the daily services (at least one POI from each category)

In [170…
```
# filter the full POI dataset to include only complete clusters
complete_clusters = clustered_pois[clustered_pois['dClusters_str'].

complete_clusters
```

Out[170…

	POI_ID	geometry	orig_index	poi_category	cluster	dClusters
299	POI_7073	POINT (426399.833 427729.555)	7073	Health	12	
300	POI_20176	POINT (426352.218 427752.25)	20176	Health	12	
301	POI_338	POINT (426398.993 427730.994)	338	Civic Services	12	
302	POI_1965	POINT (426537.383 427733.487)	1965	Transport	12	
303	POI_15936	POINT (426467.039 427725.629)	15936	Retail & Food	12	
…	…	…	…	…	…	
8408	POI_35299	POINT (419208.207 442270.16)	35299	Health	234	
8409	POI_35083	POINT (419285.666 442266.722)	35083	Health	234	
8410	POI_35523	POINT (419320.294 442303.373)	35523	Health	234	
8411	POI_35104	POINT (419211.45 442330.72)	35104	Health	234	
8412	POI_35064	POINT (419208.098 442330.127)	35064	Health	234	

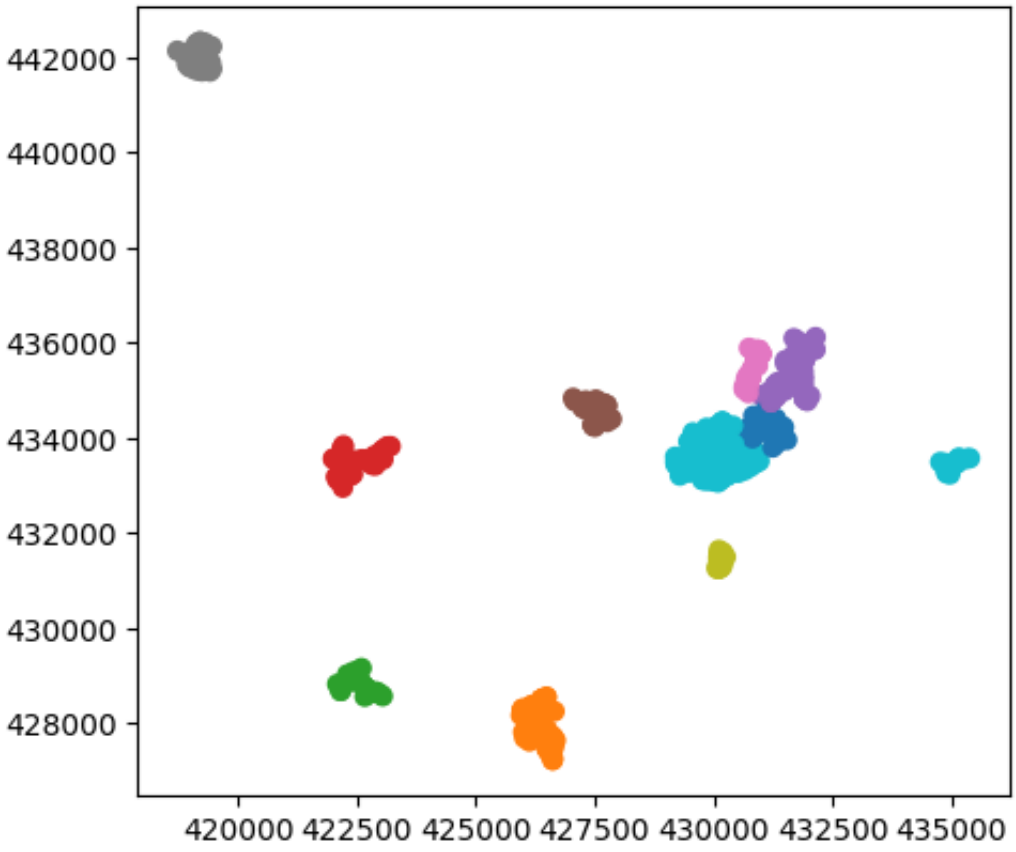
1357 rows × 6 columns

In [171…
```
complete_clusters.crs
```

Out[171…
```
<Projected CRS: EPSG:27700>
Name: OSGB36 / British National Grid
Axis Info [cartesian]:
- E[east]: Easting (metre)
- N[north]: Northing (metre)
Area of Use:
- name: United Kingdom (UK) - offshore to boundary of UKCS within
49°45'N to 61°N and 9°W to 2°E; onshore Great Britain (England, Wa
les and Scotland). Isle of Man onshore.
- bounds: (-9.01, 49.75, 2.01, 61.01)
Coordinate Operation:
- name: British National Grid
- method: Transverse Mercator
Datum: Ordnance Survey of Great Britain 1936
- Ellipsoid: Airy 1830
- Prime Meridian: Greenwich
```

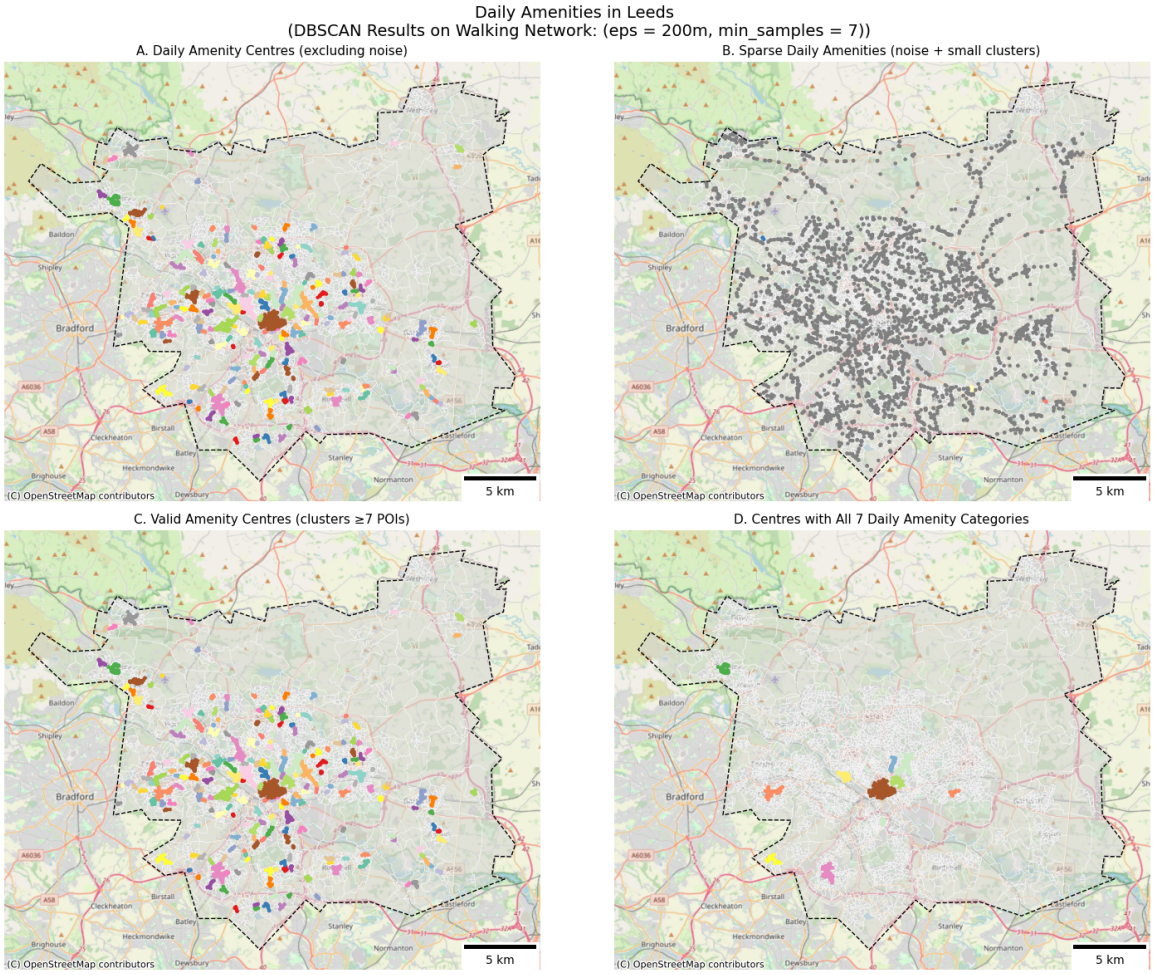
In [172…
```
complete_clusters.plot(column='dClusters_str', cmap='tab10')
```

Out[172… <Axes: >



In [173…
```
# multiple plots for the policy report

# *define fixed color mapping to be used for all cluster labels*

# all valid cluster labels as strings (excluding noises and small c
all_clusters = sorted(clustered_pois['dClusters_str'].unique(), key

# create distinct colors list to map
```

```
color_list = list(plt.cm.Set1.colors) + list(plt.cm.Set2.colors) +
color_map = {label: color_list[i % len(color_list)] for i, label in

# *subplots*
fig = plt.figure(figsize=(15, 12))
gs = GridSpec(2, 2)

axes = [
    fig.add_subplot(gs[0, 0]),  # A
    fig.add_subplot(gs[0, 1]),  # B
    fig.add_subplot(gs[1, 0]),  # C
    fig.add_subplot(gs[1, 1])   # D
]

titles = [
    'A. Daily Amenity Centres (excluding noise)',
    'B. Sparse Daily Amenities (noise + small clusters)',
    'C. Valid Amenity Centres (clusters ≥7 POIs)',
    'D. Centres with All 7 Daily Amenity Categories'
]

datasets = [clustered_pois, not_clustered_pois, valid_pois, complet

# *map*
for ax, title, data in zip(axes, titles, datasets):
    data = data.copy()
    data['cluster_color'] = data['dClusters_str'].map(color_map).fi

    # Base layers
    OA_Leeds.plot(ax=ax, color='lightgrey', edgecolor='white', line
    LeedsBou.boundary.plot(ax=ax, color='k', linestyle='dashed', li

    # POIs
    ax.scatter(data.geometry.x, data.geometry.y, color=data['cluste

    # Basemap and scale bar
    cx.add_basemap(ax, crs=27700, source=cx.providers.OpenStreetMap
    ax.add_artist(ScaleBar(dx=1, location='lower right'))

    ax.set_title(title, fontsize=11)
    ax.axis('off')

# title
fig.suptitle('Daily Amenities in Leeds \n (DBSCAN Results on Walkin

plt.tight_layout()
plt.show()
```



This figure shows the spatial distribution and clustering of daily amenities in Leeds using DBSCAN based on the walking network (eps = 200m, min_samples = 7): Figure A (Top-left): Shows all DBSCAN clusters, excluding noise points. These are interpreted as daily amenity centres. These are the places where amenities are close enough to support walkable everyday activity. Figure B (Top-right): Displays points that were not clustered, either isolated amenities (noises) or ones in small, sparse groups (<7). These areas may reflect gaps in walkability for multiple daily destinations, eventhough any of them might be in a walkable distance of any potential user. Figure C (Bottom-left): Highlights valid clusters with at least 7 POIs. The POIs in this category atr relatively dense and likely useful for local access. Figure D (Bottom-right): Shows a much smaller set of clusters that contain all 7 daily amenity categories. These represent the most complete centres but are very limited in number and don't cover the city evenly within typical walking distances (15–20 minutes).

In practice, residents often combine amenities across multiple clusters depending on preference, quality, and proximity. Therefore, while figure. C helps identify optimal diverse centres, it would be too restrictive to exclude all other clusters that may still provide valuable services. Instead, the next step will involve scoring and ranking all DBSCAN clusters based on the variety and availability of amenity types, allowing for a more inclusive and realistic

understanding of service access across Leeds:

Analysisg and socring clusters

In [174…
```
# Pivot table to count POIs in each categories per cluster
cluster_category_counts = valid_pois.pivot_table(index='cluster', c

cluster_category_counts
```

Out[174…

poi_category	cluster	Civic Services	Education & Care	Fitness & Sport	Green Space & Play	Health	Retail & Food
0	0	2	0	0	0	0	2
1	1	1	0	0	0	0	5
2	2	1	0	0	0	0	1
3	3	1	0	0	0	0	2
4	4	1	1	0	0	0	3
…	…	…	…	…	…	…	…
240	243	0	0	0	0	1	5
241	244	2	0	0	0	1	1
242	245	0	0	0	0	0	5
243	246	0	0	0	0	1	2
244	247	0	0	0	0	0	0

245 rows × 8 columns

In [175…
```
# add total no of POIs in each cluster
cluster_category_counts['total_POIs'] = cluster_category_counts.dro

cluster_category_counts
```

Out [175…

poi_category	cluster	Civic Services	Education & Care	Fitness & Sport	Green Space & Play	Health	Retail & Food
0	0	2	0	0	0	0	2
1	1	1	0	0	0	0	5
2	2	1	0	0	0	0	1
3	3	1	0	0	0	0	2
4	4	1	1	0	0	0	3
...	...	...	...	...	...	...	...
240	243	0	0	0	0	1	5
241	244	2	0	0	0	1	1
242	245	0	0	0	0	0	5
243	246	0	0	0	0	1	2
244	247	0	0	0	0	0	0

245 rows × 9 columns

In [176…
```
cluster_category_counts.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 245 entries, 0 to 244
Data columns (total 9 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   cluster             245 non-null    int64
 1   Civic Services      245 non-null    int64
 2   Education & Care    245 non-null    int64
 3   Fitness & Sport     245 non-null    int64
 4   Green Space & Play  245 non-null    int64
 5   Health              245 non-null    int64
 6   Retail & Food       245 non-null    int64
 7   Transport           245 non-null    int64
 8   total_POIs          245 non-null    int64
dtypes: int64(9)
memory usage: 17.4 KB
```

In [177…
```
# normalise each category by z-score
category_cols = [col for col in cluster_category_counts.columns if

# Apply Z-score normalization
scaler = StandardScaler()
normalised_categories = pd.DataFrame(scaler.fit_transform(cluster_c
```

In [178…
```
normalised_categories.head()
```

Out [178…

	Civic Services	Education & Care	Fitness & Sport	Green Space & Play	Health	Retail & Food	Trans
0	-0.029302	-0.511921	-0.320926	-0.432014	-0.494194	-0.267869	-0.113
1	-0.253647	-0.511921	-0.320926	-0.432014	-0.494194	-0.155300	0.065
2	-0.253647	-0.511921	-0.320926	-0.432014	-0.494194	-0.305393	-0.113
3	-0.253647	-0.511921	-0.320926	-0.432014	-0.494194	-0.267869	-0.297
4	-0.253647	0.028685	-0.320926	-0.432014	-0.494194	-0.230346	-0.113

The weights were defined using generative AI support (ChatGPT) based on general usage patterns and walkable activity chains:

In [179…

```
weights = {
    'Transport': 1.5,
    'Health': 1.5,
    'Education & Care': 1.5,
    'Retail & Food': 1.5,
    'Civic Services': 1.0,
    'Green Space & Play': 1.0,
    'Fitness & Sport': 1.0
}
```

In [180…

```
# apply weights to the z-scores
weighted_categories = normalised_categories * pd.Series(weights)
```

In [181…

```
# aggregate scores of each cluster
cluster_category_counts['weighted_score'] = weighted_categories.sum

cluster_category_counts
```

Out [181…

poi_category	cluster	Civic Services	Education & Care	Fitness & Sport	Green Space & Play	Health	Retail & Food
0	0	2	0	0	0	0	2
1	1	1	0	0	0	0	5
2	2	1	0	0	0	0	1
3	3	1	0	0	0	0	2
4	4	1	1	0	0	0	3
…	…	…	…	…	…	…	…
240	243	0	0	0	0	1	5
241	244	2	0	0	0	1	1
242	245	0	0	0	0	0	5
243	246	0	0	0	0	1	2
244	247	0	0	0	0	0	0

245 rows × 10 columns

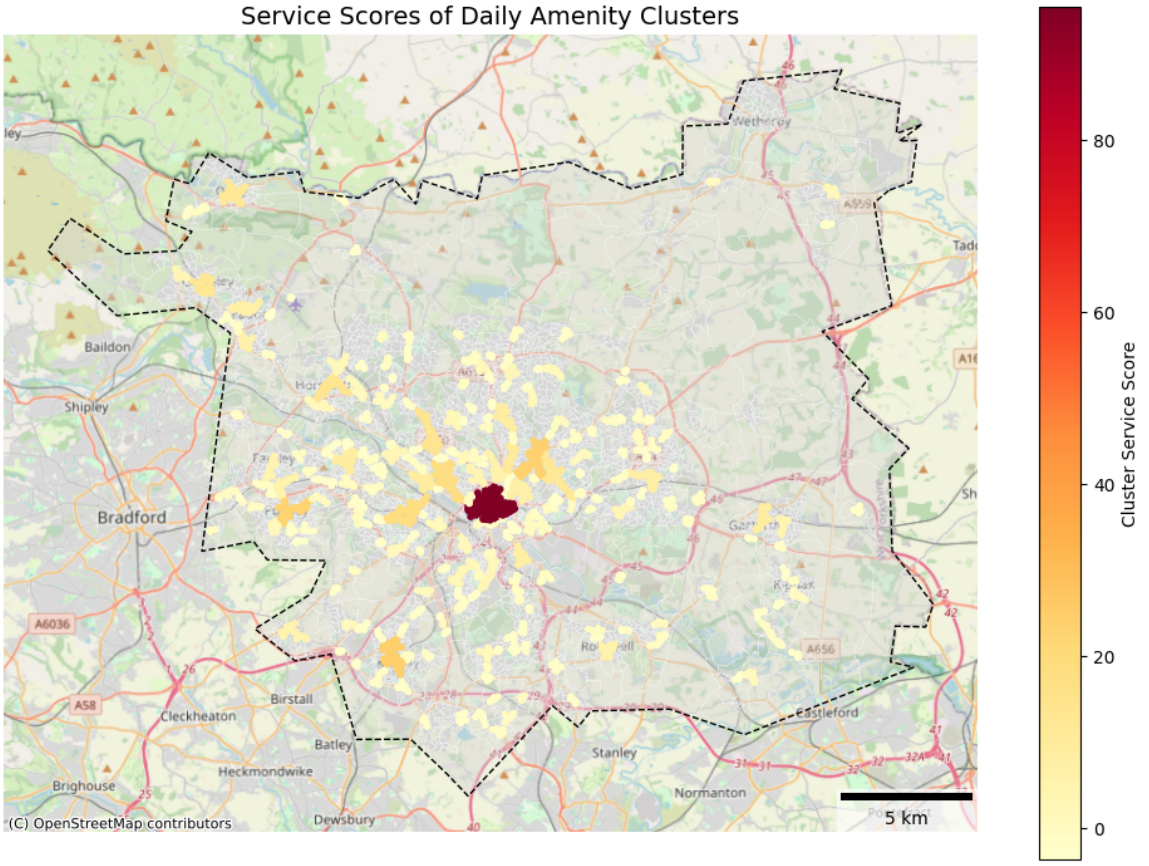
In [182…
```
# merge into valid cluster poise
valid_pois_clusters = valid_pois.merge(cluster_category_counts[['cl
```

In [183…
```
valid_pois_clusters.info()
```

```
<class 'geopandas.geodataframe.GeoDataFrame'>
RangeIndex: 5282 entries, 0 to 5281
Data columns (total 7 columns):
 #   Column          Non-Null Count  Dtype   
---  ------          --------------  -----   
 0   POI_ID          5282 non-null   object  
 1   geometry        5282 non-null   geometry
 2   orig_index      5282 non-null   int64   
 3   poi_category    5282 non-null   object  
 4   cluster         5282 non-null   int64   
 5   dClusters_str   5282 non-null   object  
 6   weighted_score  5282 non-null   float64 
dtypes: float64(1), geometry(1), int64(2), object(3)
memory usage: 289.0+ KB
```

In [184…
```
# plot
fig, ax = plt.subplots(figsize=(10, 10))

# base layers
OA_Leeds.plot(ax=ax, color='lightgrey', edgecolor='white', linewidt
LeedsBou.boundary.plot(ax=ax, color='black', linestyle='--', linewi

# valid POIs (colored by score)
valid_pois_clusters.plot(ax=ax, column='weighted_score', cmap='YlOr
```

```
# Add basemap
cx.add_basemap(ax, crs=27700, source=cx.providers.OpenStreetMap.Map

# Add scalebar
scale = ScaleBar(dx=1, location='lower right', box_alpha=0.6)
ax.add_artist(scale)

ax.set_title('Service Scores of Daily Amenity Clusters', fontsize=1
ax.axis('off')
plt.tight_layout()
plt.show()
```



Create Walking Network of Clusters Centres

To represent each service cluster, I'm selecting the most central POI based on the Closeness Centrality in the walking network, rather than using a geometric centroid.

In [185…

```
valid_pois_clusters.head()
```

Out [185…

	POI_ID	geometry	orig_index	poi_category	cluster	dClusters_str
0	POI_7123	POINT (427636.596 425254.246)	7123	Transport	0	0
1	POI_3828	POINT (427634.609 425259.234)	3828	Transport	0	0
2	POI_12456	POINT (427590.108 425392.657)	12456	Civic Services	0	0
3	POI_255	POINT (427590.108 425392.657)	255	Civic Services	0	0
4	POI_43179	POINT (427590.108 425392.657)	43179	Retail & Food	0	0

In [186…

```
# Check CRS
print(Leeds_W_Network.graph['crs'])
print(gdf_snapped.crs)
```

```
EPSG:27700
EPSG:27700
```

In [187…

```
# List to hold representative POIs
representative_pois = []

# Loop through each valid cluster label
for cluster_label in valid_pois_clusters['cluster'].unique():

    # Subset: POIs in this cluster
    cluster_pois = valid_pois_clusters[valid_pois_clusters['cluster

    # POI node names (e.g., 'POI_123')
    poi_nodes = cluster_pois['POI_ID'].tolist()

    # Get neighbors from the walking network for each POI
    subgraph_nodes = set(poi_nodes)
    for poi in poi_nodes:
        subgraph_nodes.update(G_with_POIs.neighbors(poi))

    # Create subgraph that includes POIs and their immediate walkin
    cluster_subgraph = G_with_POIs.subgraph(subgraph_nodes)

    # Compute closeness centrality using edge lengths
    centrality = nx.closeness_centrality(cluster_subgraph, distance

    # Keep only centrality values for POI nodes
    poi_centrality = {node: cent for node, cent in centrality.items

    # Find the POI node with the highest centrality
    if poi_centrality:
```

```
        best_poi_node = max(poi_centrality, key=poi_centrality.get)
        best_poi_row = cluster_pois[cluster_pois['POI_ID'] == best_
        representative_pois.append(best_poi_row)

# Combine all selected POIs into a GeoDataFrame
representative_pois_gdf = gpd.GeoDataFrame(representative_pois, geo
```

In [188…
```
representative_pois_gdf.head()
```

Out[188…

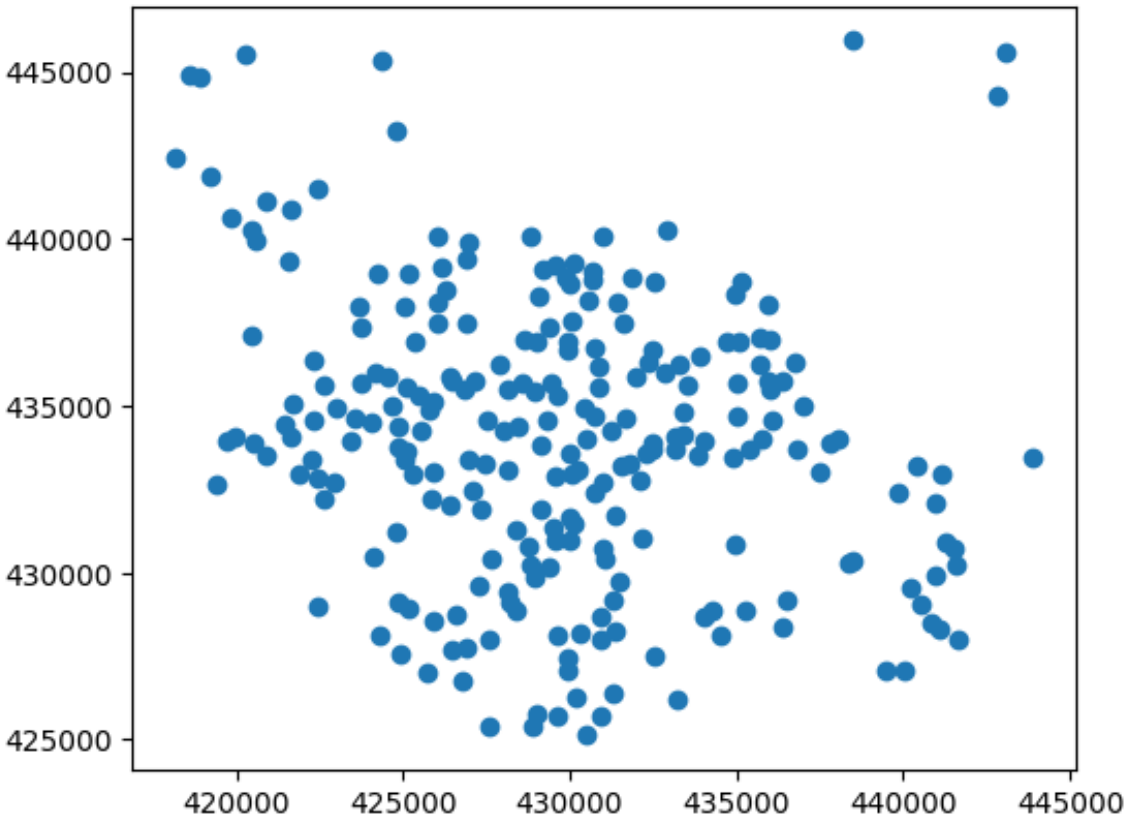
	POI_ID	geometry	orig_index	poi_category	cluster	dClusters_
3	POI_255	POINT (427590.108 425392.657)	255	Civic Services	0	
11	POI_4094	POINT (429636.469 425726.511)	4094	Transport	1	
14	POI_4307	POINT (428905.151 425406.202)	4307	Transport	2	
24	POI_19393	POINT (429026.927 425742.072)	19393	Retail & Food	3	
1395	POI_4258	POINT (430188.641 426278.336)	4258	Transport	56	

In [189…
```
representative_pois_gdf.crs
```

Out[189…
```
<Projected CRS: EPSG:27700>
Name: OSGB36 / British National Grid
Axis Info [cartesian]:
- E[east]: Easting (metre)
- N[north]: Northing (metre)
Area of Use:
- name: United Kingdom (UK) - offshore to boundary of UKCS within
49°45'N to 61°N and 9°W to 2°E; onshore Great Britain (England, Wa
les and Scotland). Isle of Man onshore.
- bounds: (-9.01, 49.75, 2.01, 61.01)
Coordinate Operation:
- name: British National Grid
- method: Transverse Mercator
Datum: Ordnance Survey of Great Britain 1936
- Ellipsoid: Airy 1830
- Prime Meridian: Greenwich
```

In [190…
```
representative_pois_gdf.plot()
```

Out[190…
```
<Axes: >
```



In [191…
```
len(representative_pois_gdf)
```

Out[191…
```
245
```

In [192…
```
# save a geojson copy
representative_pois_gdf.to_file('representative_pois_gdf.geojson',
```

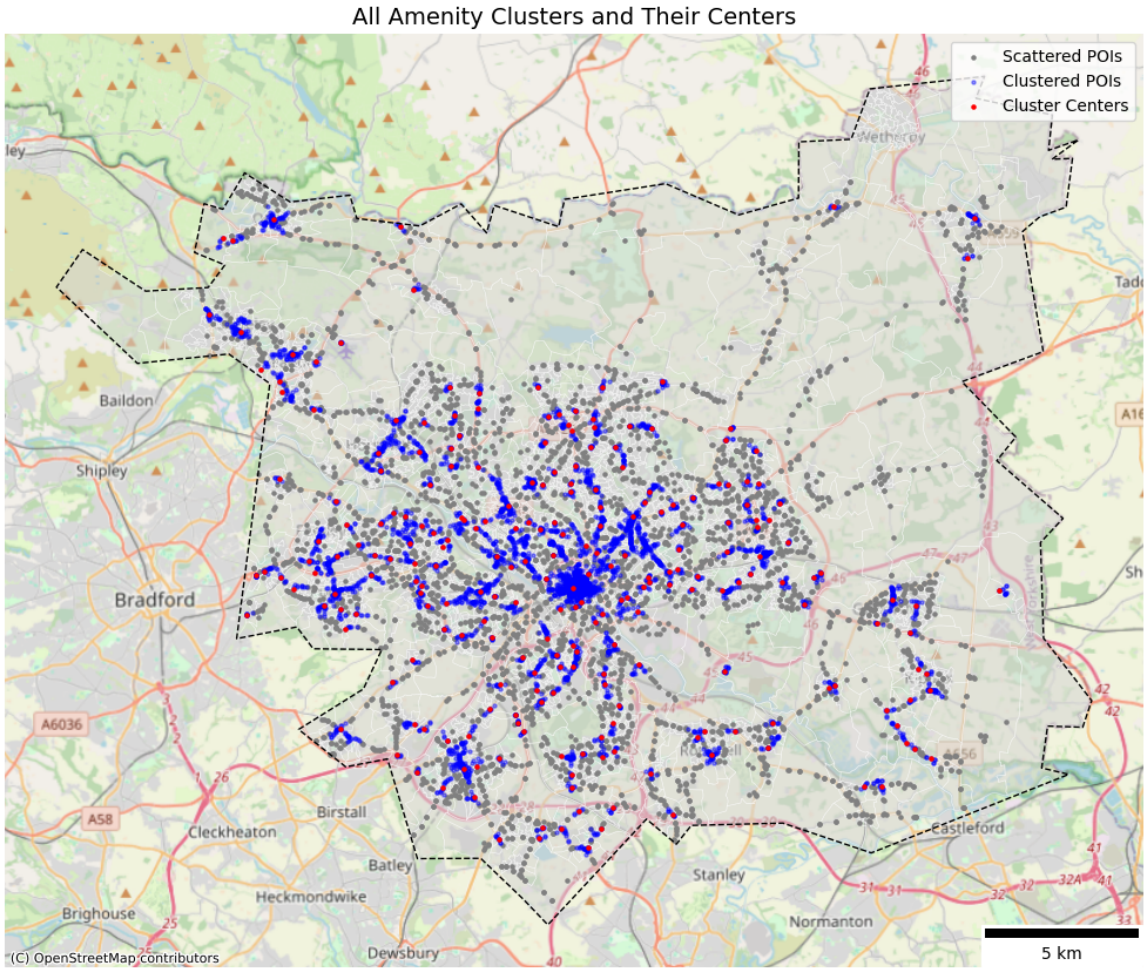
In [193…
```
# load the copy
representative_pois_gdf = gpd.read_file('representative_pois_gdf.ge

representative_pois_gdf=representative_pois_gdf.to_crs(epsg=27700)

representative_pois_gdf.crs
```

Out[193…
```
<Projected CRS: EPSG:27700>
Name: OSGB36 / British National Grid
Axis Info [cartesian]:
- E[east]: Easting (metre)
- N[north]: Northing (metre)
Area of Use:
- name: United Kingdom (UK) - offshore to boundary of UKCS within
49°45'N to 61°N and 9°W to 2°E; onshore Great Britain (England, Wa
les and Scotland). Isle of Man onshore.
- bounds: (-9.01, 49.75, 2.01, 61.01)
Coordinate Operation:
- name: British National Grid
- method: Transverse Mercator
Datum: Ordnance Survey of Great Britain 1936
- Ellipsoid: Airy 1830
- Prime Meridian: Greenwich
```

In [194…
```
representative_pois_gdf['cluster'].min()
```

Out[194…
```
np.int32(0)
```

In [195…
```
fig, ax = plt.subplots(figsize=(10, 10))

# Base layers
OA_Leeds.plot(ax=ax, color='lightgrey', edgecolor='white', linewidt
LeedsBou.boundary.plot(ax=ax, color='k', linestyle='dashed', linewi
not_clustered_pois.plot(ax=ax, color='grey', markersize=5,label='Sc

# valid clustered POIs
valid_pois_clusters.plot(ax=ax, color='blue', markersize=5, alpha=0

# cluster centres
representative_pois_gdf.plot(ax=ax, color='red', markersize=5, labe

# basemap
cx.add_basemap(ax, crs=27700, source=cx.providers.OpenStreetMap.Map

# Labels and legend
plt.title("All Amenity Clusters and Their Centers", fontsize=14)

# Add scalebar
scalebar = ScaleBar(dx=1, location='lower right')
ax.add_artist(scalebar)

plt.legend()
ax.axis('off')
plt.tight_layout()
plt.show()
```



In [196…
```
representative_pois_gdf.head()
```

Out[196…

	POI_ID	orig_index	poi_category	cluster	dClusters_str	weighted_sc
0	POI_255	255	Civic Services	0	0	-2.864
1	POI_4094	4094	Transport	1	1	-2.645(
2	POI_4307	4307	Transport	2	2	-3.144
3	POI_19393	19393	Retail & Food	3	3	-3.363
4	POI_4258	4258	Transport	56	56	-1.745

In [197…
```
representative_pois_gdf.info()
```

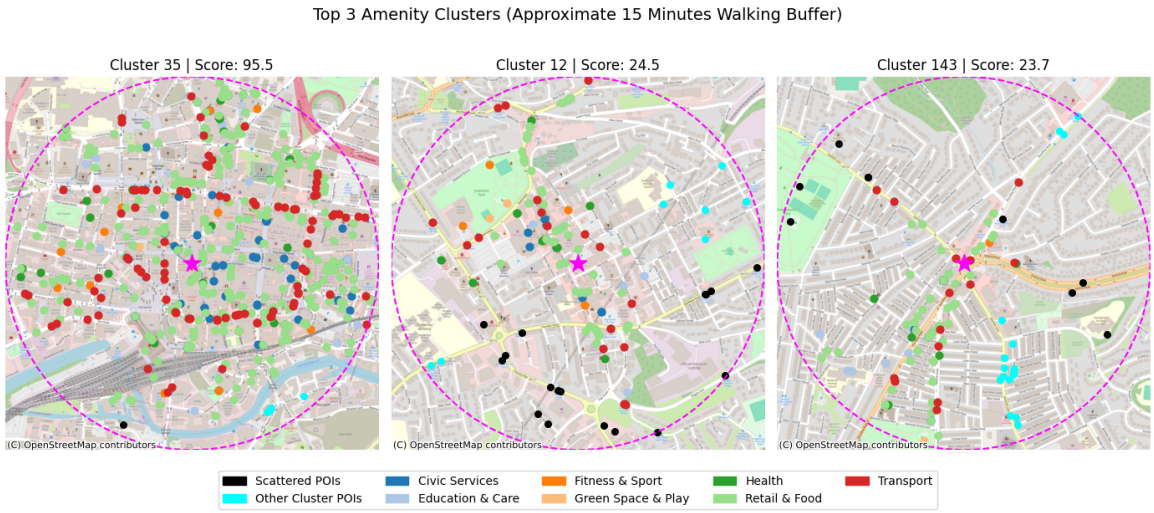
```
<class 'geopandas.geodataframe.GeoDataFrame'>
RangeIndex: 245 entries, 0 to 244
Data columns (total 7 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   POI_ID          245 non-null    object
 1   orig_index      245 non-null    int32
 2   poi_category    245 non-null    object
 3   cluster         245 non-null    int32
 4   dClusters_str   245 non-null    object
 5   weighted_score  245 non-null    float64
 6   geometry        245 non-null    geometry
dtypes: float64(1), geometry(1), int32(2), object(3)
memory usage: 11.6+ KB
```

In [198…
```
top_clusters = representative_pois_gdf.sort_values('weighted_score'
bottom_clusters = representative_pois_gdf.sort_values('weighted_sco
```

In [199…
```
# consistent categories afo all three maps

# get POI categories (sorted for consistency)
categories = sorted(valid_pois_clusters['poi_category'].unique())

# list of the colours
color_list = list(plt.cm.tab20.colors)

# map each category to a relevant color
category_colors = {cat: color_list[i] for i, cat in enumerate(categ
```

In [200…
```
# plot Top 3 Clusters
fig, axes = plt.subplots(1, 3, figsize=(14, 6.3))
axes = axes.flatten()

for i, (idx, row) in enumerate(top_clusters.iterrows()):
    ax = axes[i]

    center = row.geometry
    buffer = center.buffer(600)  # a buffer of 600 meters from cent
    buffer_gdf = gpd.GeoDataFrame(geometry=[buffer], crs=27700)

    # select only the center ranked; cluster's POIs
    pois_clip = valid_pois_clusters[valid_pois_clusters['cluster'] 

    # context
    non_clustered_clip = not_clustered_pois.clip(buffer_gdf)
    other_clusters_clip= valid_pois_clusters[valid_pois_clusters['c

    # context
    buffer_gdf.boundary.plot(ax=ax, color='#FF00FF', linestyle='--'
    non_clustered_clip .plot(ax=ax, color='#000000', markersize=30)
    other_clusters_clip.plot(ax=ax, color='#00FFFF', markersize=30)


    # POI category
    for cat in categories:
        cat_pois = pois_clip[pois_clip['poi_category'] == cat]
```

```
        if not cat_pois.empty:
            cat_pois.plot(ax=ax, color=category_colors[cat], marker

    # cluster center
    gpd.GeoDataFrame(geometry=[center], crs=27700).plot(ax=ax, colo

    # basemap
    cx.add_basemap(ax, crs=27700, source=cx.providers.OpenStreetMap

    ax.set_title(f"Cluster {row['cluster']} | Score: {round(row['we
    ax.set_xlim(buffer.bounds[0], buffer.bounds[2])
    ax.set_ylim(buffer.bounds[1], buffer.bounds[3])
    ax.axis('off')

# legend
handles = [
    mpatches.Patch(color='#000000', label='Scattered POIs'),
    mpatches.Patch(color='#00FFFF', label='Other Cluster POIs')
] + [
    mpatches.Patch(color=category_colors[cat], label=cat) for cat i
]


fig.legend(handles=handles, loc='lower center', ncol=5, fontsize=10

fig.suptitle('Top 3 Amenity Clusters (Approximate 15 Minutes Walkin
fig.tight_layout()

plt.show()
```

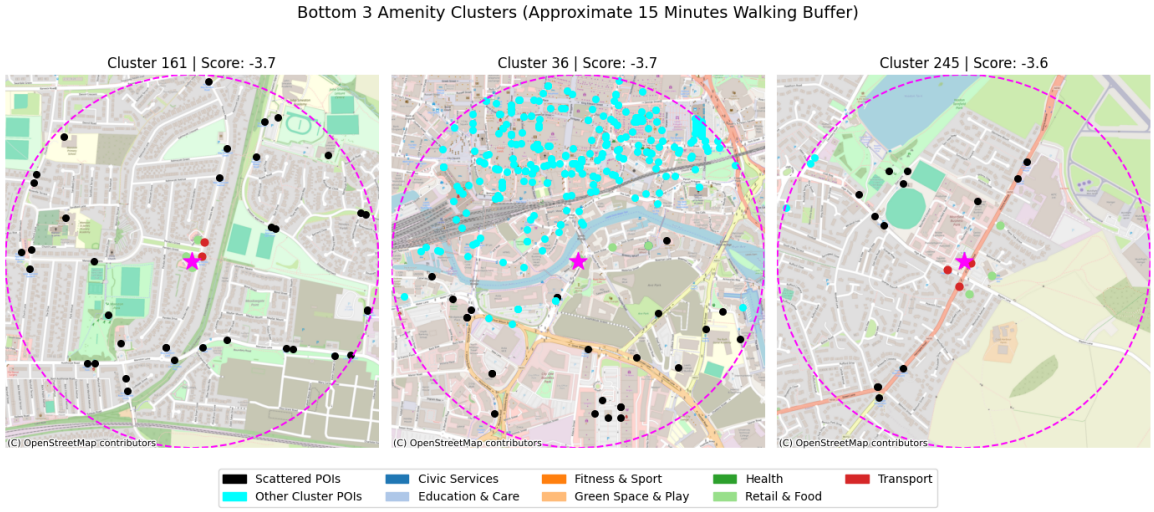


In [201…

```
# plot bottom 3 Clusters
fig, axes = plt.subplots(1, 3, figsize=(14, 6.3))
axes = axes.flatten()

for i, (idx, row) in enumerate(bottom_clusters.iterrows()):
    ax = axes[i]

    center = row.geometry
    buffer = center.buffer(600)  # a buffer of 600 meters from cent
    buffer_gdf = gpd.GeoDataFrame(geometry=[buffer], crs=27700)
```

```
    # select only the center ranked; cluster's POIs
    pois_clip = valid_pois_clusters[valid_pois_clusters['cluster'] 

    # context
    non_clustered_clip = not_clustered_pois.clip(buffer_gdf)
    other_clusters_clip= valid_pois_clusters[valid_pois_clusters['c

    # context
    buffer_gdf.boundary.plot(ax=ax, color='#FF00FF', linestyle='--'
    non_clustered_clip .plot(ax=ax, color='#000000', markersize=30)
    other_clusters_clip.plot(ax=ax, color='#00FFFF', markersize=30)

    # POI category
    for cat in categories:
        cat_pois = pois_clip[pois_clip['poi_category'] == cat]
        if not cat_pois.empty:
            cat_pois.plot(ax=ax, color=category_colors[cat], marker

    # cluster center
    gpd.GeoDataFrame(geometry=[center], crs=27700).plot(ax=ax, colo

    # basemap
    cx.add_basemap(ax, crs=27700, source=cx.providers.OpenStreetMap

    ax.set_title(f"Cluster {row['cluster']} | Score: {round(row['we
    ax.set_xlim(buffer.bounds[0], buffer.bounds[2])
    ax.set_ylim(buffer.bounds[1], buffer.bounds[3])
    ax.axis('off')

# legend
handles = [
    mpatches.Patch(color='#000000', label='Scattered POIs'),
    mpatches.Patch(color='#00FFFF', label='Other Cluster POIs')
] + [
    mpatches.Patch(color=category_colors[cat], label=cat) for cat i
]


fig.legend(handles=handles, loc='lower center', ncol=5, fontsize=10

fig.suptitle('Bottom 3 Amenity Clusters (Approximate 15 Minutes Wal
fig.tight_layout()

plt.show()
```

```
/var/folders/wd/1wf586s93vg1fbnl295w_w1r0000gn/T/ipykernel_64251/259
6605463.py:22: UserWarning: The GeoDataFrame you are attempting to p
lot is empty. Nothing has been displayed.
  other_clusters_clip.plot(ax=ax, color='#00FFFF', markersize=30)
```



Walking Network of Service Centers and Residential Centreoids

Create the network

In [202…
```
combined_centroids.head()
```

Out[202…

	OA21CD	population	geometry	source
0	E00058165	36.333333	POINT (429433.352 423049.885)	Residential Footprint Centroid
1	E00058165	36.333333	POINT (429301.237 423372.278)	Residential Footprint Centroid
2	E00058165	36.333333	POINT (429343.656 423416.469)	Residential Footprint Centroid
3	E00058165	36.333333	POINT (428853.124 423693.511)	Residential Footprint Centroid
4	E00058165	36.333333	POINT (428954.448 423741.087)	Residential Footprint Centroid

In [203…
```
combined_centroids.info()
```

```
<class 'geopandas.geodataframe.GeoDataFrame'>
RangeIndex: 6484 entries, 0 to 6483
Data columns (total 4 columns):
 #   Column      Non-Null Count  Dtype   
---  ------      --------------  -----   
 0   OA21CD      6484 non-null   object  
 1   population  6484 non-null   float64 
 2   geometry    6484 non-null   geometry
 3   source      6484 non-null   object  
dtypes: float64(1), geometry(1), object(2)
memory usage: 202.8+ KB
```

In [204…
```
combined_centroids.crs
```

Out[204…
```
<Projected CRS: EPSG:27700>
Name: OSGB36 / British National Grid
Axis Info [cartesian]:
- E[east]: Easting (metre)
- N[north]: Northing (metre)
Area of Use:
- name: United Kingdom (UK) - offshore to boundary of UKCS within
49°45'N to 61°N and 9°W to 2°E; onshore Great Britain (England, Wa
les and Scotland). Isle of Man onshore.
- bounds: (-9.01, 49.75, 2.01, 61.01)
Coordinate Operation:
- name: British National Grid
- method: Transverse Mercator
Datum: Ordnance Survey of Great Britain 1936
- Ellipsoid: Airy 1830
- Prime Meridian: Greenwich
```

In [205…
```
G_with_POIs_centroids = G_with_POIs.copy()
```

In [206…
```
edges.crs
```

Out[206…
```
<Projected CRS: EPSG:27700>
Name: OSGB36 / British National Grid
Axis Info [cartesian]:
- E[east]: Easting (metre)
- N[north]: Northing (metre)
Area of Use:
- name: United Kingdom (UK) - offshore to boundary of UKCS within
49°45'N to 61°N and 9°W to 2°E; onshore Great Britain (England, Wa
les and Scotland). Isle of Man onshore.
- bounds: (-9.01, 49.75, 2.01, 61.01)
Coordinate Operation:
- name: British National Grid
- method: Transverse Mercator
Datum: Ordnance Survey of Great Britain 1936
- Ellipsoid: Airy 1830
- Prime Meridian: Greenwich
```

In [207…
```
res_node_ids = []

for idx, row in combined_centroids.iterrows():
    u, v, projected_point = snap_poi_with_double_buffer(row.geometr

    if u is None or v is None or projected_point is None:
        continue

    res_node = f'CENTROID_{idx}'
    G_with_POIs_centroids.add_node(res_node, x=projected_point.x, y

    try:
        u_geom = Point(G_with_POIs_centroids.nodes[u]['x'], G_with_
        v_geom = Point(G_with_POIs_centroids.nodes[v]['x'], G_with_

        G_with_POIs_centroids.add_edge(res_node, u, length=projecte
        G_with_POIs_centroids.add_edge(res_node, v, length=projecte
```

```
            res_node_ids.append((res_node, row['OA21CD'], row['populati
        except KeyError:
            continue
```

In [208…
```
print(len(G_with_POIs_centroids.nodes))
print(len(G_with_POIs_centroids.edges))
```

```
110013
153891
```

In [209…
```
# Get all centroid node IDs
snapped_centroid_node = [n for n in G_with_POIs_centroids.nodes if

# Get largest connected component
largest_cc_nodes_centroids = max(nx.connected_components(G_with_POI

# Find unconnected centroids
unconnected_centroids = [n for n in snapped_centroid_node if n not

print(f"Total Centroids: {len(snapped_centroid_node)}")
print(f"Unconnected Centroids: {len(unconnected_centroids)}")
print(f"Connected Centroids: {len(snapped_centroid_node) - len(unco
```

```
Total Centroids: 6484
Unconnected Centroids: 0
Connected Centroids: 6484
```

In [210…
```
# Extract geometries
snapped_centroid_geoms = [G_with_POIs_centroids.nodes[n]['geometry'

# Build GeoDataFrame
gdf_centroids_snapped = gpd.GeoDataFrame({
    'Centroid_ID': snapped_centroid_node,
    'geometry': snapped_centroid_geoms
}, crs=edges.crs)
```

In [211…
```
gdf_centroids_snapped.head()
```

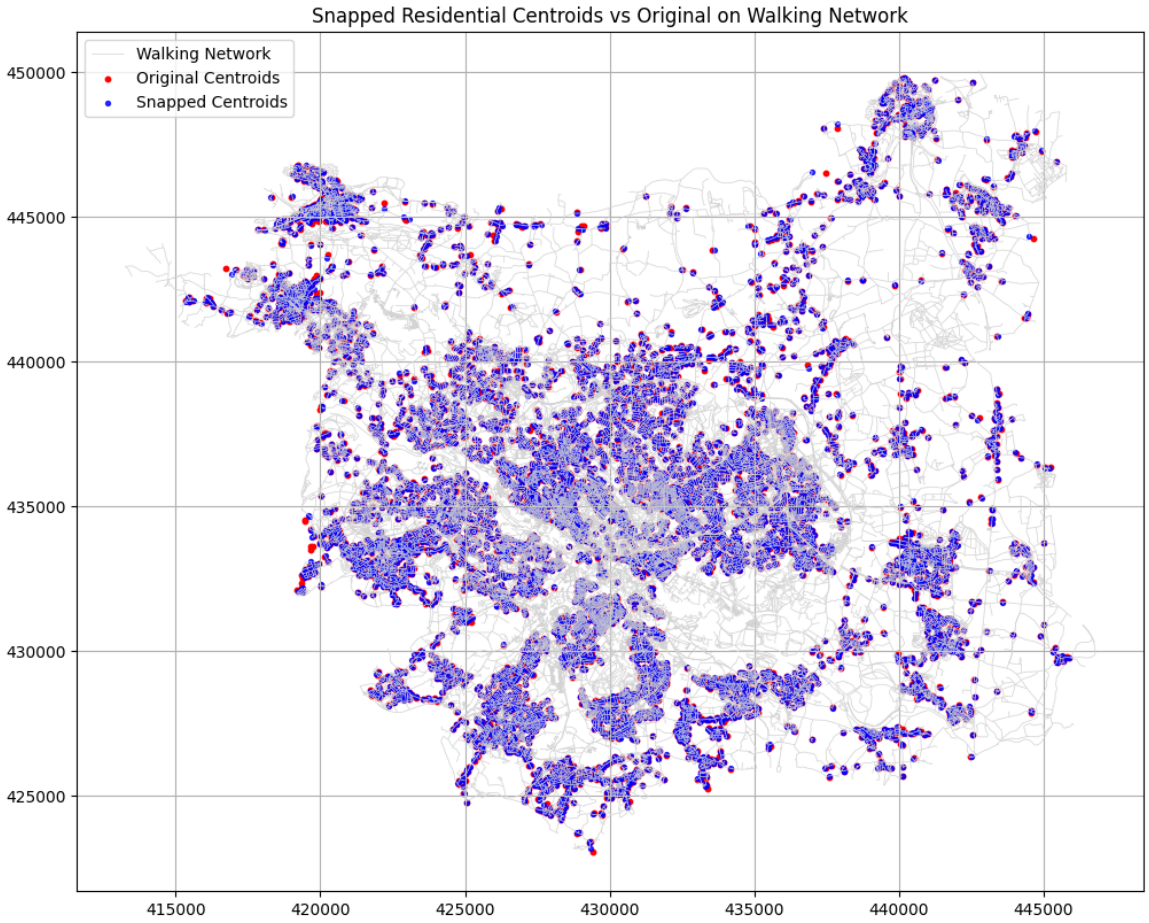
Out[211…

	Centroid_ID	geometry
0	CENTROID_0	POINT (429327.81 423196.799)
1	CENTROID_1	POINT (429316.077 423375.712)
2	CENTROID_2	POINT (429308.779 423399.26)
3	CENTROID_3	POINT (428870.995 423724.067)
4	CENTROID_4	POINT (428948.304 423748.19)

In [212…
```
gdf_centroids_snapped.crs
```

Out[212…
```
<Projected CRS: EPSG:27700>
Name: OSGB36 / British National Grid
Axis Info [cartesian]:
- E[east]: Easting (metre)
- N[north]: Northing (metre)
Area of Use:
- name: United Kingdom (UK) - offshore to boundary of UKCS within
49°45'N to 61°N and 9°W to 2°E; onshore Great Britain (England, Wa
les and Scotland). Isle of Man onshore.
- bounds: (-9.01, 49.75, 2.01, 61.01)
Coordinate Operation:
- name: British National Grid
- method: Transverse Mercator
Datum: Ordnance Survey of Great Britain 1936
- Ellipsoid: Airy 1830
- Prime Meridian: Greenwich
```

In [213…
```
fig, ax = plt.subplots(figsize=(12, 10))

# Walking network
edges.plot(ax=ax, color='lightgrey', linewidth=0.5, label='Walking

# Original centroids
combined_centroids.plot(ax=ax, color='red', markersize=10, label='O

# Snapped centroids
gdf_centroids_snapped.plot(ax=ax, color='blue', markersize=8, label

plt.title('Snapped Residential Centroids vs Original on Walking Net
plt.legend()
plt.grid(True)
plt.show()
```



In [214…
```
# Sample centroid
sample_centroid = snapped_centroid_node[0]
neighbors = list(G_with_POIs_centroids.neighbors(sample_centroid))

for neighbor in neighbors:
    edge_data = G_with_POIs_centroids.get_edge_data(sample_centroid
    print(f"Edge {sample_centroid} ↔ {neighbor}: length = {edge_dat
```

```
Edge CENTROID_0 ↔ 5771424547: length = 148.1306780665397
Edge CENTROID_0 ↔ 5771443785: length = 0.0
```

In [215…
```
G_test_centroids = G_with_POIs_centroids.copy()

for i in range(0, min(10, len(snapped_centroid_node) - 1), 2):
    c1 = snapped_centroid_node[i]
    c2 = snapped_centroid_node[i + 1]
    try:
        path = nx.shortest_path(G_test_centroids, source=c1, target
        print(f'Path from {c1} to {c2} ({len(path)} steps)')
    except nx.NetworkXNoPath:
        print(f'No path found between {c1} and {c2}')
```

```
Path from CENTROID_0 to CENTROID_1 (3 steps)
Path from CENTROID_2 to CENTROID_3 (5 steps)
Path from CENTROID_4 to CENTROID_5 (10 steps)
Path from CENTROID_6 to CENTROID_7 (3 steps)
Path from CENTROID_8 to CENTROID_9 (23 steps)
```

Walking distance from each Residential to the Service Centre

calculating distance from each centreoid to the clusters representative centre within 2000 (m)

In [216…
```
# check the graph being undirected
G_with_POIs_centroids.is_directed()
```

Out[216… False

In [217…
```
"""""""
# calculate the distance from each residential centroid to the clus
#(far beyond walking distance threshold, to ensure every centreoid

#set of representative POIs and their scores
poi_scores = representative_pois_gdf.set_index('POI_ID')['weighted_
rep_poi_set = set(poi_scores.keys())

re_ce_distance_records = []

for snapped_centroid_node, oa_code, population, source in tqdm(res_
    # Get all reachable nodes within 4000m
    lengths = nx.single_source_dijkstra_path_length(
        G_with_POIs_centroids,
        source=snapped_centroid_node,
        weight='length',
        cutoff=4000
    )

    # Check if target is a representative POI
    for poi_node, dist in lengths.items():
        if poi_node in rep_poi_set:
            re_ce_distance_records.append({
                'centroid_node': snapped_centroid_node,
                'OA21CD': oa_code,
                'population': population,
                'source': source,
                'POI_ID': poi_node,
                'cluster_score': poi_scores[poi_node],
                'distance': dist
            })
"""""""
```

Out[217…
```
'"""\n# calculate the distance from each residential centroid to th
e cluster representatives within 4000m:\n#(far beyond walking dist
ance threshold, to ensure every centreoid reaches at least one ser
vice centre/ also limiting the number of calculations)\n\n#set of
representative POIs and their scores\npoi_scores = representative_
pois_gdf.set_index(\'POI_ID\')[\'weighted_score\'].to_dict()\nrep_
poi_set = set(poi_scores.keys())\n\nre_ce_distance_records = []\n\
nfor snapped_centroid_node, oa_code, population, source in tqdm(re
s_node_ids, desc=\'Computing re-ce distances\'):\n    # Get all re
achable nodes within 4000m\n    lengths = nx.single_source_dijkstr
a_path_length(\n        G_with_POIs_centroids,\n        source=sna
pped_centroid_node,\n        weight=\'length\',\n        cutoff=40
00\n    )\n\n    # Check if target is a representative POI\n    fo
r poi_node, dist in lengths.items():\n        if poi_node in rep_p
oi_set:\n            re_ce_distance_records.append({\n
\'centroid_node\': snapped_centroid_node,\n                \'OA21C
D\': oa_code,\n                \'population\': population,\n
\'source\': source,\n                \'POI_ID\': poi_node,\n
\'cluster_score\': poi_scores[poi_node],\n                \'distan
ce\': dist\n            })\n'
```

In [218…
```
"""""""
# convert to dataframe
re_ce_df = pd.DataFrame(re_ce_distance_records)

# save the data frame
re_ce_df.to_csv('re_ce_distances.csv', index=False)

"""""""
```

Out[218…
```
'"""\n# convert to dataframe\nre_ce_df = pd.DataFrame(re_ce_distanc
e_records)\n\n# save the data frame\nre_ce_df.to_csv(\'re_ce_dista
nces.csv\', index=False)\n\n'
```

In [219…
```
# load dataframe
re_ce_df = pd.read_csv('re_ce_distances.csv')
```

In [220…
```
re_ce_df.head()
```

Out [220…

	centroid_node	OA21CD	population	source	POI_ID	cluster_sc
0	CENTROID_0	E00058165	36.333333	Residential Footprint Centroid	POI_42282	-1.504(
1	CENTROID_0	E00058165	36.333333	Residential Footprint Centroid	POI_4094	-2.645(
2	CENTROID_0	E00058165	36.333333	Residential Footprint Centroid	POI_4307	-3.1447
3	CENTROID_0	E00058165	36.333333	Residential Footprint Centroid	POI_12456	-2.864
4	CENTROID_0	E00058165	36.333333	Residential Footprint Centroid	POI_19393	-3.3631

In [221…
```
re_ce_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 144824 entries, 0 to 144823
Data columns (total 7 columns):
 #   Column         Non-Null Count   Dtype  
---  ------         --------------   -----  
 0   centroid_node  144824 non-null  object 
 1   OA21CD         144824 non-null  object 
 2   population     144824 non-null  float64
 3   source         144824 non-null  object 
 4   POI_ID         144824 non-null  object 
 5   cluster_score  144824 non-null  float64
 6   distance       144824 non-null  float64
dtypes: float64(3), object(4)
memory usage: 7.7+ MB
```

In [222…
```
re_ce_df['sc/dis'] = re_ce_df['cluster_score'] / re_ce_df['distance
```

In [223…
```
optimum_res_cluster = re_ce_df.loc[re_ce_df.groupby('centroid_node'
```

In [224…
```
optimum_res_cluster.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6289 entries, 0 to 6288
Data columns (total 8 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   centroid_node  6289 non-null   object
 1   OA21CD         6289 non-null   object
 2   population     6289 non-null   float64
 3   source         6289 non-null   object
 4   POI_ID         6289 non-null   object
 5   cluster_score  6289 non-null   float64
 6   distance       6289 non-null   float64
 7   sc/dis         6289 non-null   float64
dtypes: float64(4), object(4)
memory usage: 393.2+ KB
```

In [225…
```
optimum_res_cluster.head()
```

Out[225…

	centroid_node	OA21CD	population	source	POI_ID	cluster_s
0	CENTROID_0	E00058165	36.333333	Residential Footprint Centroid	POI_7664	-0.65
1	CENTROID_1	E00058165	36.333333	Residential Footprint Centroid	POI_7618	-0.37
2	CENTROID_10	E00058165	36.333333	Residential Footprint Centroid	POI_7664	-0.65
3	CENTROID_100	E00058668	51.125000	Residential Footprint Centroid	POI_7621	5.33
4	CENTROID_1000	E00057494	96.666667	Residential Footprint Centroid	POI_29497	7.73

In [226…
```
gdf_centroids_snapped.head()
```

Out[226…

	Centroid_ID	geometry
0	CENTROID_0	POINT (429327.81 423196.799)
1	CENTROID_1	POINT (429316.077 423375.712)
2	CENTROID_2	POINT (429308.779 423399.26)
3	CENTROID_3	POINT (428870.995 423724.067)
4	CENTROID_4	POINT (428948.304 423748.19)

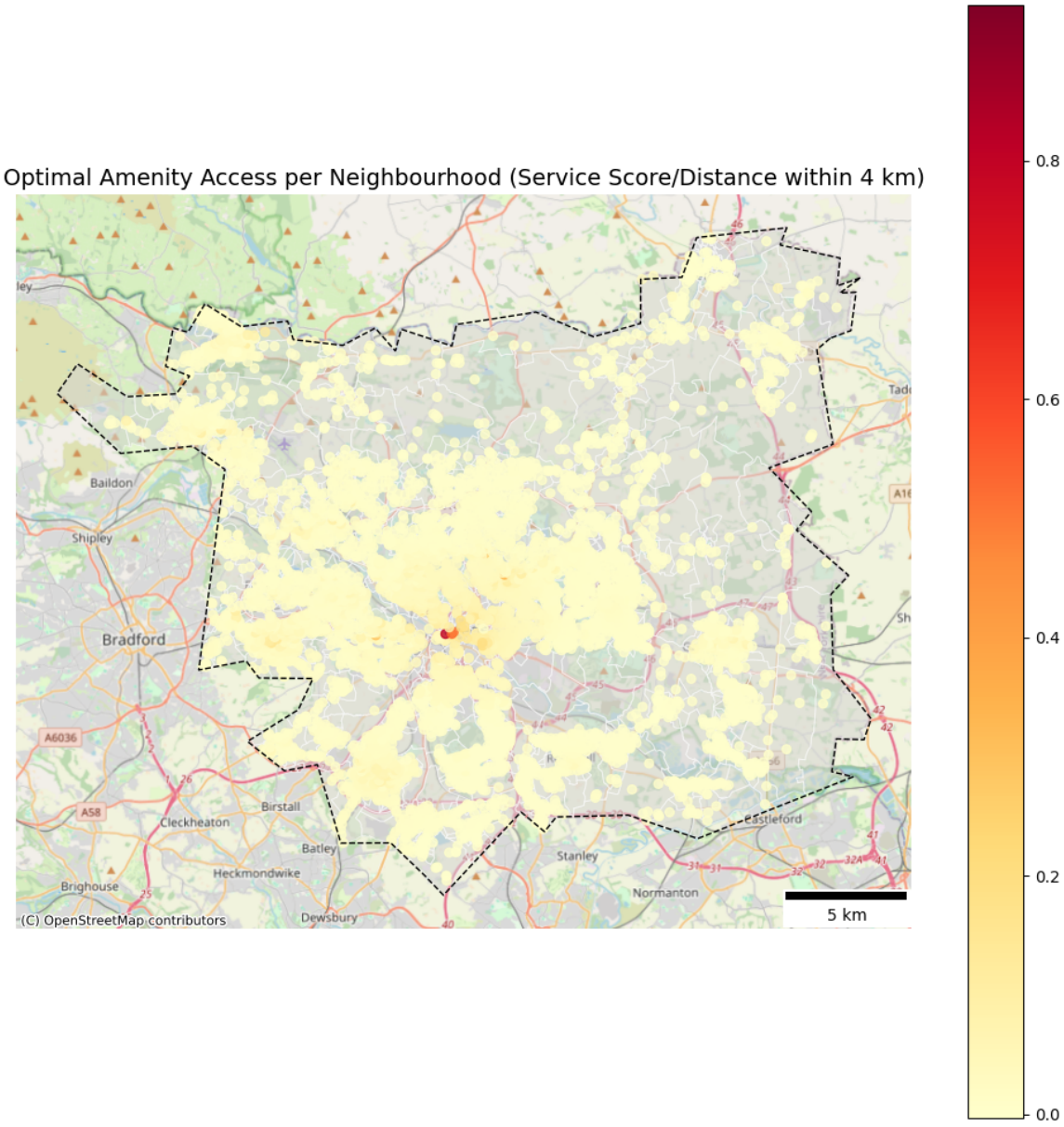
In [227…
```
# merge with centroid gdf
Re_Se = gdf_centroids_snapped.merge(
    optimum_res_cluster[['centroid_node', 'cluster_score', 'distanc
```

```
    left_on='Centroid_ID',
    right_on='centroid_node',
    how='left'
)
```

In [228…
```
Re_Se.info()
```

```
<class 'geopandas.geodataframe.GeoDataFrame'>
RangeIndex: 6484 entries, 0 to 6483
Data columns (total 8 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   Centroid_ID    6484 non-null   object
 1   geometry       6484 non-null   geometry
 2   centroid_node  6289 non-null   object
 3   cluster_score  6289 non-null   float64
 4   distance       6289 non-null   float64
 5   POI_ID         6289 non-null   object
 6   sc/dis         6289 non-null   float64
 7   population     6289 non-null   float64
dtypes: float64(4), geometry(1), object(3)
memory usage: 405.4+ KB
```

In [229…
```
fig, ax = plt.subplots(figsize=(10, 10))

# Base layers
OA_Leeds.plot(ax=ax, color='lightgrey', edgecolor='white', linewidt
LeedsBou.boundary.plot(ax=ax, color='k', linestyle='dashed', linewi

# optimal service-centres
Re_Se.plot(column='sc/dis', cmap='YlOrRd', legend=True, ax=ax, mark

# basemap
cx.add_basemap(ax, crs=27700, source=cx.providers.OpenStreetMap.Map

# Labels and legend
plt.title('Optimal Amenity Access per Neighbourhood (Service Score/

# Add scalebar
scalebar = ScaleBar(dx=1, location='lower right')
ax.add_artist(scalebar)


ax.axis('off')
plt.tight_layout()
plt.show()
```



In [230…
```
# average walking speed
walking_speed = 75  # meters per minute
```

In [231…
```
# walking time (minutes)
Re_Se['walk_time_min'] = Re_Se['distance'] / 75
```

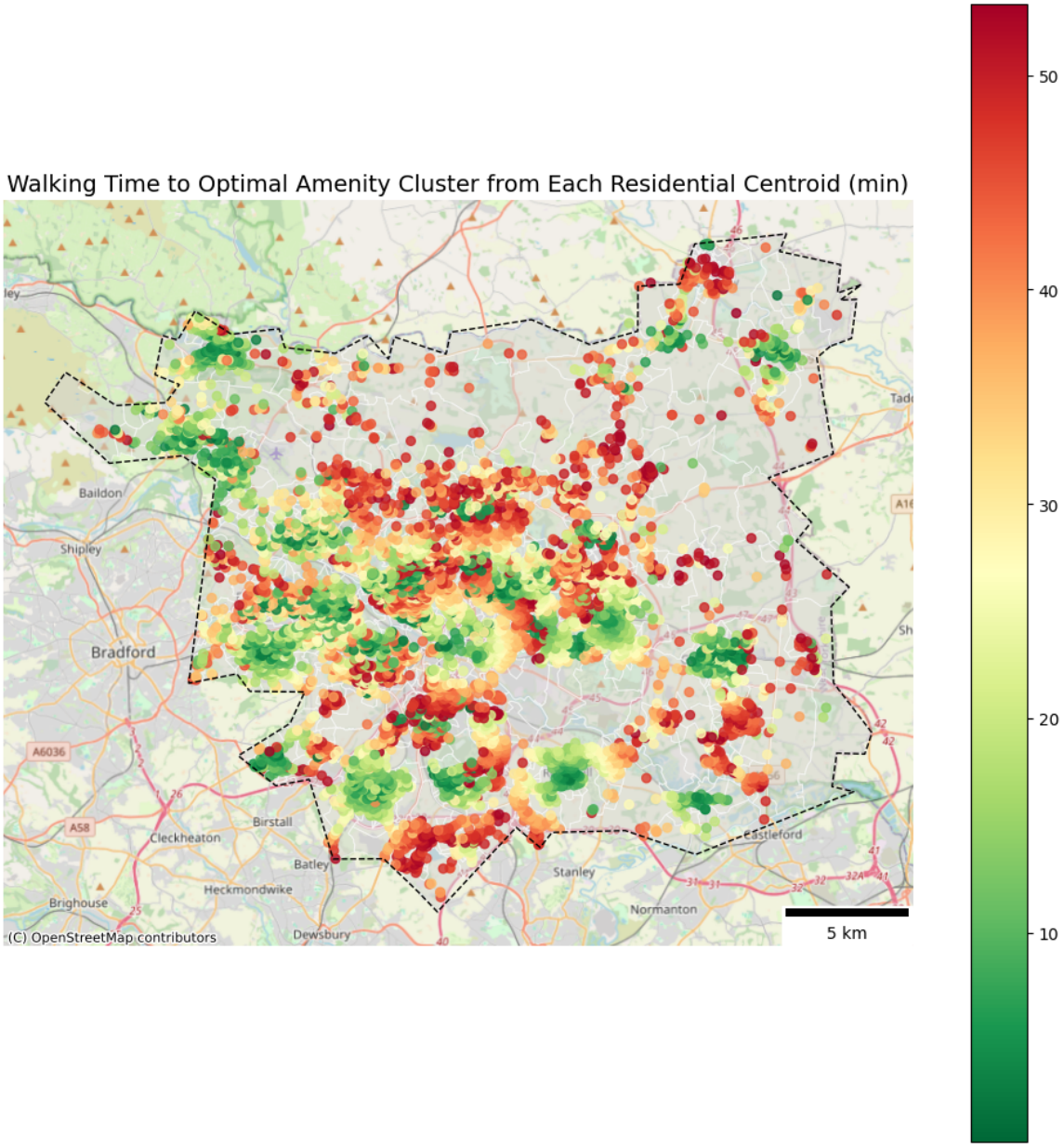
In [232…
```
Re_Se.head()
```

Out [232…

	Centroid_ID	geometry	centroid_node	cluster_score	distance	
0	CENTROID_0	POINT (429327.81 423196.799)	CENTROID_0	-0.659835	3482.119893	P
1	CENTROID_1	POINT (429316.077 423375.712)	CENTROID_1	-0.378857	2930.693244	P
2	CENTROID_2	POINT (429308.779 423399.26)	CENTROID_2	-0.378857	2910.722029	P
3	CENTROID_3	POINT (428870.995 423724.067)	CENTROID_3	-0.378857	3208.181179	P
4	CENTROID_4	POINT (428948.304 423748.19)	CENTROID_4	-0.378857	2701.169540	P

In [233…

```
fig, ax = plt.subplots(figsize=(10, 10))

# Base layers
OA_Leeds.plot(ax=ax, color='lightgrey', edgecolor='white', linewidt
LeedsBou.boundary.plot(ax=ax, color='k', linestyle='dashed', linewi

# optimal service-centres
Re_Se.plot(column='walk_time_min', cmap='RdYlGn_r', legend=True, ax

# basemap
cx.add_basemap(ax, crs=27700, source=cx.providers.OpenStreetMap.Map

# Labels and legend
plt.title('Walking Time to Optimal Amenity Cluster from Each Reside

# Add scalebar
scalebar = ScaleBar(dx=1, location='lower right')
ax.add_artist(scalebar)

ax.axis('off')
plt.tight_layout()
plt.show()
```



coverage map within walking distance from optimal pois POIs

In [234…
```
# gdf from optimal POI
optimal_pois = representative_pois_gdf[representative_pois_gdf['POI
```

In [235…
```
# Set walking threshold (meters)
walk_threshold = 1200
```

In [236…
```
"""""""
# Collect reachable edge geometries
covered_edges = []

# Loop through optimal POIs and collect reachable edges
for poi_id in tqdm(optimum_res_cluster['POI_ID'], desc='Collecting

    # Get reachable nodes from the POI within threshold
    lengths = nx.single_source_dijkstra_path_length(
        G_with_POIs_centroids,
        source=poi_id,
```

```
        weight='length',
        cutoff=walk_threshold
    )

    # Loop through edges and collect those with both ends reachable
    for u, v, data in G_with_POIs_centroids.edges(data=True):
        if u in lengths and v in lengths:
            if 'geometry' in data:
                covered_edges.append(data['geometry'])

# Combine all covered edges into a GeoDataFrame for plotting
covered_paths_gdf = gpd.GeoDataFrame(geometry=covered_edges, crs=27
"""
```

Out[236… '"""\n# Collect reachable edge geometries\ncovered_edges = []\n\n# Loop through optimal POIs and collect reachable edges\nfor poi_id in tqdm(optimum_res_cluster[\'POI_ID\'], desc=\'Collecting walkable paths\'):\n   \n   # Get reachable nodes from the POI within threshold\n   lengths = nx.single_source_dijkstra_path_length(\n G_with_POIs_centroids,\n   source=poi_id,\n   weight=\'length\',\n   cutoff=walk_threshold\n   )\n\n   # Loop through edges and collect those with both ends reachable\n   for u, v, data in G_with_POIs_centroids.edges(data=True):\n   if u in lengths and v in lengths:\n   if \'geometry\' in data:\n covered_edges.append(data[\'geometry\'])\n\n# Combine all covered edges into a GeoDataFrame for plotting\ncovered_paths_gdf = gpd.GeoDataFrame(geometry=covered_edges, crs=27700)\n'

In [237…

```
"""
# save
covered_paths_gdf.to_file('covered_paths_gdf.geojson', driver='GeoJ
"""
```

Out[237… '"""\n# save\ncovered_paths_gdf.to_file(\'covered_paths_gdf.geojson\', driver=\'GeoJSON\')\n'

In [238…

```
"""
fig, ax = plt.subplots(figsize=(12, 10))

# Base
edges.plot(ax=ax, color='lightgrey', linewidth=0.3, alpha=0.5)
OA_Leeds.plot(ax=ax, facecolor='none', edgecolor='black', linewidth

# Covered walking paths (within 15 min walk to optimal clusters)
covered_paths_gdf.plot(ax=ax, color='#0571b0', linewidth=0.7, alpha

# Add basemap
cx.add_basemap(ax, crs=27700, source=cx.providers.OpenStreetMap.Map

# Label
plt.title('Walkable Network Coverage from Optimal Amenity Clusters
ax.axis('off')
plt.legend()
plt.tight_layout()
plt.show()
"""
```

Out[238…
```
'"""\nfig, ax = plt.subplots(figsize=(12, 10))\n\n# Base\nedges.plo
t(ax=ax, color=\'lightgrey\', linewidth=0.3, alpha=0.5)\nOA_Leeds.
plot(ax=ax, facecolor=\'none\', edgecolor=\'black\', linewidth=0.
3, alpha=0.3)\n\n# Covered walking paths (within 15 min walk to op
timal clusters)\ncovered_paths_gdf.plot(ax=ax, color=\'#0571b0\',
linewidth=0.7, alpha=0.8, label=\'Covered paths\')\n\n# Add basema
p\ncx.add_basemap(ax, crs=27700, source=cx.providers.OpenStreetMa
p.Mapnik)\n\n# Label\nplt.title(\'Walkable Network Coverage from O
ptimal Amenity Clusters (1.2 km almost 15 min Walk)\', fontsize=1
4)\nax.axis(\'off\')\nplt.legend()\nplt.tight_layout()\nplt.show()
\n'
```

In [239…
```
"""
print('Total features:', len(covered_paths_gdf))
print('Unique geometries:', covered_paths_gdf.geometry.nunique())
"""
```

Out[239…
```
'"""\nprint(\'Total features:\', len(covered_paths_gdf))\nprint(\'U
nique geometries:\', covered_paths_gdf.geometry.nunique())\n'
```

Since covered paths are generated from buffers around multiple POI clusters, there are overlapped covered paths clear in the map, also the output of the last cell. It is essential to remove these duplicated or overlapping geometries to ensure accurate results:

In [240…
```
"""
# look if there is duplicated geometry
covered_paths_gdf['dup'] = covered_paths_gdf.duplicated(subset='geo
"""
```

Out[240…
```
'"""\n# look if there is duplicated geometry\ncovered_paths_gdf[\'d
up\'] = covered_paths_gdf.duplicated(subset=\'geometry\')\n'
```

In [241…
```
"""
covered_paths_gdf.head()
"""
```

Out[241…
```
'"""\ncovered_paths_gdf.head()\n'
```

In [242…
```
"""
covered_paths_gdf['dup'].sum()
"""
```

Out[242…
```
'"""\ncovered_paths_gdf[\'dup\'].sum()\n'
```

In [243…
```
"""
covered_paths_gdf.info()
"""
```

Out[243…
```
'"""\ncovered_paths_gdf.info()\n'
```

In [244…
```
"""
# drop duplicated geometeries
covered_paths_clean = covered_paths_gdf.drop_duplicates(subset='geo
```

```
"""
```

Out[244…
```
'"""\n# drop duplicated geometeries\ncovered_paths_clean = covered_
paths_gdf.drop_duplicates(subset=\'geometry\').reset_index(drop=Tr
ue)\n'
```

In [245…
```
"""
print('Total features:', len(covered_paths_clean))
print('Unique geometries:', covered_paths_clean.geometry.nunique())
"""
```

Out[245…
```
'"""\nprint(\'Total features:\', len(covered_paths_clean))\nprint(
\'Unique geometries:\', covered_paths_clean.geometry.nunique())\n'
```

In [246…
```
"""
# save
covered_paths_clean.to_file('covered_paths_clean.geojson', driver='
"""
```

Out[246…
```
'"""\n# save\ncovered_paths_clean.to_file(\'covered_paths_clean.geo
json\', driver=\'GeoJSON\')\n'
```

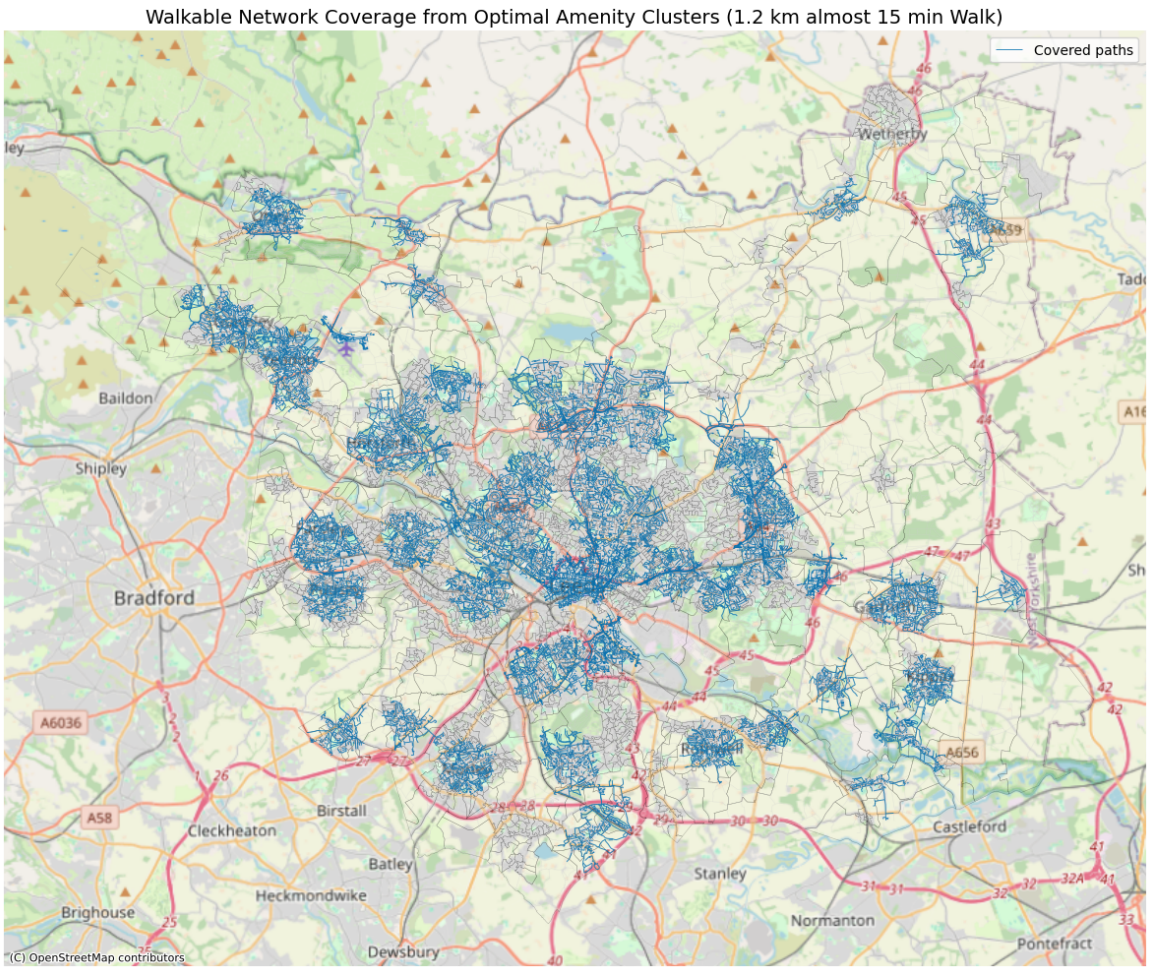
In [247…
```
# load
covered_paths_gdf = gpd.read_file('covered_paths_clean.geojson')
```

In [248…
```
covered_paths_gdf.crs
```

Out[248…
```
<Projected CRS: EPSG:27700>
Name: OSGB36 / British National Grid
Axis Info [cartesian]:
- E[east]: Easting (metre)
- N[north]: Northing (metre)
Area of Use:
- name: United Kingdom (UK) - offshore to boundary of UKCS within
49°45'N to 61°N and 9°W to 2°E; onshore Great Britain (England, Wa
les and Scotland). Isle of Man onshore.
- bounds: (-9.01, 49.75, 2.01, 61.01)
Coordinate Operation:
- name: British National Grid
- method: Transverse Mercator
Datum: Ordnance Survey of Great Britain 1936
- Ellipsoid: Airy 1830
- Prime Meridian: Greenwich
```

In [249…
```
fig, ax = plt.subplots(figsize=(12, 10))

# Base
edges.plot(ax=ax, color='lightgrey', linewidth=0.3, alpha=0.5)
OA_Leeds.plot(ax=ax, facecolor='none', edgecolor='black', linewidth

# Covered walking paths (within 15 min walk to optimal clusters)
covered_paths_gdf.plot(ax=ax, color='#0571b0', linewidth=0.7, alpha

# Add basemap
cx.add_basemap(ax, crs=27700, source=cx.providers.OpenStreetMap.Map
```

```
# Label
plt.title('Walkable Network Coverage from Optimal Amenity Clusters
ax.axis('off')
plt.legend()
plt.tight_layout()
plt.show()
```



Aggregate results in LSOA level

In [250…

```
Re_Se.info()
```

```
<class 'geopandas.geodataframe.GeoDataFrame'>
RangeIndex: 6484 entries, 0 to 6483
Data columns (total 9 columns):
 #   Column         Non-Null Count  Dtype   
---  ------         --------------  -----   
 0   Centroid_ID    6484 non-null   object  
 1   geometry       6484 non-null   geometry
 2   centroid_node  6289 non-null   object  
 3   cluster_score  6289 non-null   float64 
 4   distance       6289 non-null   float64 
 5   POI_ID         6289 non-null   object  
 6   sc/dis         6289 non-null   float64 
 7   population     6289 non-null   float64 
 8   walk_time_min  6289 non-null   float64 
dtypes: float64(5), geometry(1), object(3)
memory usage: 456.0+ KB
```

In [251…
```
Re_Se_LSOA = gpd.sjoin(Re_Se, Leeds_LSOA, how='left', predicate='in
```

In [252…
```
Re_Se_LSOA.info()
```

```
<class 'geopandas.geodataframe.GeoDataFrame'>
Index: 6484 entries, 0 to 6483
Data columns (total 18 columns):
 #   Column         Non-Null Count  Dtype   
---  ------         --------------  -----   
 0   Centroid_ID    6484 non-null   object  
 1   geometry       6484 non-null   geometry
 2   centroid_node  6289 non-null   object  
 3   cluster_score  6289 non-null   float64 
 4   distance       6289 non-null   float64 
 5   POI_ID         6289 non-null   object  
 6   sc/dis         6289 non-null   float64 
 7   population     6289 non-null   float64 
 8   walk_time_min  6289 non-null   float64 
 9   index_right    6479 non-null   float64 
 10  LSOA11CD       6479 non-null   object  
 11  LSOA11NM       6479 non-null   object  
 12  BNG_E          6479 non-null   float64 
 13  BNG_N          6479 non-null   float64 
 14  LONG_          6479 non-null   float64 
 15  LAT            6479 non-null   float64 
 16  Shape_Leng     6479 non-null   float64 
 17  GlobalID       6479 non-null   object  
dtypes: float64(11), geometry(1), object(6)
memory usage: 962.5+ KB
```

In [253…
```
Re_Se_LSOA['LSOA11CD'].isna().sum()
```

Out[253… np.int64(5)

In [254…
```
# filter to non-null values
Re_Se_LSOA = Re_Se_LSOA[Re_Se_LSOA['LSOA11CD'].notna()]
```

In [255…
```
# Weighted aggregation to LSOA using population
weighted = Re_Se_LSOA.dropna(subset=['walk_time_min', 'cluster_scor
```

In [256…
```
# Grouped weighted averages
LSOA_Summary = weighted.groupby('LSOA11CD').apply(
    lambda g: pd.Series({
        'avg_walk_time': (g['walk_time_min'] * g['population']).sum
        'avg_cluster_score': (g['cluster_score'] * g['population'])
    })
).reset_index()
```

```
/var/folders/wd/1wf586s93vg1fbnl295w_w1r0000gn/T/ipykernel_64251/1086050871.py:2: DeprecationWarning: DataFrameGroupBy.apply operated on the grouping columns. This behavior is deprecated, and in a future version of pandas the grouping columns will be excluded from the operation. Either pass `include_groups=False` to exclude the groupings or explicitly select the grouping columns after groupby to silence this warning.
  LSOA_Summary = weighted.groupby('LSOA11CD').apply(
```

In [257…
```
LSOA_Summary.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 481 entries, 0 to 480
Data columns (total 3 columns):
 #   Column             Non-Null Count  Dtype  
---  ------             --------------  -----  
 0   LSOA11CD           481 non-null    object 
 1   avg_walk_time      481 non-null    float64
 2   avg_cluster_score  481 non-null    float64
dtypes: float64(2), object(1)
memory usage: 11.4+ KB
```

In [258…
```
# add geoemetry col missed in the process of grouping
geometry_LSOA = Leeds_LSOA[['LSOA11CD', 'geometry']].copy()
LSOA_Summary = pd.merge(LSOA_Summary, geometry_LSOA, on='LSOA11CD',

LSOA_Summary = gpd.GeoDataFrame(LSOA_Summary, geometry='geometry',
```

In [259…
```
LSOA_Summary.info()
```

```
<class 'geopandas.geodataframe.GeoDataFrame'>
RangeIndex: 481 entries, 0 to 480
Data columns (total 4 columns):
 #   Column             Non-Null Count  Dtype   
---  ------             --------------  -----   
 0   LSOA11CD           481 non-null    object  
 1   avg_walk_time      481 non-null    float64 
 2   avg_cluster_score  481 non-null    float64 
 3   geometry           481 non-null    geometry
dtypes: float64(2), geometry(1), object(1)
memory usage: 15.2+ KB
```

In [260…
```
LSOA_Summary.head()
```

Out[260…

	LSOA11CD	avg_walk_time	avg_cluster_score	geometry
0	E01011264	17.069101	10.970629	POLYGON ((421284.072 442306.911, 421307.406 44...
1	E01011265	14.759119	11.578122	POLYGON ((418637.561 442552.855, 418663.429 44...
2	E01011266	16.470318	11.457796	POLYGON ((417899.555 443280.11, 417909.186 443...
3	E01011267	21.196697	8.912556	POLYGON ((419612.752 442323.051, 419611 442322...
4	E01011268	16.096181	10.481461	POLYGON ((420443.313 442419.094, 420456.276 44...

LSOA walking path coverage

Calculating the proportion of each LSOA's walking path network that is within a 15-minute walk to optimal amenity clusters:

total walking path in each LASOA

In [261…
```
edges_in_lsoa = gpd.overlay(edges, Leeds_LSOA, how='intersection')
```

In [262…
```
edges_in_lsoa['length'] = edges_in_lsoa.geometry.length
```

In [263…
```
total_W_path_length = edges_in_lsoa.groupby('LSOA11CD')['length'].s
```

In [264…
```
total_W_path_length.head()
```

Out[264…

	LSOA11CD	length
0	E01011264	8019.446431
1	E01011265	19273.769923
2	E01011266	31282.201964
3	E01011267	11676.131913
4	E01011268	10966.539009

In [265…
```
total_W_path_length.rename(columns={'length': 'total_walking_length
```

covered path from optimal POIs in each LASO

In [266…
```
covered_in_lsoa = gpd.overlay(covered_paths_gdf, Leeds_LSOA, how='i
```

In [267…
```
covered_in_lsoa['length'] = covered_in_lsoa.geometry.length
```

In [268…
```
covered_in_lsoa.head()
```

Out[268…

	dup	LSOA11CD	LSOA11NM	BNG_E	BNG_N	LONG_	LAT	Shape
0	False	E01011551	Leeds 107C	431131	425643	-1.52966	53.7263	6218.1
1	False	E01011551	Leeds 107C	431131	425643	-1.52966	53.7263	6218.1
2	False	E01011551	Leeds 107C	431131	425643	-1.52966	53.7263	6218.1
3	False	E01011537	Leeds 107A	430307	424904	-1.54222	53.7197	6196.8
4	False	E01011537	Leeds 107A	430307	424904	-1.54222	53.7197	6196.8

In [269…
```
covered_path_length = covered_in_lsoa.groupby('LSOA11CD')['length']
```

In [270…
```
covered_path_length.head()
```

Out[270…

	LSOA11CD	length
0	E01011264	3563.024646
1	E01011265	13469.070892
2	E01011266	16947.061166
3	E01011267	7311.422630
4	E01011268	6638.863488

In [271…
```
covered_path_length.rename(columns={'length': 'covered_length'}, in
```

Covered Path Ratio

In [272…
```
# Merge and calculate coverage ratio
coverage_df = pd.merge(total_W_path_length, covered_path_length, on
```

In [273…
```
coverage_df.head()
```

Out[273…

	LSOA11CD	total_walking_length	covered_length
0	E01011264	8019.446431	3563.024646
1	E01011265	19273.769923	13469.070892
2	E01011266	31282.201964	16947.061166
3	E01011267	11676.131913	7311.422630
4	E01011268	10966.539009	6638.863488

In [274…
```
coverage_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 482 entries, 0 to 481
Data columns (total 3 columns):
 #   Column                Non-Null Count  Dtype  
---  ------                --------------  -----  
 0   LSOA11CD              482 non-null    object 
 1   total_walking_length  482 non-null    float64
 2   covered_length        418 non-null    float64
dtypes: float64(2), object(1)
memory usage: 11.4+ KB
```

In [275…
```
coverage_df[['covered_length', 'total_walking_length']].describe()
```

Out[275…

	covered_length	total_walking_length
count	418.000000	482.000000
mean	4801.929430	13963.325989
std	3887.995448	13234.796974
min	51.053199	3209.436647
25%	1971.601647	6987.007556
50%	3983.075542	10007.500831
75%	6590.472841	15685.526489
max	23353.419773	105070.985448

In [276…
```
coverage_df['covered_length'] = coverage_df['covered_length'].filln
```

In [277…
```
coverage_df['coverage_ratio'] = coverage_df['covered_length'] / cov
```

In [278…
```
coverage_df['coverage_ratio'].describe()
```

Out[278…
```
count    482.000000
mean       0.357514
std        0.262638
min        0.000000
25%        0.090719
50%        0.361627
75%        0.604930
max        0.864008
Name: coverage_ratio, dtype: float64
```

Merge coverage with lsoa_summary

In [279…
```
LSOA_Summary2 = pd.merge(LSOA_Summary,coverage_df[['LSOA11CD', 'cov

LSOA_Summary2.head()
```

Out[279…

	LSOA11CD	avg_walk_time	avg_cluster_score	geometry	coverage_rat
0	E01011264	17.069101	10.970629	POLYGON ((421284.072 442306.911, 421307.406 44...	0.4442
1	E01011265	14.759119	11.578122	POLYGON ((418637.561 442552.855, 418663.429 44...	0.6988
2	E01011266	16.470318	11.457796	POLYGON ((417899.555 443280.11, 417909.186 443...	0.5417
3	E01011267	21.196697	8.912556	POLYGON ((419612.752 442323.051, 419611 442322...	0.6261
4	E01011268	16.096181	10.481461	POLYGON ((420443.313 442419.094, 420456.276 44...	0.6053

In [280…
```
LSOA_Summary2.info()
```

```
<class 'geopandas.geodataframe.GeoDataFrame'>
RangeIndex: 481 entries, 0 to 480
Data columns (total 5 columns):
 #   Column             Non-Null Count  Dtype   
---  ------             --------------  -----   
 0   LSOA11CD           481 non-null    object  
 1   avg_walk_time      481 non-null    float64 
 2   avg_cluster_score  481 non-null    float64 
 3   geometry           481 non-null    geometry
 4   coverage_ratio     481 non-null    float64 
dtypes: float64(3), geometry(1), object(1)
memory usage: 18.9+ KB
```

Daily Accessibility Index (DAI)

In [281…
```
# invert walk time (polarity matters!)
LSOA_Summary2['inv_walk_time'] = LSOA_Summary2['avg_walk_time'].max
```

In [282…
```
LSOA_Summary2.info()
```

```
<class 'geopandas.geodataframe.GeoDataFrame'>
RangeIndex: 481 entries, 0 to 480
Data columns (total 6 columns):
 #   Column             Non-Null Count  Dtype   
---  ------             --------------  -----   
 0   LSOA11CD           481 non-null    object  
 1   avg_walk_time      481 non-null    float64 
 2   avg_cluster_score  481 non-null    float64 
 3   geometry           481 non-null    geometry
 4   coverage_ratio     481 non-null    float64 
 5   inv_walk_time      481 non-null    float64 
dtypes: float64(4), geometry(1), object(1)
memory usage: 22.7+ KB
```

In [283…
```
DAI_Inputs = LSOA_Summary2[[
    'LSOA11CD',
    'inv_walk_time',
    'avg_cluster_score',
    'coverage_ratio',
    'geometry'
]].copy()
```

In [284…
```
# min-max normalization
scaler = MinMaxScaler()
normalized = scaler.fit_transform(DAI_Inputs[['inv_walk_time', 'avg
```

In [285…
```
# add normalized columns
DAI_Inputs['walktime_score_norm']   = normalized[:, 0]
DAI_Inputs['cluster_score_norm']    = normalized[:, 1]
DAI_Inputs['coverage_score_norm']   = normalized[:, 2]
```

In [286…
```
DAI_Inputs.head()
```

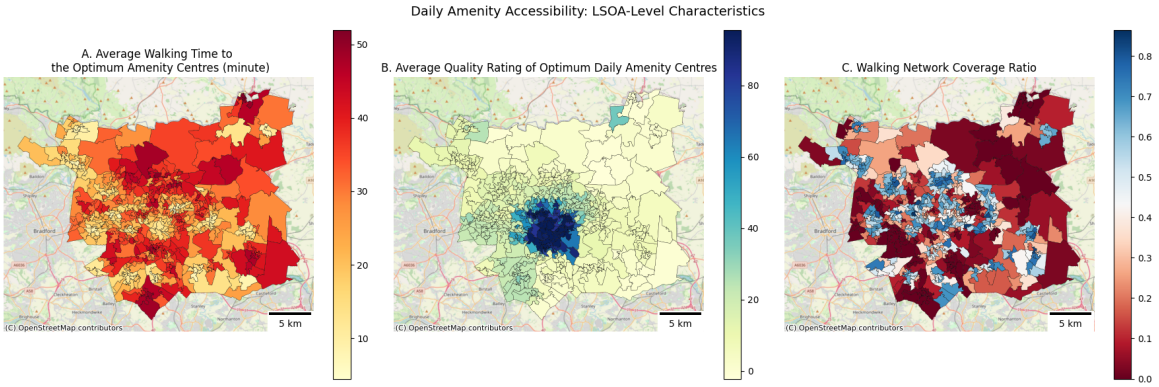
Out [286…

	LSOA11CD	inv_walk_time	avg_cluster_score	coverage_ratio	geometr
0	E01011264	34.881608	10.970629	0.444298	POLYGO ((421284.07 442306.91 421307.40 44.
1	E01011265	37.191590	11.578122	0.698829	POLYGO ((418637.56 442552.85 418663.42 44.
2	E01011266	35.480391	11.457796	0.541748	POLYGO ((417899.55 443280.1 417909.18 443.
3	E01011267	30.754011	8.912556	0.626185	POLYGO ((419612.75 442323.05 41961 442322.
4	E01011268	35.854527	10.481461	0.605375	POLYGO ((420443.31 442419.094 420456.27 44.

In [287…

```
# titles for each subplot
titles = [
    'A. Average Walking Time to \n the Optimum Amenity Centres (min
    'B. Average Quality Rating of Optimum Daily Amenity Centres',
    'C. Walking Network Coverage Ratio'
]

columns = ['avg_walk_time', 'avg_cluster_score', 'coverage_ratio']

# custom colormap per variable
colormaps = {
    'avg_walk_time': 'YlOrRd',
    'avg_cluster_score': 'YlGnBu',
    'coverage_ratio': 'RdBu'
}

# create subplots
fig = plt.figure(figsize=(18, 6))
gs = GridSpec(1, 3)
axes = [fig.add_subplot(gs[0, i]) for i in range(3)]

# plot each input with appropriate colormap
for ax, col, title in zip(axes, columns, titles):
    LSOA_Summary2.plot(
        column=col,
```

```
        cmap=colormaps[col],
        legend=True,
        linewidth=0.2,
        edgecolor='black',
        ax=ax
    )
    cx.add_basemap(ax, crs=LSOA_Summary2.crs, source=cx.providers.O
    ax.add_artist(ScaleBar(dx=1, location='lower right'))
    ax.set_title(title, fontsize=12)
    ax.axis('off')

# figure title
fig.suptitle('Daily Amenity Accessibility: LSOA-Level Characteristi
plt.tight_layout()
plt.show()
```



In [288…
```
# compute DAI
DAI_Inputs['DAI'] = DAI_Inputs[['walktime_score_norm', 'cluster_sco
```

In [289…
```
DAI_Inputs['DAI'].describe()
```

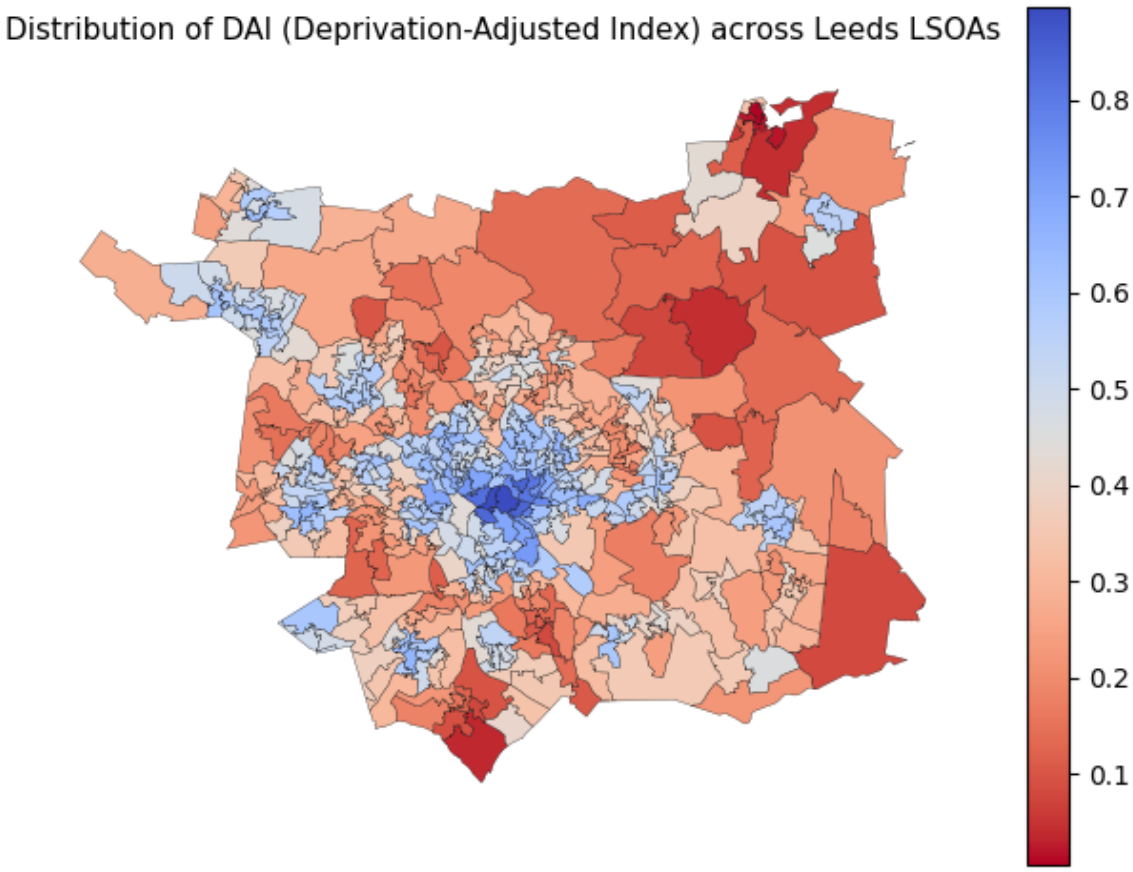
Out[289…
```
count    481.000000
mean       0.407701
std        0.173518
min        0.005563
25%        0.274568
50%        0.415262
75%        0.545384
max        0.895999
Name: DAI, dtype: float64
```

In [290…
```
DAI_Inputs.info()
```

```
<class 'geopandas.geodataframe.GeoDataFrame'>
RangeIndex: 481 entries, 0 to 480
Data columns (total 9 columns):
 #   Column               Non-Null Count  Dtype
---  ------               --------------  -----
 0   LSOA11CD             481 non-null    object
 1   inv_walk_time        481 non-null    float64
 2   avg_cluster_score    481 non-null    float64
 3   coverage_ratio       481 non-null    float64
 4   geometry             481 non-null    geometry
 5   walktime_score_norm  481 non-null    float64
 6   cluster_score_norm   481 non-null    float64
 7   coverage_score_norm  481 non-null    float64
 8   DAI                  481 non-null    float64
dtypes: float64(7), geometry(1), object(1)
memory usage: 33.9+ KB
```

In [291…

```
fig, ax = plt.subplots()
DAI_Inputs.plot(column='DAI',cmap='coolwarm_r',legend=True,linewidt

ax.set_title('Distribution of DAI (Deprivation-Adjusted Index) acro
ax.axis('off')
plt.tight_layout()
plt.show()
```



DAI-IMD Analysis

Merging & Visualisation

In [292…
```
IMD.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 482 entries, 10947 to 32185
Data columns (total 6 columns):
 #   Column                                  Non-Null Count  Dty
pe
---  ------                                  --------------  ---
--
 0   LSOA code (2011)                        482 non-null    obj
ect
 1   LSOA name (2011)                        482 non-null    obj
ect
 2   Local Authority District code (2019)    482 non-null    obj
ect
 3   Local Authority District name (2019)    482 non-null    obj
ect
 4   Index of Multiple Deprivation (IMD) Rank    482 non-null    int
64
 5   Index of Multiple Deprivation (IMD) Decile  482 non-null    int
64
dtypes: int64(2), object(4)
memory usage: 26.4+ KB
```

In [293…
```
# merge with IMD
DAI_Inputs = DAI_Inputs.merge(IMD[['LSOA code (2011)', 'Index of Mu
                                  left_on='LSOA11CD',
                                  right_on='LSOA code (2011)',
                                  how='left')
```

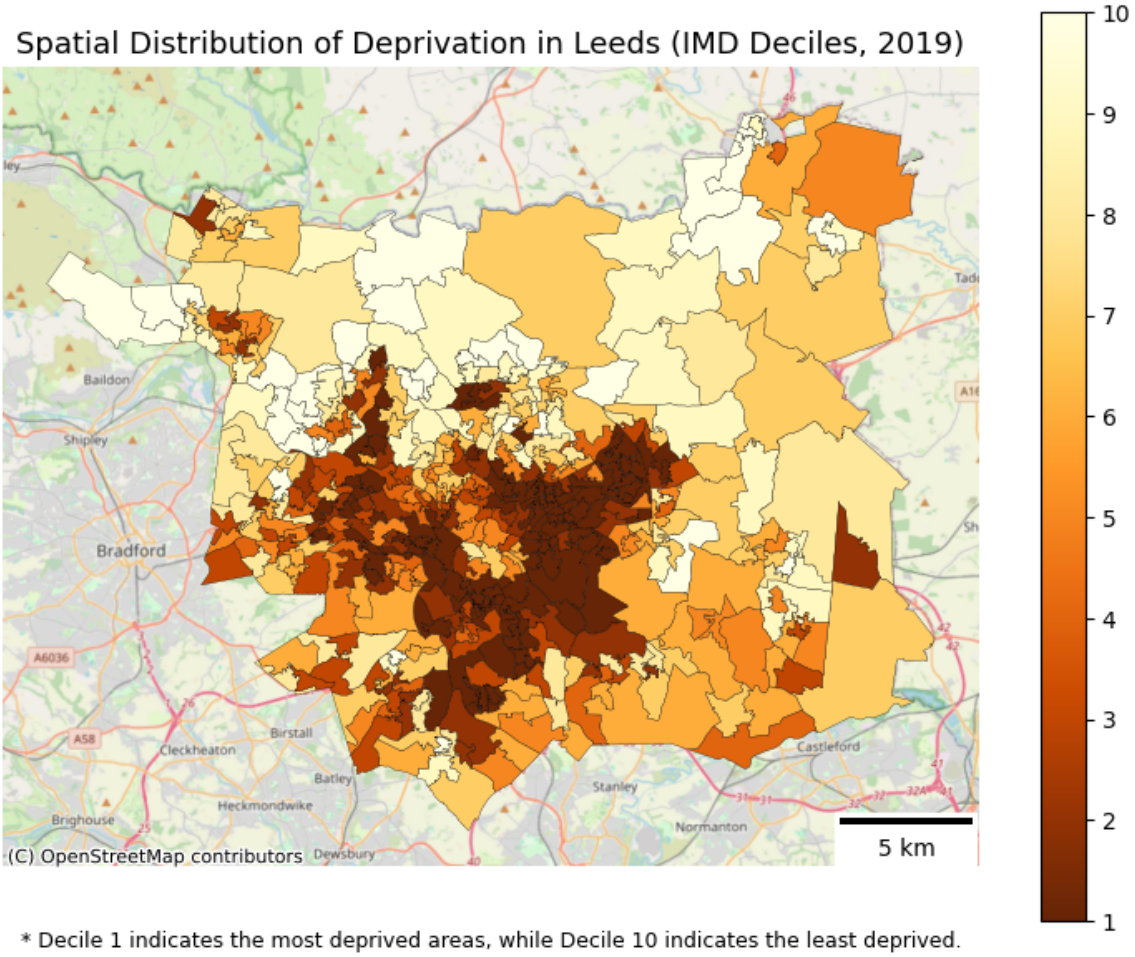
In [294…
```
DAI_Inputs.info()
```

```
<class 'geopandas.geodataframe.GeoDataFrame'>
RangeIndex: 481 entries, 0 to 480
Data columns (total 11 columns):
 #   Column                                    Non-Null Count  Dty
pe
---  ------                                    --------------  ---
--
 0   LSOA11CD                                  481 non-null    obj
ect
 1   inv_walk_time                             481 non-null    flo
at64
 2   avg_cluster_score                         481 non-null    flo
at64
 3   coverage_ratio                            481 non-null    flo
at64
 4   geometry                                  481 non-null    geo
metry
 5   walktime_score_norm                       481 non-null    flo
at64
 6   cluster_score_norm                        481 non-null    flo
at64
 7   coverage_score_norm                       481 non-null    flo
at64
 8   DAI                                       481 non-null    flo
at64
 9   LSOA code (2011)                          481 non-null    obj
ect
 10  Index of Multiple Deprivation (IMD) Decile  481 non-null    int
64
dtypes: float64(7), geometry(1), int64(1), object(2)
memory usage: 41.5+ KB
```

In [367…

```
# Set up the figure
fig, ax = plt.subplots(figsize=(8, 6))

# Plot deprivation map
DAI_Inputs.plot(ax=ax, column='Index of Multiple Deprivation (IMD)

# Add basemap and scalebar
cx.add_basemap(ax, crs=DAI_Inputs.crs, source=cx.providers.OpenStre
scalebar = ScaleBar(dx=1, location='lower right')
ax.add_artist(scalebar)

# Set title
ax.set_title('Spatial Distribution of Deprivation in Leeds (IMD Dec

# Add note
ax.text(
    0.5, -0.08,
    '* Decile 1 indicates the most deprived areas, while Decile 10
    fontsize=9,
    ha='center',
    va='top',
    transform=ax.transAxes
)
```

```
# Remove axes
ax.axis('off')
plt.tight_layout()
plt.show()
```

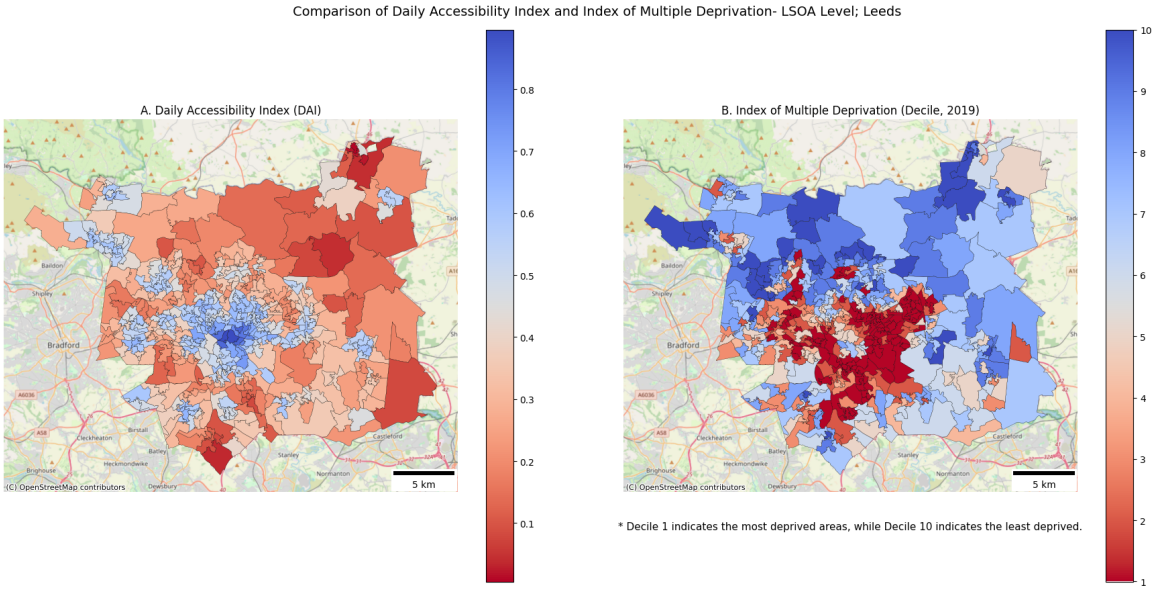


* Decile 1 indicates the most deprived areas, while Decile 10 indicates the least deprived.

In [369…

```
# Set up the figure
fig = plt.figure(figsize=(18, 9))
gs = GridSpec(1, 2)
axes = [fig.add_subplot(gs[0, i]) for i in range(2)]

# Titles and column names
titles = [
    'A. Daily Accessibility Index (DAI)',
    'B. Index of Multiple Deprivation (Decile, 2019)'
]
columns = ['DAI', 'Index of Multiple Deprivation (IMD) Decile']
colormaps = ['coolwarm_r', 'coolwarm_r']

# Plot both maps
for ax, col, title, cmap in zip(axes, columns, titles, colormaps):
    DAI_Inputs.plot(ax=ax, column=col, cmap=cmap, legend=True, line

    cx.add_basemap(ax, crs=DAI_Inputs.crs, source=cx.providers.Open
    ax.add_artist(ScaleBar(dx=1, location='lower right'))
    ax.set_title(title, fontsize=12)
    ax.axis('off')

# Add note
```

```
ax.text(
    0.5, -0.08,
    '* Decile 1 indicates the most deprived areas, while Decile 10
    fontsize=11,
    ha='center',
    va='top',
    transform=ax.transAxes
)

# Figure title
fig.suptitle('Comparison of Daily Accessibility Index and Index of
plt.tight_layout()
plt.show()
```



Pearson Correlation

In [297…
```
# a copy of dataframe,
DAI_V = DAI_Inputs[['DAI', 'Index of Multiple Deprivation (IMD) Dec
```

In [298…
```
# Pearson correlation
pearson_corr, p_value = pearsonr(DAI_V['DAI'], DAI_V['Index of Mult

print('Pearson correlation:', round(pearson_corr, 3))
print('p-value:', round(p_value, 3))
```

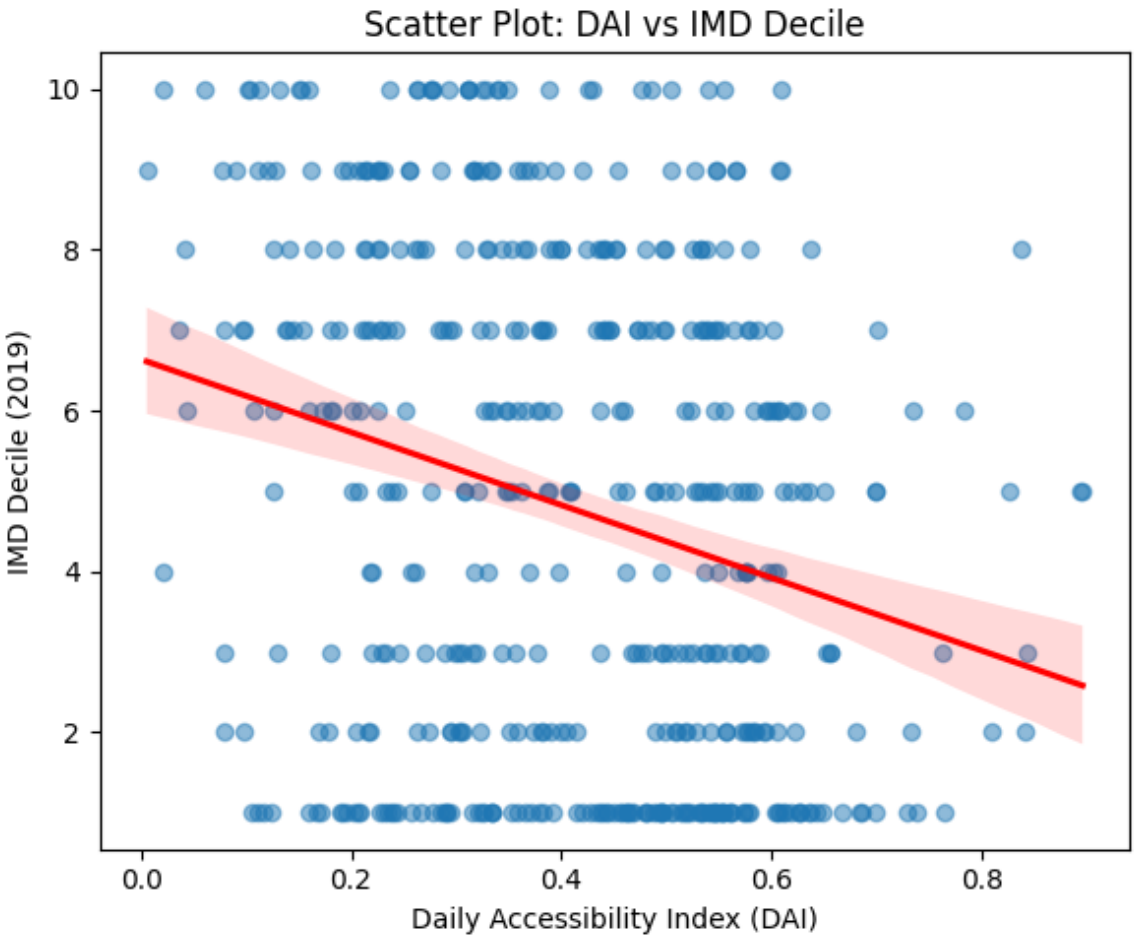
```
Pearson correlation: -0.255
p-value: 0.0
```

Pearson correlation shows a weak neagtive relation, which is statistically significant.

Linear Regression

In [299…
```
# scatterplot with linear regression
plt.figure(figsize=(6, 5))
sns.regplot(data=DAI_Inputs, x='DAI', y='Index of Multiple Deprivat
plt.xlabel('Daily Accessibility Index (DAI)')
```

```
plt.ylabel('IMD Decile (2019)')
plt.title('Scatter Plot: DAI vs IMD Decile')

plt.tight_layout()
plt.show()
```



Hardly any linear regression found (red line), also the R2 in global regression (in the following cells) is close to 0.055. Therefore, no significant linear relationship found between two variables. To explore local varoiations, GWR and LISA anlaysis is applied:

Local Analysis

Geographical Regression

In [300…

```
DAI_V.crs
```

Out[300…
```
<Projected CRS: EPSG:27700>
Name: OSGB36 / British National Grid
Axis Info [cartesian]:
- E[east]: Easting (metre)
- N[north]: Northing (metre)
Area of Use:
- name: United Kingdom (UK) - offshore to boundary of UKCS within
49°45'N to 61°N and 9°W to 2°E; onshore Great Britain (England, Wa
les and Scotland). Isle of Man onshore.
- bounds: (-9.01, 49.75, 2.01, 61.01)
Coordinate Operation:
- name: British National Grid
- method: Transverse Mercator
Datum: Ordnance Survey of Great Britain 1936
- Ellipsoid: Airy 1830
- Prime Meridian: Greenwich
```

In [301…
```
# coordinates
DAI_V['x'] = DAI_V.geometry.centroid.x
DAI_V['y'] = DAI_V.geometry.centroid.y
coords = DAI_V[['x', 'y']].values

# extract variables
X = DAI_V[['DAI']].values
y = DAI_V[['Index of Multiple Deprivation (IMD) Decile']].values

# standardize X
gwr_scaler = StandardScaler()
X_std = scaler.fit_transform(X)
```

In [302…
```
# calculate optimised bandwidth
gwr_selector = Sel_BW(coords, y, X_std)

bw = gwr_selector.search()

bw
```

```
/Users/naiemegolzari/Library/Python/3.9/lib/python/site-packages/url
lib3/__init__.py:35: NotOpenSSLWarning: urllib3 v2 only supports Ope
nSSL 1.1.1+, currently the 'ssl' module is compiled with 'LibreSSL
2.8.3'. See: https://github.com/urllib3/urllib3/issues/3020
  warnings.warn(
/Users/naiemegolzari/Library/Python/3.9/lib/python/site-packages/url
lib3/__init__.py:35: NotOpenSSLWarning: urllib3 v2 only supports Ope
nSSL 1.1.1+, currently the 'ssl' module is compiled with 'LibreSSL
2.8.3'. See: https://github.com/urllib3/urllib3/issues/3020
  warnings.warn(
/Users/naiemegolzari/Library/Python/3.9/lib/python/site-packages/url
lib3/__init__.py:35: NotOpenSSLWarning: urllib3 v2 only supports Ope
nSSL 1.1.1+, currently the 'ssl' module is compiled with 'LibreSSL
2.8.3'. See: https://github.com/urllib3/urllib3/issues/3020
  warnings.warn(
/Users/naiemegolzari/Library/Python/3.9/lib/python/site-packages/url
lib3/__init__.py:35: NotOpenSSLWarning: urllib3 v2 only supports Ope
nSSL 1.1.1+, currently the 'ssl' module is compiled with 'LibreSSL
2.8.3'. See: https://github.com/urllib3/urllib3/issues/3020
  warnings.warn(
/Users/naiemegolzari/Library/Python/3.9/lib/python/site-packages/url
lib3/__init__.py:35: NotOpenSSLWarning: urllib3 v2 only supports Ope
nSSL 1.1.1+, currently the 'ssl' module is compiled with 'LibreSSL
2.8.3'. See: https://github.com/urllib3/urllib3/issues/3020
  warnings.warn(
/Users/naiemegolzari/Library/Python/3.9/lib/python/site-packages/url
lib3/__init__.py:35: NotOpenSSLWarning: urllib3 v2 only supports Ope
nSSL 1.1.1+, currently the 'ssl' module is compiled with 'LibreSSL
2.8.3'. See: https://github.com/urllib3/urllib3/issues/3020
  warnings.warn(
/Users/naiemegolzari/Library/Python/3.9/lib/python/site-packages/url
lib3/__init__.py:35: NotOpenSSLWarning: urllib3 v2 only supports Ope
nSSL 1.1.1+, currently the 'ssl' module is compiled with 'LibreSSL
2.8.3'. See: https://github.com/urllib3/urllib3/issues/3020
  warnings.warn(
/Users/naiemegolzari/Library/Python/3.9/lib/python/site-packages/url
lib3/__init__.py:35: NotOpenSSLWarning: urllib3 v2 only supports Ope
nSSL 1.1.1+, currently the 'ssl' module is compiled with 'LibreSSL
2.8.3'. See: https://github.com/urllib3/urllib3/issues/3020
  warnings.warn(
```

Out[302…

```
np.float64(45.0)
```

In [303…

```
# GWR
gwr_model = GWR(coords, y, X_std, bw=bw)
gwr_results = gwr_model.fit()

gwr_results.summary()
```

```
===========================================================================
=======
Model type                                                                G
aussian
Number of observations:
481
Number of covariates:
```

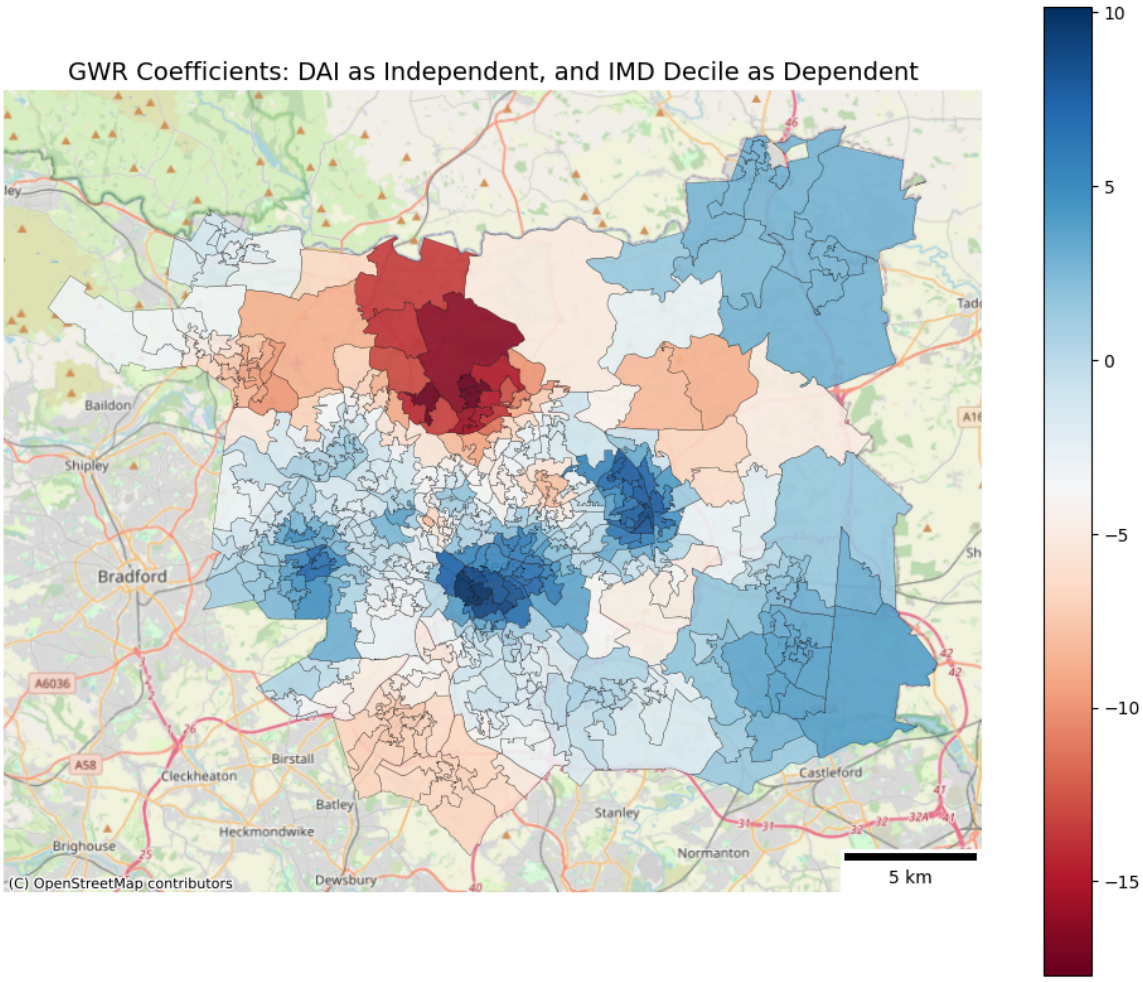
```
2

Global Regression Results
------------------------------------------------------------------------
-------
Residual sum of squares:                                              4
247.382
Log-likelihood:                                                      -1
206.364
AIC:                                                                  2
416.729
AICc:                                                                 2
418.779
BIC:                                                                  1
289.142
R2:
0.065
Adj. R2:
0.063

Variable                          Est.         SE  t(Est/SE)
p-value
------------------------------ ---------- ---------- ---------- ---
-------
X0                                6.612      0.343     19.275
0.000
X1                               -4.029      0.697     -5.776
0.000

Geographically Weighted Regression (GWR) Results
------------------------------------------------------------------------
-------
Spatial kernel:                                          Adaptive b
isquare
Bandwidth used:
45.000

Diagnostic information
------------------------------------------------------------------------
-------
Residual sum of squares:                                              1
789.981
Effective number of parameters (trace(S)):
53.899
Degree of freedom (n - trace(S)):
427.101
Sigma estimate:
2.047
Log-likelihood:                                                      -
998.549
AIC:                                                                  2
106.896
AICc:                                                                 2
121.334
BIC:                                                                  2
336.147
```

```
R2:
0.606
Adjusted R2:
0.556
Adj. alpha (95%):
0.002
Adj. critical t value (95%):
3.130

Summary Statistics For GWR Parameter Estimates
------------------------------------------------------------------------
-------
Variable                  Mean        STD        Min     Median
Max
-------------------- ---------- ---------- ---------- ---------- ---
-------
X0                       5.094      3.526     -3.842      5.121
13.484
X1                      -1.391      4.872    -17.703     -1.416
10.146
========================================================================
=======
```

With an adjusted R^2 of 0.535, the GWR model is far better in explaining the changes in IMD according to DAI changes, compared to the global linear model. The relationship between accessibility and deprivation varies spatially. While some areas show a strong negative association, other LSOAS show weaker or even positive relationships. (local coefficients range between –3.14 to +2.04).

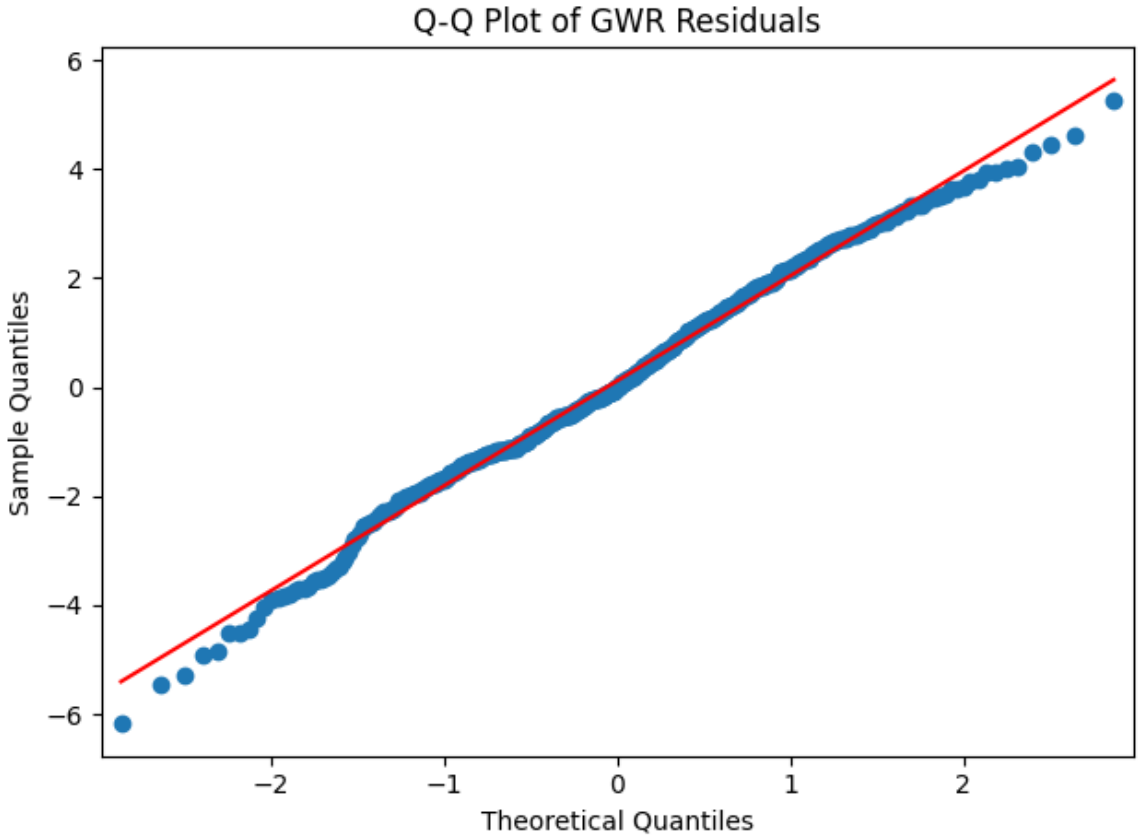
In [304…
```
# add results
DAI_V['gwr_beta'] = gwr_results.params[:, 1]
DAI_V['gwr_intercept'] = gwr_results.params[:, 0]
DAI_V['gwr_R2'] = gwr_results.localR2
```

In [305…
```
# plot GWR beta
fig, ax = plt.subplots(figsize=(10, 8))

DAI_V.plot(ax=ax, column='gwr_beta', cmap='RdBu', linewidth=0.2, ed


cx.add_basemap(ax, crs=DAI_V.crs, source=cx.providers.OpenStreetMap

ax.set_title('GWR Coefficients: DAI as Independent, and IMD Decile
ax.axis('off')
ax.add_artist(ScaleBar(dx=1, location='lower right'))

plt.tight_layout()
plt.show()
```



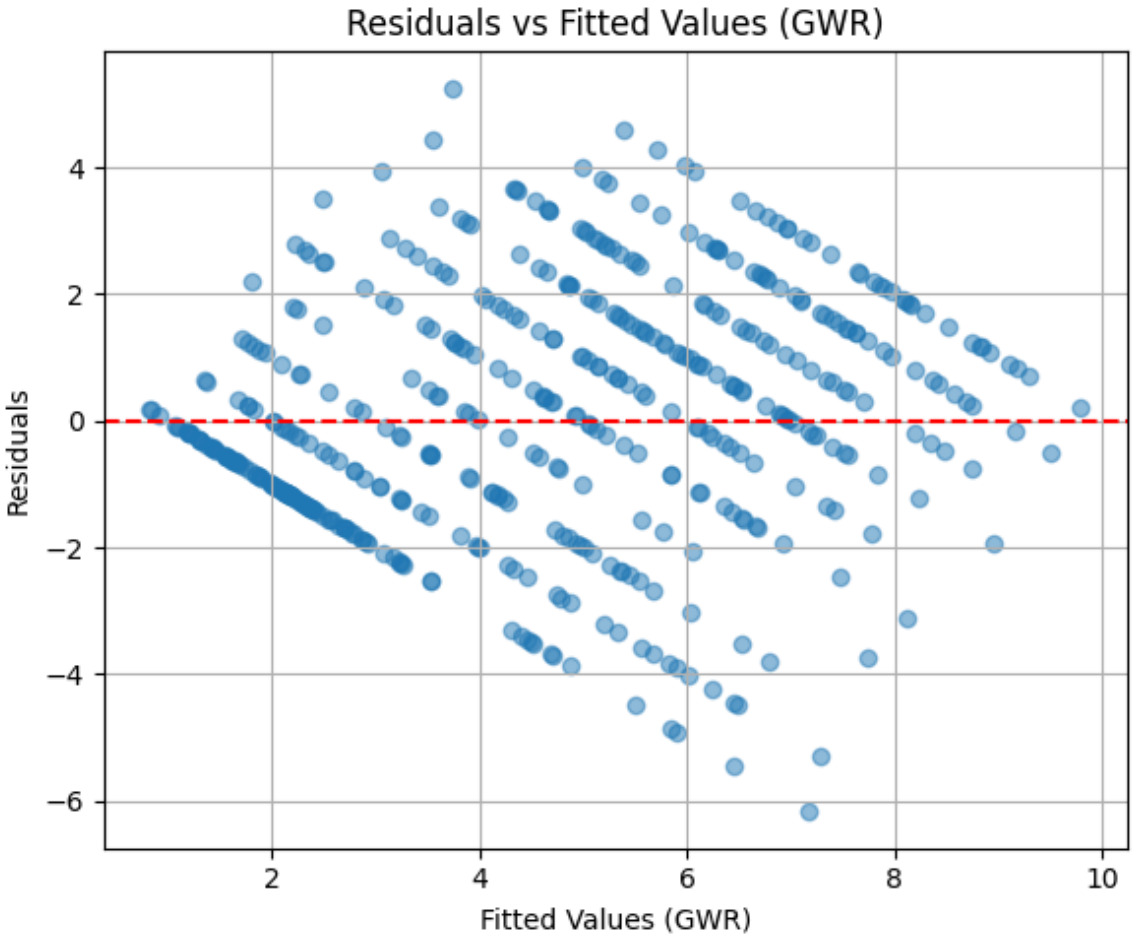
Map shows the spatial variation in GWR coefficients across Leeds, where blue areas indicate a positive local relationship. This means that higher accessibility (DAI) is associated with lower deprivation. In contrast, red areas reflect a negative relationship. In these areas, unexpectedly, higher accessibility to daily amenities is associated with higher deprivation levels.

In [ ]:
```
# Q-Q plot, GWR residuals
sm.qqplot(gwr_results.resid_response, line='s')
plt.title('Q-Q Plot of GWR Residuals')
plt.tight_layout()
plt.show()
```



The Q-Q plot shows that the GWR residuals mostly follow a normal pattern, though there are some small deviations at the ends.

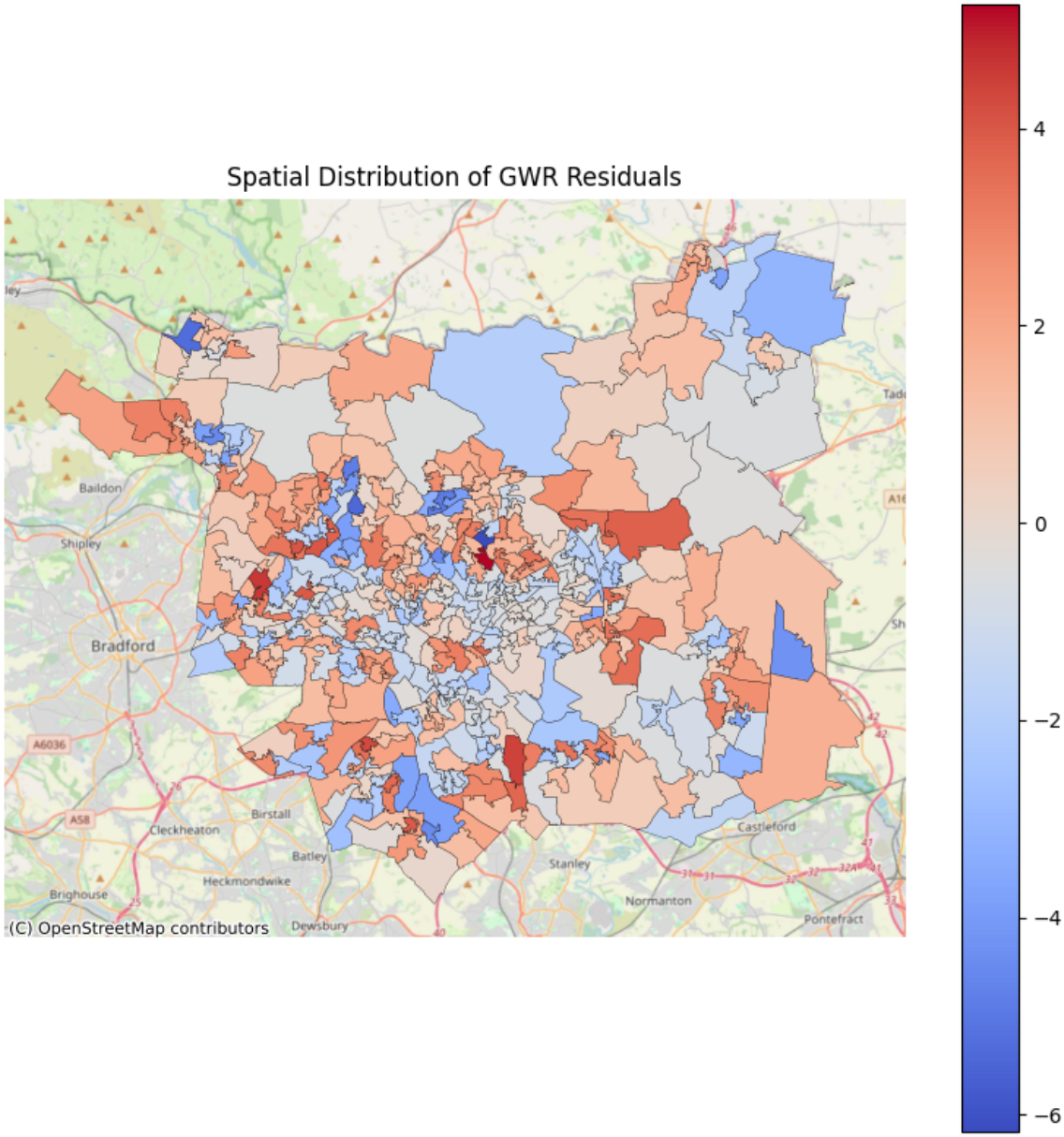
In [ ]:
```
plt.figure(figsize=(6, 5))
plt.scatter(gwr_results.predy, gwr_results.resid_response, alpha=0.
plt.axhline(0, color='red', linestyle='--')
plt.xlabel('Fitted Values (GWR)')
plt.ylabel('Residuals')
plt.title('Residuals vs Fitted Values (GWR)')
plt.grid(True)
plt.tight_layout()
plt.show()
```



The residuals seem mostly spread out, but there's a bit of a pattern. It looks like the model might not fit equally well everywhere.

In [366…

```
DAI_Inputs['GWR_residuals'] = gwr_results.resid_response

fig, ax = plt.subplots(figsize=(8, 8))
DAI_Inputs.plot(ax=ax, column='GWR_residuals', cmap='coolwarm',lege

cx.add_basemap(ax, crs=DAI_Inputs.crs, source=cx.providers.OpenStre
ax.set_title('Spatial Distribution of GWR Residuals')
ax.axis('off')

plt.tight_layout()
plt.show()
```



The map shows where the model underestimates or overestimates. Some unclear patterns specifically in rings from the centre to the peripheral areas stand out, so I will look more closely on LISA clustering results:

Moran's I

In [328…
```
# Queen weights from dataframe
w_queen = Queen.from_dataframe(DAI_V, use_index=True)
```

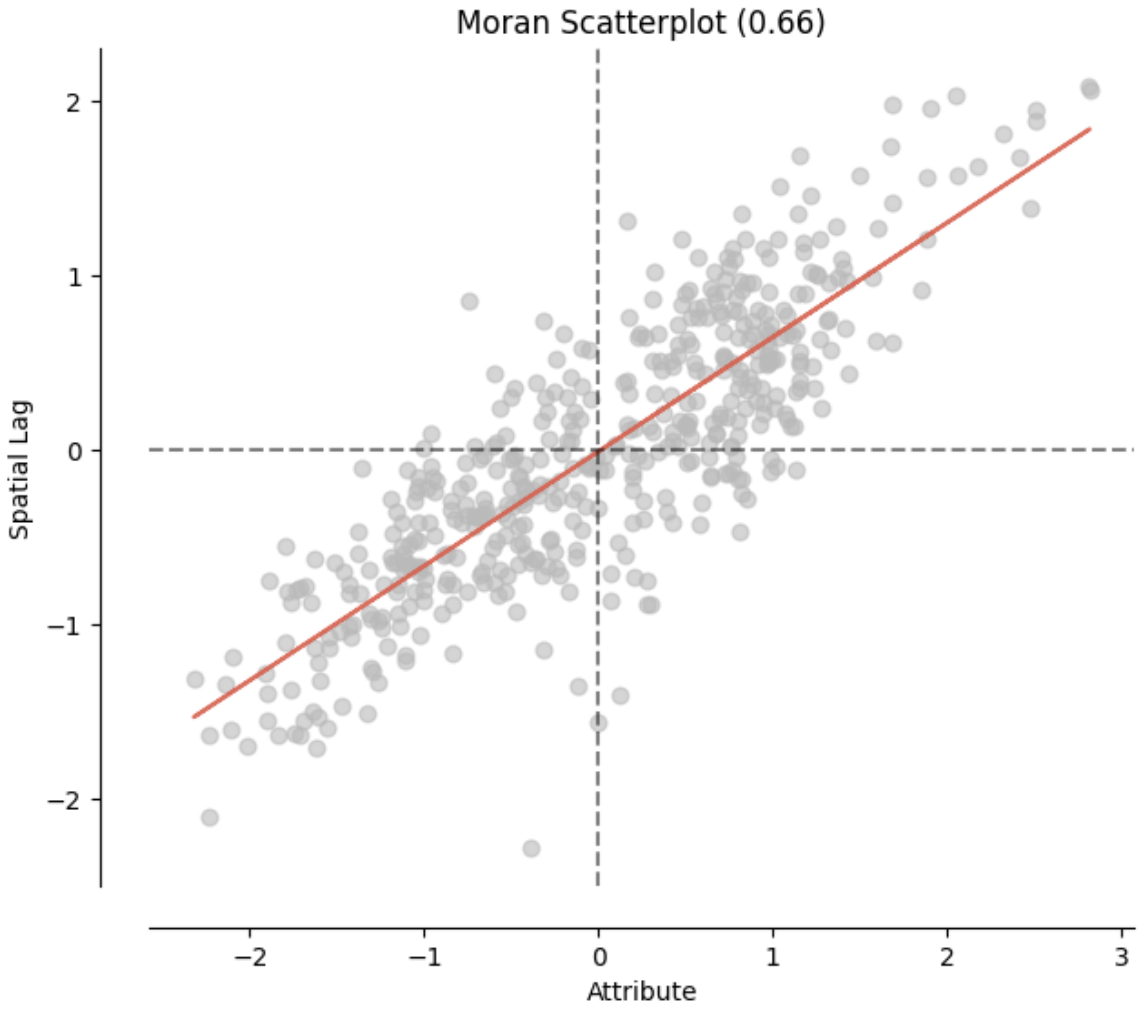
DAI

In [329…
```
#  Moran's I on the accessibility score column
mi = Moran(DAI_V['DAI'], w_queen)

# Print results
print(mi.I)
print(mi.p_sim)
```

```
0.6562051367152769
0.001
```

In [330…
```
moran_scatterplot(mi) #create the scatterplot
```

Out[330…
```
(<Figure size 700x700 with 1 Axes>,
 <Axes: title={'center': 'Moran Scatterplot (0.66)'}, xlabel='Attr
ibute', ylabel='Spatial Lag'>)
```



The Moran's I values and the relavant scatterplot suggests that areas with high accessibility are clustered together, and low-accessibility areas are also clustered together.

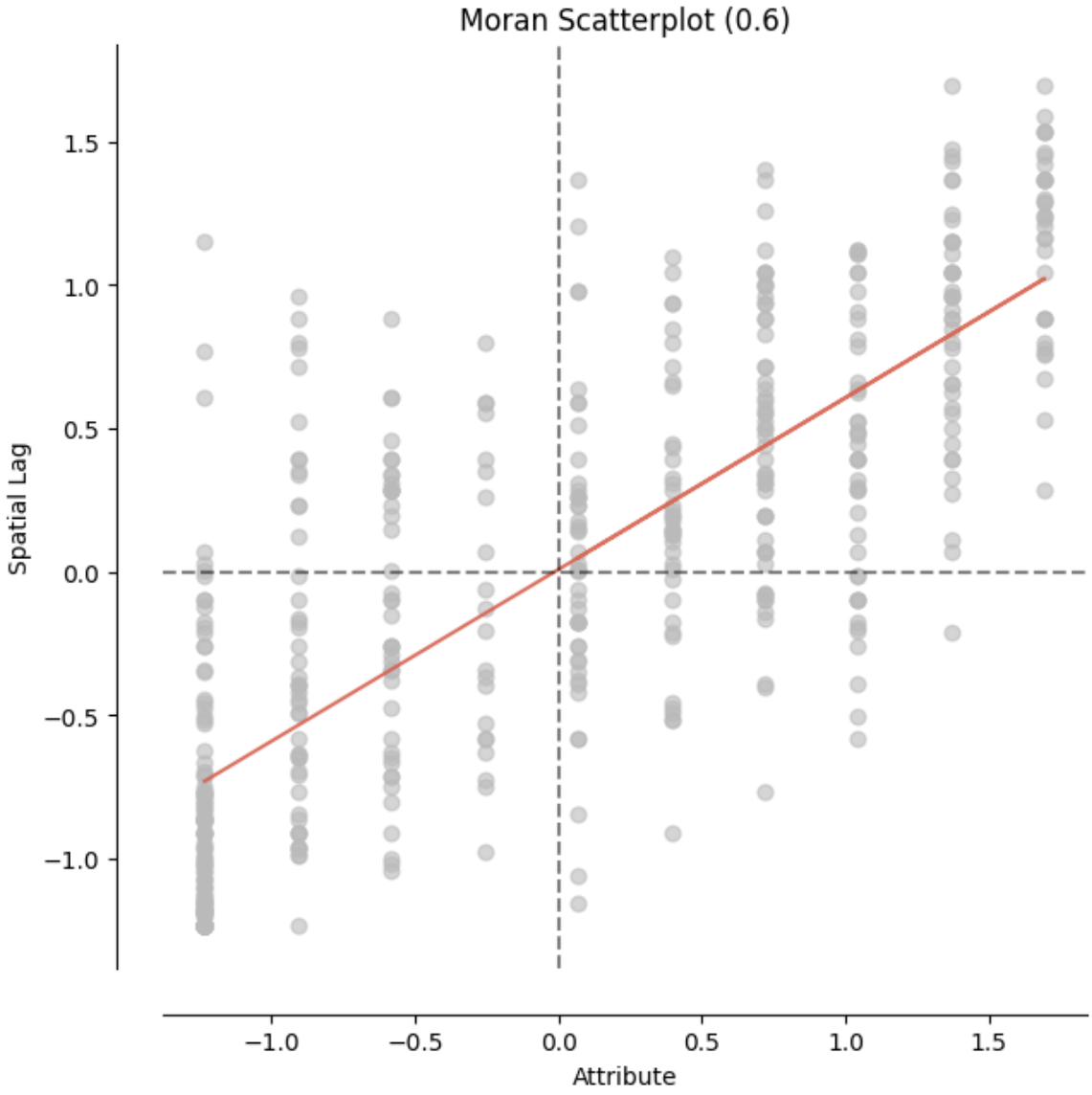
IMD

In [331…
```
#  Moran's I on the accessibility score column
mi = Moran(DAI_V['Index of Multiple Deprivation (IMD) Decile'], w_q

# Print results
print(mi.I)
print(mi.p_sim)
```

```
0.5984814768235366
0.001
```

In [332…
```
moran_scatterplot(mi) #create the scatterplot
```

Out[332… (<Figure size 700x700 with 1 Axes>,
<Axes: title={'center': 'Moran Scatterplot (0.6)'}, xlabel='Attribute', ylabel='Spatial Lag'>)



Results shows a moderate positive spatial autocorrelation means areas with high deprivation are generally surrounded by other highly deprived areas and same story about the lower deprived areas.

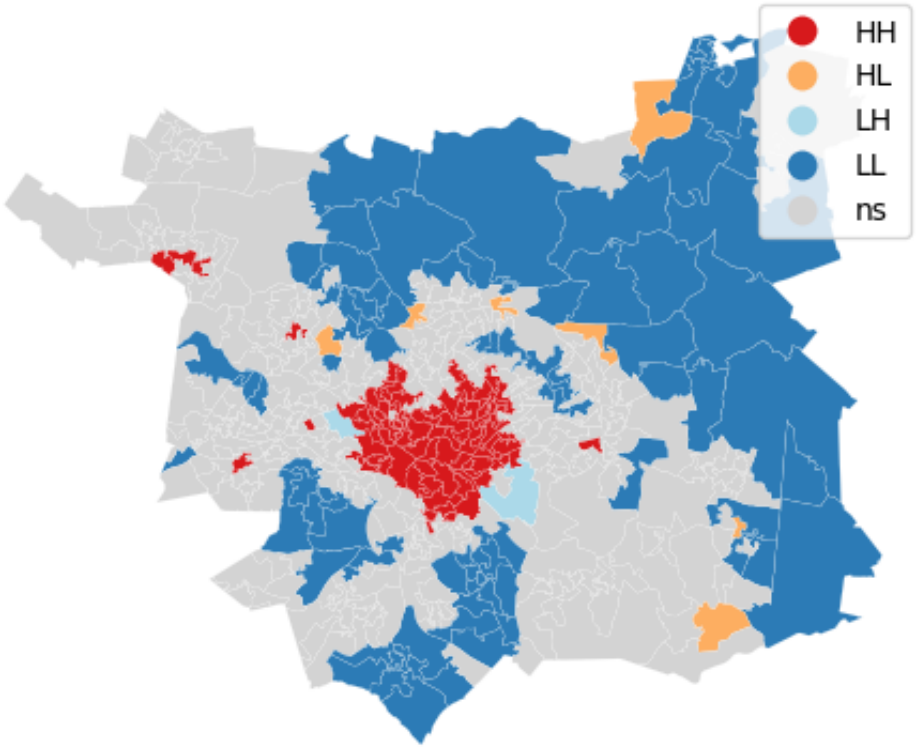
LISA

DAI

In [333…
```
# LISA
dai_lisa = esda.Moran_Local(DAI_V['DAI'], w_queen)
```

In [334…
```
lisa_cluster(dai_lisa, DAI_V)
```

Out[334… (<Figure size 640x480 with 1 Axes>, <Axes: >)



High-High (HH) areas with high accessibility surrounded by similarly high areas—are concentrated in central Leeds, while Low-Low (LL) clusters are in peripheral zones, highlighting spatial inequalities in access to daily amenities.

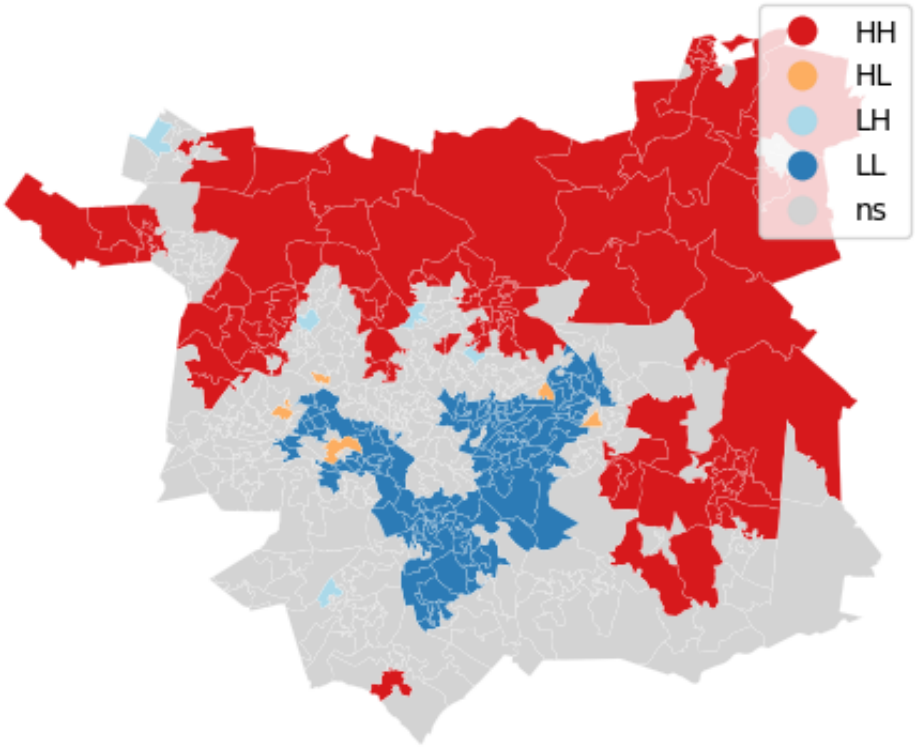
In [335…
```
# Classify LISA clusters for DAI
DAI_Inputs['dai_lisa_cluster'] = 'ns'
DAI_Inputs.loc[(dai_lisa.q == 1) & (dai_lisa.p_sim < 0.05), 'dai_li
DAI_Inputs.loc[(dai_lisa.q == 2) & (dai_lisa.p_sim < 0.05), 'dai_li
DAI_Inputs.loc[(dai_lisa.q == 3) & (dai_lisa.p_sim < 0.05), 'dai_li
DAI_Inputs.loc[(dai_lisa.q == 4) & (dai_lisa.p_sim < 0.05), 'dai_li
```

IMD

In [336…
```
# LISA
imd_lisa = esda.Moran_Local(DAI_V['Index of Multiple Deprivation (I
```

In [337…
```
lisa_cluster(imd_lisa, DAI_V)
```

Out[337… (<Figure size 640x480 with 1 Axes>, <Axes: >)



Red areas (H-H clusters) represent zones with low deprivation surrounded by similarly low-deprivation neighbours, concentrated mostly in the north and outer suburbs. While highly deprived areas surrounded by other high deprived ones (L-L) are primarily concentrated in central and south Leeds. This pattern highlights persistent spatial inequalities across the city.

In [347…
```
# Classify LISA clusters for IMD
DAI_Inputs['imd_lisa_cluster'] = 'ns'
DAI_Inputs.loc[(imd_lisa.q == 1) & (imd_lisa.p_sim < 0.05), 'imd_li
DAI_Inputs.loc[(imd_lisa.q == 2) & (imd_lisa.p_sim < 0.05), 'imd_li
DAI_Inputs.loc[(imd_lisa.q == 3) & (imd_lisa.p_sim < 0.05), 'imd_li
DAI_Inputs.loc[(imd_lisa.q == 4) & (imd_lisa.p_sim < 0.05), 'imd_li
```

Overlayed LISA

In [357…
```
# merge both clusters into a single column
DAI_Inputs['combined_lisa'] = DAI_Inputs['dai_lisa_cluster'] + '-'
```

In [358…
```
DAI_Inputs['combined_lisa'].value_counts()
```

Out[358…
```
combined_lisa
ns-ns    180
LL-LL    107
HH-HH     85
HH-ns     48
LL-ns     47
HL-HL      5
LH-LH      5
HL-ns      4
Name: count, dtype: int64
```

Only choosing the meaningful clusters and treat the others like ns.

In [ ]:
```
# custom colors for some key combinations
highlighted = {
    'LL-LL': '#d7191c',       # Low Access – High Deprivation
    'HH-HH': '#fdae61',       # High Access – Low Deprivation
    'HL-HL': '#2c7bb6',       # High Access – High Deprivation
    'LH-LH': '#abd9e9',       # Low Access – Low Deprivation
    'ns-ns': 'lightgrey',     # Not significant
}
# set default colors for not celected combinations
DAI_Inputs['combined_color'] = DAI_Inputs['combined_lisa'].map(high
```

In [365…
```
# Plot
legend_labels = {
    'LL-LL': 'Low Access – High Deprivation',
    'HH-HH': 'High Access – Low Deprivation',
    'HL-HL': 'High Access – High Deprivation',
    'LH-LH': 'Low Access – Low Deprivation',
    'ns-ns': 'Other Combinations'
}

fig, ax = plt.subplots(figsize=(10, 10))
DAI_Inputs.plot(ax=ax, color=DAI_Inputs['combined_color'], linewidt

cx.add_basemap(ax, crs=DAI_Inputs.crs, source=cx.providers.OpenStre
ax.set_title('Overlay of Daily Accessibility Index (DAI) and Index
ax.add_artist(ScaleBar(dx=1, location='lower right'))
ax.axis('off')

# Legend
legend_elements = [
    mpatches.Patch(color=highlighted[key], label=legend_labels[key]
]
ax.legend(handles=legend_elements, loc='lower left')

plt.tight_layout()
plt.show()
```

Overlay of Daily Accessibility Index (DAI) and Index of Multiple Deprivation (IMD) LISA Clusters